Mathematical Methods

B. Grey Vandeberg

3 Parables of Polynomials

In the previous chapter, we spent a considerable amount of time developing Fourier series partly in the hope that, by the end, their existence and convergence properties felt not as if they were handed from on high, but instead emerge as natural consequences of a function's *underlying structure*. What we can learn from the Master theorem is that there isn't *a single unifying theory* of which functions are "Fourier-anenable" — indeed, a function seems to *accumulate* desirable Fourier-analytic properties as it is endowed with greater and greater regularity, but that regularity comes in a variety of different flavors: absolute integrability L^1, square-integrability L^2, k-times continuous differentiability C^k, bounded variation BV, Dini conditions to establish regularity at particular points in the domain, etc.

So, taking a step back, what is the *common theme* among these conditions? Indeed, each characterizes a very particular kind of regularity on behalf of a function f, but what we can understand is that the conditions elucidate something about how *cooperative* f is as a target for Fourier approximation. What we understand about the procedure from a bird's-eye perspective is: given a function $f\colon [-\pi,\pi] \to \mathbb{R}$, we want to approximate $f(x)$ pointwise for $x \in [-\ell,\ell]$ without ever "plugging in $x \mapsto f(x)$ explicitly." To this end, we take the trigonometric functions

$$\mathcal{T} = \{1,\ \cos(t),\ \sin(t),\ \cos(2t),\ \sin(2t),\ \dots\} = \{1,\ \cos(nt),\ \sin(nt)\}_{n=1}^{\infty},$$

and we approximate $f(x)$ by a **linear combination** of members of $\mathcal{T}$ evaluated at $t = x$:

$$S_N f(x) = \frac{a_0}{2} + \sum_{n=1}^{N} [a_n \cos(nx) + b_n \sin(nx)], \qquad x \in [-\pi,\pi],$$

where the Fourier coefficients are

$$a_0 = \frac{1}{\pi}\int_{-\pi}^{\pi} f(t)\,dt, \quad a_n = \frac{1}{\pi}\int_{-\pi}^{\pi} f(t)\cos(nt)\,dt, \quad b_n = \frac{1}{\pi}\int_{-\pi}^{\pi} f(t)\sin(nt)\,dt.$$

In the limit as $N \to \infty$, the partial sums $S_N f \to \mathcal{S}f$, and $\mathcal{S}f(x) = f(x)$ pointwise for points of continuity $x \in [-\pi,\pi]$ — or to the midpoint of the left- and right-sided limits $\mathcal{S}f(x) = \frac{f(x^+)+f(x^-)}{2}$ at points of discontinuity — so long as the target function f was "sufficiently regular." What we mean by sufficiently regular, of course, came down to what question we're wanting to answer, and this is what led us to characterize properties of the Fourier expansion by function class (as in the master theorem).

Now, however, we want to turn our attention to the other half of the assumption: why did we imagine this approximation of functions $f\colon [-\pi,\pi] \to \mathbb{R}$ by members of $\mathcal{T}$ *could* be appropriate in the first place? We've alluded to the answer in passing, but here, it will prove central to our development and, hopefully, will elucidate just how powerful an *appropriately chosen basis* is for productive analysis.

3.1 The Practitioner's Parables

From Fourier to Regression: An Overview

We begin our examination here, which will ultimately spann this and the following chapter, within the same well-trodden territory of Fourier series. Immediately, we want to direct our attention toward a luxury we often take for granted in analysis: namely, in constructing the Fourier representation $\mathcal{S}f$, we've assumed thus far that the *entire function* $f\colon [-\pi,\pi] \to \mathbb{R}$ is accessible to us.

Concretely, let us stop to consider how we *really* compute the integral defining the constant Fourier coefficient,

$$a_0 = \frac{1}{\pi}\int_{-\pi}^{\pi} f(t)\,dt.$$

What we're tempted to believe is that "an integral" unilaterally directs us to apply the Fundamental Theorem of Calculus:

$$\int_{-\pi}^{\pi} f(t)\,dt \stackrel{?}{=} F(\pi) - F(-\pi) \quad \text{for some antiderivative } F \text{ satisfying } F'(t) = f(t),\ t \in [-\pi,\pi].$$

However, what's clear is that when we strip away all of the ambient assumptions on the functional form of f, what remains in the notation $\int_{-\pi}^{\pi} f(t)\,dt$ is a much more concrete directive: "form the Riemann sums $\sum_{\mathcal{P}} f(c_i)\Delta t_i$ over finer and finer partitions $\mathcal{P} = \{-\pi = t_0, t_1, \dots, t_k = \pi\}$ of $[-\pi,\pi]$, and verify that the sums stabilize" — where, to be clear, we require a very particular form of stability for the integral to make sense. In particular, the Riemann integral demands that the sums converge to the same value over *every* choice of partition $\mathcal{P}$ and over *every* choice of sample points c_i within each subinterval $[t_{i-1}, t_i]$ — only then is the notation $\int f(t)\,dt$ well-defined.

What an integral quietly presupposes, then, is that we possess an *infinite observational budget* for querying f. That is, in order to integrate a function *absent* sufficient regularity (or absent our ability to verify its presence), we're effectively being directed to query f at arbitrary points $c_i \in [t_{i-1}, t_i]$, where the bin-widths $\Delta t_i = t_i - t_{i-1} \to 0$ as we let the mesh of our partition $\|\mathcal{P}\| \to 0$.

What we understand is that, in "theoretical" settings, the functions of interest are of a *known form*. We're *given* $f(t) = t^2$ for $t \in [-\pi,\pi]$, at which point we can ascertain the function's behavior directly — we can derive $f'(t) = 2t$ at every $t \in [-\pi,\pi]$, that $f''(t) = 2$ a constant for every $t \in [-\pi,\pi]$, that all higher order derivatives $f^{(k)}(t), k \geq 3$ vanish on $[-\pi,\pi]$, and so on. Most importantly, once we possess such an exact functional form, the observational budget *is* infinite, because the formula *is* the oracle: hand me any $t \in [-\pi,\pi]$ and I hand you back $f(t) = t^2$. There is no gap between "what f does" and "what we know about f."

But, in practice, we almost never know the form of a function of interest — in this light, let us *flip* the perspective and see what falls out. Concretely, suppose that we do not possess the functional form of f — instead, what we have is a finite sample

$$\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}.$$

Here, we suppose that the **data** $\mathcal{D}$ has been sorted in terms of the $x_i \in [-\pi, \pi]$ — i.e., $-\pi \leq x_1 < x_2 < \cdots < x_n \leq \pi$ — and that y_i is the "evaluation of interest" at location x_i. Perhaps we sampled a signal (y) at regular time intervals (x); perhaps we measured temperature (y) at n points along a rod (x); perhaps we recorded a patient's response (y) at each of n clinic visits indexed by date (x). In each case, we *believe* there is some underlying process f generating the measurements y_i according to the corresponding "location" x_i, but that the exact form of the **data generating process** (DGP) — i.e., the functional form of f — eludes us.

Notice what we *do* have, though. The ordered sample points $x_1 < x_2 < \cdots < x_n$ carve $[-\pi, \pi]$ into subintervals $[x_{i-1}, x_i]$ — they are, in effect, a bona fide partition of $[-\pi, \pi]$ (more precisely, of $[x_1, x_n]$ where x_1 is close to $-\pi$ and x_n is close to π). At each sample point x_i, we observe a sample on the associated y_i. Supposing, then, that we agree to imagine that the y_i are a result of such a functional relationship with f — suppose, for instance, that $y_i = f(x_i)$ *exactly* for some function $f\colon [-\pi, \pi] \to \mathbb{R}$ — then the sum

$$\sum_{i=1}^{n} y_i \Delta x_i := \sum_{i=1}^{n} f(x_i)\Delta x_i, \qquad \Delta x_i := x_i - x_{i-1},$$

is a *perfectly legitimate* Riemann sum when the partition $\mathcal{P}$ is taken to be the x-coordinates in our given data.

What is different in this situation, however, is that we're *stuck* with the partition our data provides. The mesh $\|\mathcal{P}\| = \max_i \Delta x_i$ is whatever it happens to be — we cannot send it to zero. Even if we observed perfectly even bin widths $\Delta x_i = 2\pi/n$ for every i, we would *still be stuck* with the given partition, unable to refine the Riemann sum $\sum_{i=1}^{n} f(x_i)\Delta x_i$ without collecting more data.

Moreover, we've quietly been supposing the best possible case: that $y_i = f(x_i)$ *exactly*. In practice, this is almost never true. What we *actually* observe, should it really be the case that a DGP generates y_i as a function of the x_i, are the *noisy* observations $y_i \approx f(x_i)$. In practice, we specify such a DGP by supposing

$$y_i = f(x_i) + \epsilon_i, \qquad i = 1, \ldots, n,$$

where the **errors** $\epsilon_i = y_i - f(x_i)$ serve as a catch-all bucket absorbing everything our model cannot capture — we might imagine the errors accumulate from, among other sources, measurement error, environmental variation, the influence of other unobserved variables driving the DGP, or simply the inadequacy of whatever form we have assumed for f.

This is the setting we'll inhabit for the remainder of this chapter and, in various guises, for much of this book: we have n noisy observations from a process we don't fully understand, and we want to recover as much of the underlying structure as we can. What changes, of course, is how we *discern* this structure. What is perhaps most surprising, then, is how we're able to cast such a seemingly general class of problems in a *canonical framework*, all while greatly simplifying our bookkeeping.

Definition 3.1.1 (The Linear Model (V1))

Suppose we observe the **data**

$$\mathcal{D} = \{(x_1, y_1), \ldots, x_n, y_n)\},$$

where $n \in \mathbb{N}$ is a known **sample size**, and where we suppose that the relationship between **outcome** y_i and **predictor** x_i pairs, (x_i, y_i), is "well-described" by a **linear combination** of

$p = k + 1$ known functions $\phi_0, \phi_1, \dots, \phi_k$ of the predictor x; i.e., that

$$y_i = \beta_0\phi_0(x_i) + \beta_1\phi_1(x_i) + \cdots + \beta_k\phi_k(x_i) + \epsilon_i, \qquad i = 1, \dots, n,$$

for some $\beta_0, \beta_1, \dots, \beta_k \in \mathbb{R}$.

The p functions $\phi_0, \phi_1, \dots, \phi_k$ are called the **basis functions** (or **regressors**; here, we're imagining that "the same predictor is being reused in various functional forms across the model").

The scalars $\beta_0, \beta_1, \dots, \beta_k$ are the **coefficients** (or **model parameters**) which are typically the unknown quantities of interest in a "regression problem."

The ϵ_i are perceived to be **random errors**, comprising "everything the model can't (or won't) explain" after accounting for the functions ϕ_j of our predictor x. (Occasionally, we perceive these errors to be **normally distributed** — but we're quick to clarify that this is certainly *not* a requirement for the linear model to make sense.

If we assemble everything in **matrix notation**:

$$\boldsymbol{\Phi} = \underbrace{\begin{pmatrix} \phi_0(x_1) & \phi_1(x_1) & \cdots & \phi_k(x_1) \\ \phi_0(x_2) & \phi_1(x_2) & \cdots & \phi_k(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_0(x_n) & \phi_1(x_n) & \cdots & \phi_k(x_n) \end{pmatrix}}_{p=k+1 \text{ columns}} \in \mathbb{R}^{n \times p},$$

$$\mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} \in \mathbb{R}^n, \qquad \boldsymbol{\beta} = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_k \end{pmatrix} \in \mathbb{R}^p, \qquad \boldsymbol{\epsilon} = \begin{pmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{pmatrix} \in \mathbb{R}^n,$$

then we can equivalently write the system of n equations defining the linear model as

$$\mathbf{y} = \boldsymbol{\Phi}\boldsymbol{\beta} + \boldsymbol{\epsilon}.$$

We call $\mathbf{y}$ the **outcome vector**, $\boldsymbol{\Phi}$ the **design matrix** (or **model matrix**), $\boldsymbol{\beta}$ the **coefficient vector** (or **parameter vector**), and $\boldsymbol{\epsilon}$ the **error vector**.

Remark.

The distinction that p, the *number of model parameters*, equals $k+1$, is something of a customary choice of notation. What we'll find is that, most often, the **constant term** $\beta_0 \in \boldsymbol{\beta}$ is endowed with some "distinguishing properties" that warrant its special treatment. In particular, most often, the linear model specifies $\phi_0 = \mathbf{1}$ the constant function equal to 1 for any x, so that the leading column in $\boldsymbol{\Phi}$ is just a vector of 1s. In this way, the $\beta_0\phi_0(x_i)$ term is usually just $\beta_0 \cdot 1 = \beta_0$ for each $i = 1, \dots, n$. Combined with the rest of the predictors, what we imagine the model doing is "estimating the baseline mean β_0 across the full vector $\mathbf{y}$, and then modeling *conditional deviations* from the global mean of the y's given the ith participant's predictor was observed to be $x = x_i$.

In essence, we call $p = k+1$ so that p can denote "the number of $p$arameters we need to estimate, *including* the baseline mean," while k denotes "the number of relationships we've specified as driving conditional deviations *away from* the baseline mean."

Almost immediately, one raises a qualm: "what if the relationship is *really nonlinear*?" The answer is that it almost certainly *is* nonlinear — but the linear model accomodates this just fine. The word "linear" in "linear model" does *not* refer to the relationship between y and x — it refers to the relationship between y and the *coefficients* β_j. Imagine the situation when we want to model a vector of $n = 3$ outcomes y_i as conditional deviations from a global mean driven by some observed predictor x_i and its associated quadratic x_i^2 (where, again, we assume that $x_1 < x_2 < x_3$ are ordered and distinct). Then, in the notation above, we'd have

$$\phi_0(x_i) = 1, \quad \phi_1(x_i) = x_i, \quad \phi_2(x_i) = x_i^2, \qquad i = 1, 2, 3,$$

and we'd write the model whose system of equations is described by

$$\begin{aligned} y_i &= \phi_0(x_i)\beta_0 + \phi_1(x_i)\beta_1 + \phi_2(x_i)\beta_2 + \epsilon_i \\ &= 1 \cdot \beta_0 + x_i \cdot \beta_1 + x_i^2 \cdot \beta_2 + \epsilon_i \\ &= \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \epsilon_i, \qquad\qquad i = 1, 2, 3, \end{aligned}$$

as

$$\underbrace{\begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix}}_{\mathbf{y}} = \underbrace{\begin{pmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \end{pmatrix}}_{\mathbf{\Phi}} \underbrace{\begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{pmatrix}}_{\boldsymbol{\beta}} + \underbrace{\begin{pmatrix} \epsilon_1 \\ \epsilon_2 \\ \epsilon_3 \end{pmatrix}}_{\boldsymbol{\epsilon}}.$$

What we understand, then, is that the relationship between y and x as specified is *certainly nonlinear* — we've explicitly modeled y_i as a function including a quadratic term, x_i^2. What makes it a "linear model" is that, once we imagine (x_1, x_2, x_3) as "a list of numbers," then so too is (x_1^2, x_2^2, x_3^2) "just a list of numbers." How we've *obtained* these numbers is quite immaterial to the model itself — it doesn't really care whether you imagine x_i^2 as "deriving from x_i" or as "a completely new number" (at least, not on its face — we will see that it *really does* care, it just won't tell you so.)

An Initial Model Fit

What makes this immateriality concrete is how we proceed to *estimate* the unknown quantities in the model. To start, suppose we're working in the *noiseless* case, so that $\epsilon_1 = \epsilon_2 = \epsilon_3 = 0$ (equivalently, $\boldsymbol{\epsilon} = \mathbf{0}$ — we use the symbol $\mathbf{0}$ to denote a vector of zeros in $\mathbb{R}^3$.) When $\boldsymbol{\epsilon} = \mathbf{0}$, the system in the linear model becomes

$$\mathbf{y} = \mathbf{\Phi}\boldsymbol{\beta}.$$

This is not yet a *statistical* problem — it's an *algebraic* one. Concretely, realize that if we write the noiseless system out as

$$\begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \beta_0 \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} + \beta_1 \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} + \beta_2 \begin{pmatrix} x_1^2 \\ x_2^2 \\ x_3^2 \end{pmatrix},$$

then the question is stated in terms of whether there exists a vector $\boldsymbol{\beta} \in \mathbb{R}^3$ such that $\mathbf{y}$ can be expressed as a **linear combination** of the columns of $\mathbf{\Phi}$. The questions, then, are twofold: whether such a solution vector β *exists* and, if it does, whether it can be *computed*. What we understand is that verifying existence explicitly at the beginning is almost always faster than "letting unsolvability emerge as an artifact of a solution-finding algorithm."

For instance, suppose that we'd only sampled $n = 2$ observations (instead of $n = 3$). Then, the system of equations could be written explicitly as

$$y_1 = \beta_0 + \beta_1 x_1 + \beta_2 x_1^2,$$
$$y_2 = \beta_0 + \beta_1 x_2 + \beta_2 x_2^2.$$

Suppose we'd found a solution vector $\beta^\star$. Then, as a consequence of the *linearity of the model*, it would follow that if we subtract the equations above and plug in the solution, we would have

$$y_1 - y_2 = \beta_1^\star(x_1 - x_2) + \beta_2^\star(x_1^2 - x_2^2) = (\beta_1^\star + \beta_2^\star(x_1 + x_2))(x_1 - x_2).$$

In particular, divide both sides by $x_1 - x_2$ to form the **first-order divided difference**, given by

$$\frac{y_1 - y_2}{x_1 - x_2} = \beta_1^\star + \beta_2^\star(x_1 + x_2).$$

To see how this looks in action, suppose that we'd observed the values $(x_1, y_1) = (0, 2)$ and $(x_2, y_2) = (1, 3)$. We compute

$$\frac{y_1 - y_2}{x_1 - x_2} = 1, \quad x_1 + x_2 = 1,$$

so that the first-order divided difference condition reduces to

$$\beta_1^\star + \beta_2^\star = 1.$$

This is one equation in two unknowns — in particular, what we observe upon rearrangment is that

$$\beta_1^\star = 1 - \beta_2^\star.$$

Read this to mean: For any choice of $\beta_2^\star$, we can set $\beta_1^\star = 1 - \beta_2^\star$, and then recover $\beta_0^\star$ from either of the original equations — say the first:

$$\beta_0^\star = y_1 - \beta_1^\star x_1 - \beta_2^\star x_1^2 = 2 - (1 - \beta_2^\star)(0) - \beta_2^\star(0)^2 = 2.$$

What we can say, then, is that when we observe the data

$$\mathcal{D} = \{(0, 2), (1, 3)\},$$

the full family of solutions to the noiseless linear system

$$y_1 = \beta_0 + \beta_1 x_1 + \beta_2 x_1^2,$$
$$y_2 = \beta_0 + \beta_1 x_2 + \beta_2 x_2^2,$$

is given by

$$(\beta_0^\star,\ \beta_1^\star,\ \beta_2^\star) = (2,\ 1 - \beta_2^\star,\ \beta_2^\star), \qquad \beta_2^\star \in \mathbb{R}.$$

Let us take a moment to *intuitively* restate the model's reasoning as reflected by the derivations above. First of all, we observe that the original parameterization of the β vector was specified in terms of three distinct components: an intercept β_0, a linear-term slope β_1, and a quadratic-term slope β_2. Nevertheless, in our *solution* $\beta^\star$ based on the observed data $\mathcal{D}$, we recognize that only *one* parameter varies independently — namely, the quadratic slope $\beta_2^\star \in \mathbb{R}$. In particular, once we set *any* $\beta_2^\star \in \mathbb{R}$, we're instructed to take as the solution's linear slope $\beta_1^\star = 1 - \beta_2^\star$, and for any such pair of slopes $(\beta_1^\star = 1 - \beta_2^\star, \beta_2^\star)$, taking our intercept to be $\beta_0^\star = 2$ yields a solution vector

$(\beta_0^\star, \beta_1^\star, \beta_2^\star) = (2, 1 - \beta_2^\star, \beta_2^\star)$ solving $y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2$, $i = 1, 2$ under our observed data $\mathcal{D}$: in particular, plugging in $(x_1, y_1) = (0, 2)$ and $(x_2, y_2) = (1, 3)$, this says that the solution vector $\beta^\star$ described above satisfies

$$\begin{aligned} 2 &= \beta_0^\star + \beta_1^\star(0) + \beta_2^\star(0)^2 = \beta_0^\star, \\ 3 &= \beta_0^\star + \beta_1^\star(1) + \beta_2^\star(1)^2 = \beta_0^\star + \beta_1^\star + \beta_2^\star. \end{aligned}$$

And, of course, this is reasonable in light of the given *noiseless model* — the noiseless model instructs us to take our observed data $\mathcal{D}$ as *gospel*. What the model is literally telling us is that "if $\beta^\star$ solves the system and you've observed a sample point $x_i = 0$ in $\mathcal{D}$, then according to the noiseless specification, I'm *forced* to believe its corresponding y_i characterizes the truth. Since you've told me that $y_i = 2$ at $x_i = 0$, and since you've not given me any reason to doubt the exactness of this correspondence, I'll *force* the solution to pass through $(x, y) = (0, 2)$ no matter what by setting the y-intercept of any solution accordingly." And indeed, $\beta_0^\star$ here *really is* "just the y-intercept" — i.e., when $x_i = 0$, the model predicts $y_i = \beta_0^\star$ exactly.

From here, what we understand is that, once the model sets $\beta_0^\star = 2$ in order for the quadratic $y = \beta_0^\star + \beta_1^\star x + \beta_2^\star x^2$ to pass through the point $(x, y) = (0, 2)$, it treats setting $(\beta_1^\star, \beta_2^\star)$ like a balancing act: if $\beta_2^\star = 1$, then we obtain $(\beta_0^\star, \beta_1^\star, \beta_2^\star) = (2, 0, 1)$, corresponding to the quadratic model

$$f(x) = x^2 + 2.$$

At the point $x = 0$, the curve *predicts* $f(x) = 0^2 + 2 = 2$, verifying that the observed sample point $(x, y) = (0, 2) \in \mathcal{D}$ falls on the curve. Similarly, at $x = 1$, the model predicts $f(x) = 1^2 + 2 = 3$, so that the other sample point $(x, y) = (1, 3) \in \mathcal{D}$ also falls on the curve.

But, in the above, we set $\beta_1^\star = 1 - \beta_2^\star$ according to the constraint $\beta_1^\star + \beta_2^\star = 1$, but this wasn't our only recourse — indeed, we can certainly ask what happens if we set $\beta_2^\star = 1 - \beta_1^\star$ and set the linear coefficient $\beta_1^\star = 1$? Then the quadratic coefficient $\beta_2^\star = 1 - \beta_1^\star = 0$ vanishes, we obtain a solution $(\beta_0^\star, \beta_1^\star, \beta_2^\star) = (2, 1, 0)$, and we obtain the linear model

$$f(x) = x + 2.$$

Again, one can verify the model conforms with the two points in our given data $\mathcal{D}$, as do, for instance,

$$f(x) = \frac{1}{2}x^2 + \frac{1}{2}x + 2; \qquad f(x) = 100x^2 - 99x + 2; \qquad f(x) = -\pi x^2 + (1 + \pi)x + 2.$$

Again, *all* of these curves agree with the data $\mathcal{D}$; what changes is how they behave *away* from our observed data. To see just how dramatically, evaluate a few of these curves at $x = 3$: for instance, at $\beta^\star = (2, 1, 0)$, we have

$$f(x) = x + 2 \implies f(3) = 3 + 2 = 5,$$

while at $\beta^\star = (2, -99, 100)$,

$$f(x) = 100x^2 - 99x + 2 \implies f(3) = 900 - 297 + 2 = 605.$$

At $x = 0$ and $x = 1$, every member of the family agrees — they must, by construction. But two units to the right of the data, the predictions range from 5 to 605, and there is *nothing* in the model telling us which to prefer. And, ultimately, this is a price we're forced to pay if we choose to use *this model* to model *this data*.

Let us pause for a moment, though, and really think about what we're trying to do here. Given only two data points — $\mathcal{D} = \{(0, 2), (1, 3)\}$ — we somewhat absentmindedly "went along" with the quadratic model we'd proposed *assuming* we'd possess $n = 3$ data points. The problem is that we were given only $n = 2$ data points; we simply never had enough information to justify our model in the first place. We say the model is **overparameterized** relative to what we could hope for the model to reasonably learn.

Concretely, given only two data points, the seemingly most natural choice would be to model their relationship is not a *quadratic*; it is simply a *line*. Intuitively, what we understand is that the segment connecting the two data points $(0, 2)$ and $(1, 3)$ represents a fairly natural choice for predicting outputs $\hat{y}$ at levels of the predictor lying inside the range of our observed data, $\hat{x} \in [0, 1]$. In such a case, if we *truly don't know* "what's going on" in the regime where the predictor x takes values in $[0, 1]$ strictly, such a **linear interpolation** would seem to "minimize the level of undue influence we're able to exert on the resultant predictions." Having then enforced linearity *inside* the range of our observed predictors $[0, 1]$ — and absent any reason to believe otherwise — the most reasonable predictions *outside* $[0, 1]$ would, intuitively, seem to be **linear extrapolations** based on the data-derived segment connecting $(0, 2)$ and $(1, 3)$. In this way, what we might assert is that the "most appropriate model" in this case is the linear one

$$f(x) = 2 + x, \qquad x \in \mathbb{R}.$$

Remark.

Of course, we still came to the conclusion that the linear model was "most appropriate" even though we started in the broader class of quadratic models. Yet, we can't help but imagine how much simpler life would have been if we'd *started* by only considering the class of linear models — i.e., those of the form

$$y = \beta_0 + \beta_1 x.$$

Such a move reduces what was a fairly intensive logical argument to simple algebra. Our hope, quite often, is to reduce a complex problem's structure to something we can solve algorithmically.

What changes when we add a third data point, then, is that we permit the model to discover *curvature*. Suppose we observe the additional sample $(x_i, y_i) = (3, 8)$, so that our full dataset becomes

$$\mathcal{D}_3 = \{(0, 2), (1, 3), (3, 8)\},$$

where we've indexed the data by the sample size $n = 3$.

At this point, one observes that the prediction yielded by our previous linear model, $f(3) = 5$, undershoots the observed $y_3 = 8$ at the same level of the predictor. What we choose to do with this information is, of course, up to us — nevertheless, if we're supposing that the data has been generated perfectly in accordance with the rest of the process, we really should ask ourselves how we intend to proceed. On the one hand, when we only had $n = 2$ samples, the linear model $f(x) = x+2$ proved to be "best" at capturing our original data, supposing our goal is to minimize the amount of influence we're exerting on the model's predictions.

On the other hand, now that we have $n = 3$ samples, it appears as if the *class* of linear models is inadequate — try as we might, we'll *never* find a line that goes through all three points in $\mathcal{D}_3$. So, suppose we were to go back to the original class of quadratic noiseless models given by

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2, \qquad i = 1, 2, 3.$$

Plugging in the observed data gives

$$\begin{aligned} 2 &= \beta_0 \\ 3 &= \beta_0 + \beta_1 + \beta_2 \\ 8 &= \beta_0 + 3\beta_1 + 9\beta_2, \end{aligned}$$

which, from $\beta_0 = 2$, immediately gives $\beta_1 + \beta_2 = 1$, the same constraint from the original data, *and a second constraint*, $3\beta_1 + 9\beta_2 = 6$. One finds from our previous elimination that the noiseless quadratic model corresponding to our data $\mathcal{D}_3$ is

$$p_3(x) = 2 + \frac{1}{2}x + \frac{1}{2}x^2,$$

where we've again indexed the model f_n by the sample size $n = 3$.

One again verifies that the model p_3 predicts $\mathcal{D}_3$ perfectly. Furthermore, this is the simplest polynomial function capable of perfectly fitting the data — we could also do so with a class of cubic polynomials, but barring any reason to believe such a structure *should* exist, the quadratic class is perfectly capable of describing the data, and it does so in a *simpler way*. (We tend to prefer **parsimonious** models — that is, within a group of models which perform comparably on our observed data, we tend to prefer those which minimize model complexity). We can verify that

$$p_3(4) = 2 + 2 + 8 = 12,$$

while we also note that our previous linear model, $p_2(x) := 2 + x$, would have predicted

$$p_2(4) = 2 + 4 = 6.$$

One might observe this and begin to think that the models p_2 and p_3 are in conflict — recall that the prediction by model p_2 at $x = 3$ disagreed with our third sample point $(x_3, y_3) = (3, 8)$ (where we've set the quadratic model in order for $p_3(3) = 8$) by $|p_3(3) - p_2(3)| = |8 - 5| = 3$ units. Now, at $x = 4$, we observe that the predictions by the two models disagree by a more substantial $|p_3(4) - p_2(4)| = |12 - 6| = 6$ units. Suppose, then, that observe a fourth data point $(x_4, y_4) = (4, 6)$ — where, again, we assume that the new sample point has been generated perfectly, in accordance with the existing data — and we update our data

$$\mathcal{D}_4 = \{(0, 2), (1, 3), (3, 8), (4, 6)\}.$$

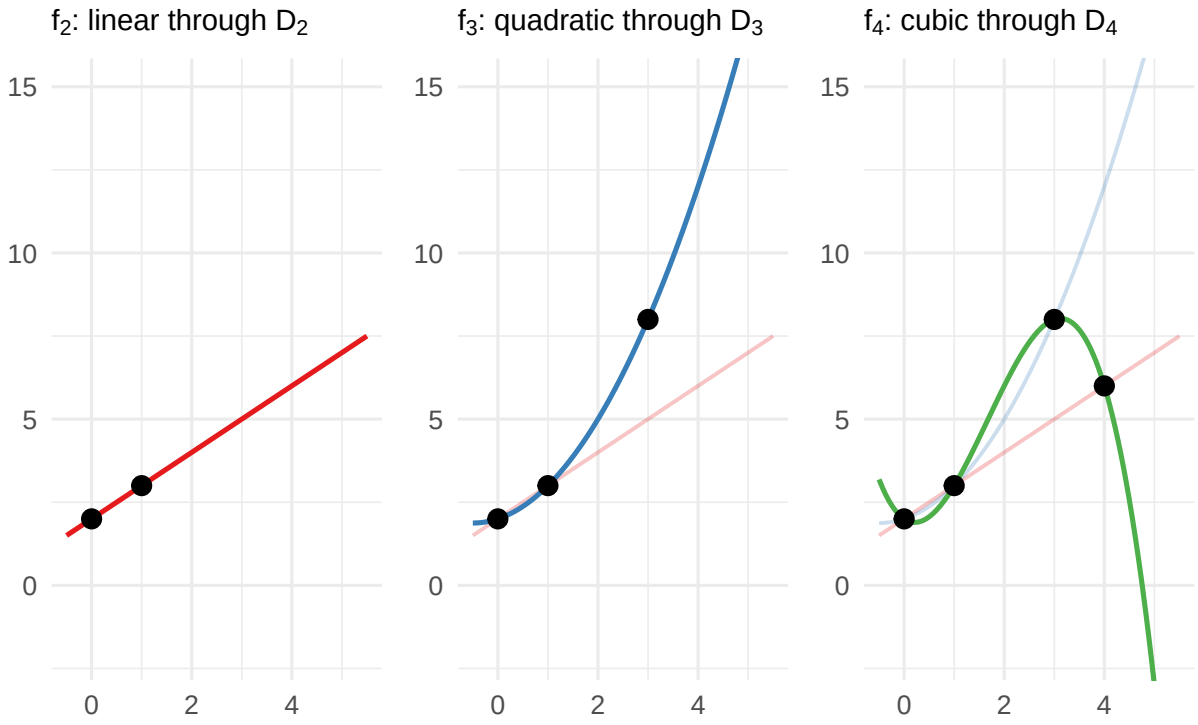
Notice that the fourth sample *agrees* with the prediction given by the linear model p_2: $|y_4 - p_2(4)| = |6-6| = 0$. On the other hand, y_4 clearly disagrees with the quadratic model's prediction at the same level of the predictor: $|y_4 - p_3(4)| = |6 - 12| = 6$.

Furthermore, what we observe is that, just as the class of linear models was insufficient for characterizing the three samples in $\mathcal{D}_3$, the class of quadratic models $f(x_i) = \beta_0 + \beta_1 x_i + \beta_2 x_i^2$ is insufficient for completely characterizing the four samples in $\mathcal{D}_4$ — in particular, one can verify there is *no* quadratic that passes through all four points of $\mathcal{D}_4$ simultaneously. If we trust the trend, it would seem as if a *cubic* model would be sufficient; i.e., if we set

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3, \qquad i = 1, 2, 3, 4,$$

then we'd be able to solve for a unique $\beta^\star = (\beta_0^\star, \beta_1^\star, \beta_2^\star, \beta_3^\star)$ such that the sample points $(x_i, y_i) \in \mathcal{D}_4$ are all generated by the model, and it would be the lowest order polynomial capable of passing through all four points in $\mathcal{D}_4$ uniquely. Indeed, this is true; one obtains (after a somewhat more involved elimination than before)

$$p_4(x) = 2 - x + \frac{5}{2}x^2 - \frac{1}{2}x^3$$



Looking back on the story thus far, we have now built three successive models, each tailored to its generation of data:

$$p_2(x) = 2 + x, \qquad p_3(x) = 2 + \frac{1}{2}x + \frac{1}{2}x^2, \qquad p_4(x) = 2 - x + \frac{5}{2}x^2 - \frac{1}{2}x^3.$$

Each is the *lowest-degree polynomial* that passes through all available data perfectly. Each was, at the time of its construction, the parsimonious choice. And each was *invalidated* by the next observation: p_2 predicted $p_2(3) = 5$ but we got $(x_3, y_3) = (3, 8)$; p_3 predicted $p_3(4) = 12$ but we got $(x_4, y_4) = (4, 6)$. Right now, p_4 is perfect at fitting $\mathcal{D}_4$, but what right have we earned to be *confident* in how p_4 will perform on a future sample?

Very little. And the situation is, on reflection, *even worse* than it first appears. Look at the trajectory of the coefficients across the three models:

	β_0	β_1	β_2	β_3
p_2	2	1	—	—
p_3	2	1/2	1/2	—
p_4	2	−1	5/2	−1/2

The intercept $\beta_0 = 2$ has been stable — this is entirely a consequence (a gift, even) of having observed the sample point $(x_1, y_1) = (0, 2)$. But the linear coefficient β_1 went from 1 down to 1/2 then even more to -1 — it does not seem that the coefficient is stabilizing as we observe more samples, but rather oscillating quite wildly in an effort to catch up with the new information. Each new observation isn't *refining* the previous model; just *destabilizing* it.

Once again, we stress that, here, we've been working with *perfect data*. We can't pin the problems on "measurement error" or "randomness" — the appropriate models always return

$f_n(x_i) = y_i$ exactly. It would seem somewhat paradoxical; something about what we're doing is simply *wrong*, but it would appear as though we're making a principled move at every step along the way.

3.1.1 Chasing the Dragon

A Setup For Disaster

Still, all of this would still seem perfectly reasonable supposing the data were *truly generated by a polynomial*. Concretely, supposing the DGP really were a fixed polynomial of degree $d > 0$, say

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_d x^d,$$

then as soon as the sample size $n > d$, the table above would begin to stabilize: the extra data points would be consistent with the existing model, no new terms would be needed, and things would start to settle down. This is fine — and, if we're so lucky as to receive such a finitely parameterized polynomial DGP, we should accept the interpolation strategy as perfectly legitimate.

But it would already seem quite wishful thinking to imagine more complex processes would be characterized by a polynomial of finite-degree. Yet, what we *might hope* is for there to be a saving grace hidden in plain sight: suppose, for instance, that we were to observe samples drawn from an exponential DGP — say, one of the form

$$g(x) := e^{-x^2}, \qquad x \in \mathbb{R}.$$

Of course, we recall the Taylor series for e^x is just

$$e^x \sim 1 + x + \frac{x^2}{2} + \frac{x^3}{6} + \cdots = \sum_{n=0}^{\infty} \frac{x^n}{n!};$$

plugging $x \mapsto -x^2$ into this expression, we can write the Taylor series for g as

$$e^{-x^2} \sim 1 - x^2 + \frac{1}{2}x^4 - \frac{1}{6}x^6 + \cdots = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n}}{n!}, \qquad \forall x \in \mathbb{R}.$$

We would then perhaps hope that, as our sample size $n \to \infty$, the polynomial interpolants g_n constructed in the manner described above would, term by term, begin to resemble the Taylor coefficients: $\beta_0 = 1, \beta_1 = 1, \beta_2 = \frac{1}{2}, \beta_3 = \frac{1}{6}$, and so on. That is, we'd hope that, eventually, our strategy of interpolating the data would succeed under an exponential DGP — and, indeed, in *this case*, it's a reasonable wish. The reason why is that the function $g(x) = e^{-x^2}$ is **analytic** (or **entire**): i.e., its Taylor series converges for all $x \in \mathbb{R}$.

But we know that e^x is special, particularly with regards to its Taylor series. By contrast, suppose that we observed data generated by

$$h(x) := \frac{1}{1 + x^2}, \qquad x \in \mathbb{R}.$$

We can recall (or verify, as appropriate) that the Taylor series for $\frac{1}{1-x}$ which converges for $|x| < 1$ is

$$\frac{1}{1-x} \sim 1 + x + x^2 + x^3 + \cdots = \sum_{n=0}^{\infty} x^n,$$

and realizing that, equivalently,

$$h(x) = \frac{1}{1-(-x^2)}, \qquad |-x^2| < 1 \implies |x| < 1,$$

we can form a Taylor series for $h(x)$, convergent for x within the radius of convergence $R = 1$ — i.e., for $x \in (-1, 1)$ — as

$$h(x) = \frac{1}{1+x^2} \sim 1 - x^2 + x^4 - x^6 + \cdots = \sum_{n=0}^{\infty} (-1)^n x^{2n}, \qquad |x| < 1.$$

When we juxtapose the Taylor series — namely, the above with

$$g(x) = e^{-x^2} \sim 1 - x^2 + \frac{1}{2}x^4 - \frac{1}{6}x^6 + \cdots = \sum_{n=0}^{\infty} \frac{1}{n!}(-1)^n x^{2n}, \qquad \forall x \in \mathbb{R},$$

what we observe is that the two are astonishingly similar in form. Nevertheless, it is just as immediate *why* the former series for $h(x)$ — which, for convenience, we defined for *all* $x \in \mathbb{R}$ (even the divergent x), as

$$\mathcal{T}[h(x)] := \sum_{n=0}^{\infty} (-1)^n x^{2n}, \qquad x \in \mathbb{R}.$$

In particular, we observe that $\mathcal{T}[h(x)]$ diverges for $|x| \geq 1$: in such a case, $\mathcal{T}[h]$ is an alternating series whose terms are strictly increasing in magnitude whenever $|x| \geq 1$:

$$\forall n \geq 1 : \quad |(-1)^n x^{2n}| = |x|^{2n} \cdot 1 \leq |x|^{2n} \cdot |x| = |(-1)^{n+1} x^{2n+1}|.$$

The terms are oscillating more and more wildly for larger $|x| \geq 1$ — at the ends, for instance, one may verify that each $x = \pm 1$ give the harmonic series.

It is just as clear why the series

$$\mathcal{T}[g(x)] := \sum_{n=0}^{\infty} \frac{1}{n!}(-1)^n x^{2n}, \qquad x \in \mathbb{R},$$

does not suffer such issues: its scaling factor of $1/n!$ ensures that, at some point, the successive multiplications by x^{2n} for $|x| \geq 1$ are eventually overtaken by the ever-growing factors of $n \to \infty$ in $1/n!$. "Even though $|x|^{2n} = |x| \cdot |x| \cdots |x|$ may have $|x|$ large enough that it beats $n! = n \cdot (n-1) \cdots 2 \cdot 1$ for a while, eventually $n!$ is going to catch up; and, when it does, it's going to stay ahead forever after."

The Gaussian and The Cauchy

We've deliberately been somewhat illusory about the whole affair, but what we've really done here has a salient punchline that brings us back to the previous discussion: imagine that we take h and g as defined above to be the Data Generating Processes underlying some data,

$$\mathcal{G} = \{(x_i, g(x_i)\}_{i=1}^m, \qquad \mathcal{H} = \{(x_j, h(x_j)\}_{j=1}^n.$$

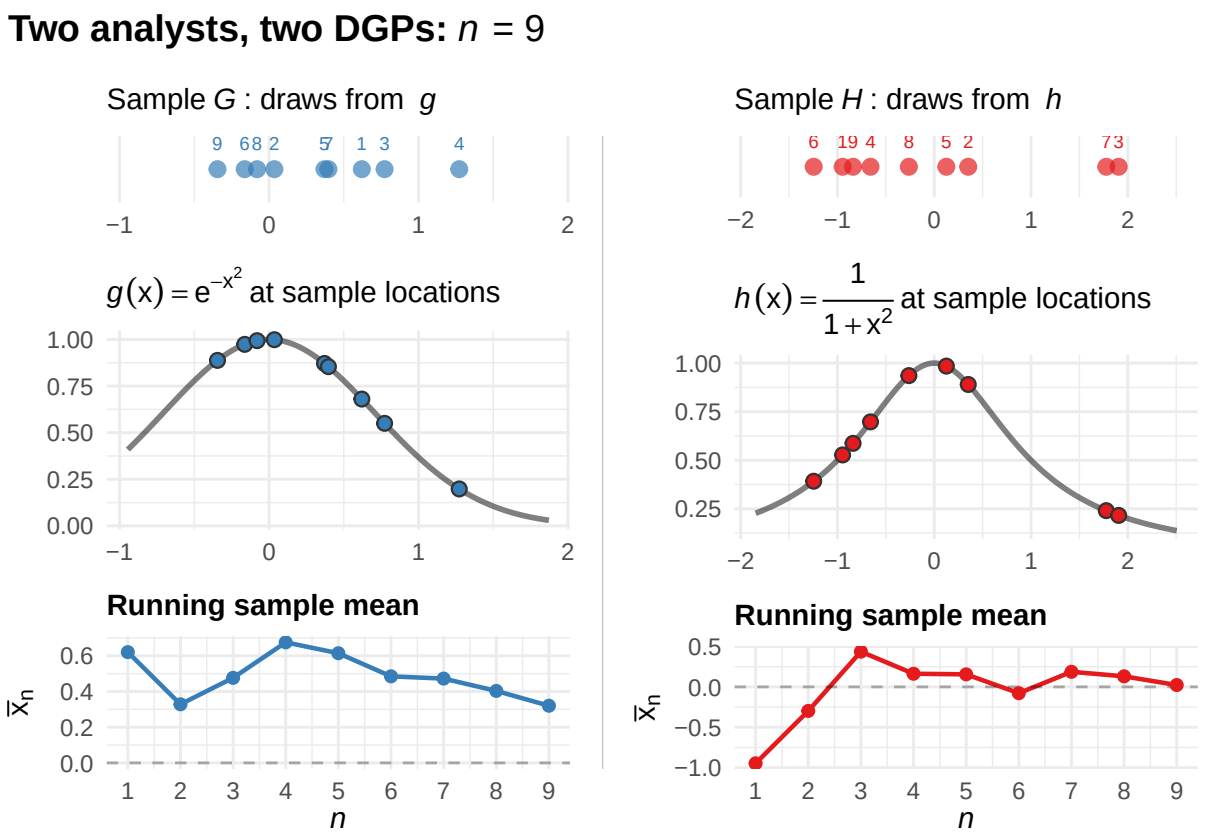
We again suppose that the observed "y_i" are generated perfectly in accordance with the DGPs as defined above. Two analysts are given data $\mathcal{G}$ and $\mathcal{H}$ respectively, but neither knows the precise functional forms of g or h, and they're asked to reverse-engineer the functional relationship to

the best of their ability — largely the same setup as before, only now, *we're* dictating the DGP so that we may learn from the cautionary tale of the analyst of $\mathcal{H}$.

Remark.

In an analogous, more realistic setting, all the analysts would observe are the raw draws x_i/x_j — the density evaluations $g(x_i)/h(x_j)$ would be *unobservable*, and we'd hope to infer the approximate shape of the density by "how many draws we observed near a given x." Here, we *give* these evaluations to the analysts; it is a more relaxed setting than what we'd encounter in practice. As we will see, it leads to catastrophe regardless.

We imagine that, as the data "continue to roll in," the analysts repeatedly recompute the sample means of their observed data in an effort to summarize the center of location of the data. The plot below relays their computations at each sample size n.



Concretely, rather than imagining $g(x) = e^{-x^2}$ and $h(x) = \frac{1}{1+x^2}$ as "functions to be evaluated at chosen input locations $x \in \mathbb{R}$," we imagine that we the plot above depicts samples *drawn from* these curves in accordance with the DGP itself. That is, again, we might imagine going into the populations characterizing $\mathcal{G}/\mathcal{H}$ and drawing sample points x_i/x_j from each population, where we interpret the evaluations $g(x_i)/h(x_j)$ as *unobserved* quantities that are "driving the likelihood" we observe a given x_i/x_j.

To really make this "likelihood" intuition concrete, we could imagine *normalizing* g and h:

indeed, one verifies (quite instructively, if you haven't seen it before; see the exercises at the end of the chapter for a pair of guided derivations) that

$$g(x) \geq 0 \quad \text{and} \quad h(x) \geq 0, \qquad \forall x \in \mathbb{R},$$

that

$$\int_{-\infty}^{\infty} g(x)\,dx = \int_{-\infty}^{\infty} e^{-x^2}\,dx = \sqrt{\pi},$$

and that

$$\int_{-\infty}^{\infty} h(x)\,dx = \int_{-\infty}^{\infty} \frac{1}{1+x^2}\,dx = \pi.$$

In particular, then, g and h can each be normalized to form a probability density function. For simplicity, we overwrite the previous unnormalized DGPs by the new, "practically equivalent" functions

$$g(x) = \frac{1}{\sqrt{\pi}} e^{-x^2}, \qquad x \in \mathbb{R},$$

and

$$h(x) = \frac{1}{\pi} \frac{1}{1+x^2}, \qquad x \in \mathbb{R}.$$

Though we haven't formally defined it in the case when the samples are *infinitely supported* (i.e., when we can observe $x \in \mathbb{R}$ anywhere in the entire real line rather than from strictly within a bounded interval $x \in [a, b]$), it would seem as if, upon this normalization, each g and h have been endowed with a probabilistic interpretation: we observe $g(x) \geq 0$ and $h(x) \geq 0$ for any $x \in \mathbb{R}$, and the integral of the newly normalized densities over $\mathbb{R}$ each equal 1. Indeed, even if things are a bit hand-wavy at this point, we can assure you that this *is valid* — i.e., we really can imagine the normalized g and h as valid probability densities.

But, again, the analysts don't know any of this — *all* they see are the scatterplots of the data. Really, they were *always* "imagining the situation probabilistically," even when draming g and h as deterministic functions to be queried at particular inputs. The normalization just makes their intuition explicit: the data points are drawn from a distribution, and the shape of that density determines what "typical" data looks like.

So, at $n = 9$, both analysts see roughly the same thing: a cluster of observations near the origin, roughly symmetric, and both with sample means hovering around zero: i.e., under the observed data with $m = n = 9$,

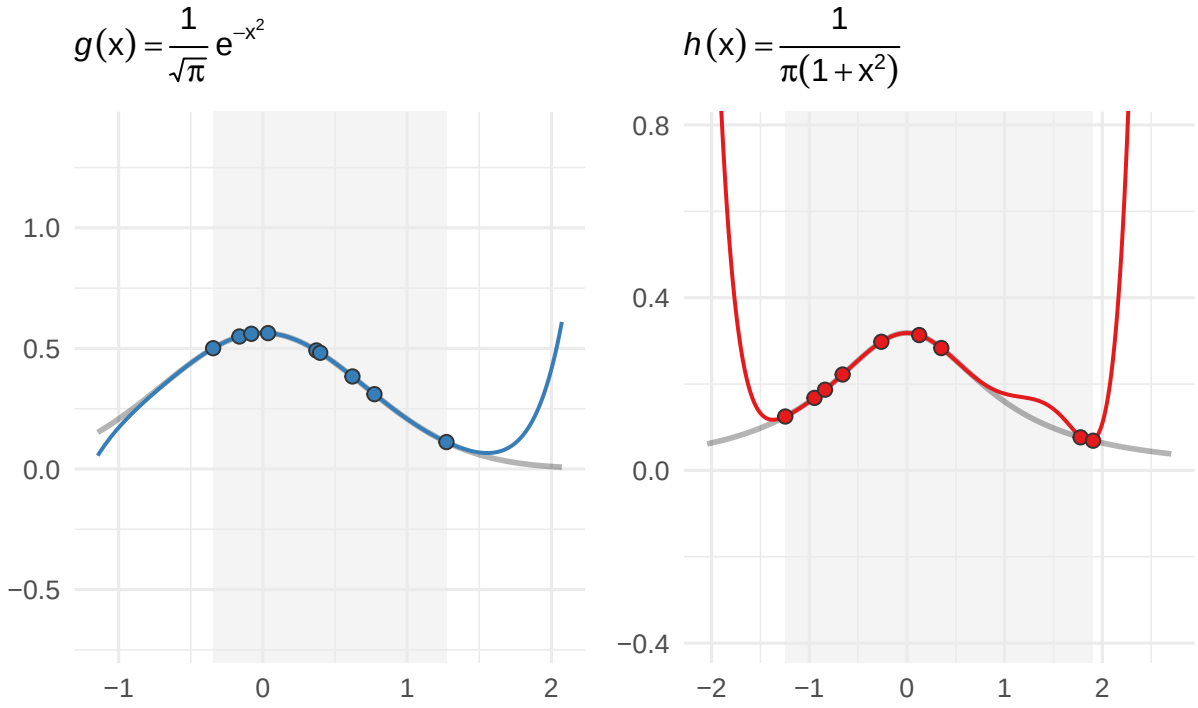
$$\frac{1}{m}\sum_{i=1}^{m} x_i = \frac{1}{n}\sum_{j=1}^{n} x_j \approx 0.$$

Nothing has yet emerged to suggest the data differ in a meaningful way. By almost every reasonable metric, the data *are* the same. What we imagine is that the analysts, noticing the structural similarities, might opt to pursue similar modeling strategies.

Suppose, then, that they each decide to pursue the strategy of interpolating the data by polynomials, the same strategy we illustrated in motivating the current discussion. They obtain two polynomials, and they plot each polynomial curve as a function of x, overlaying the original data points in $\mathcal{G}/\mathcal{H}$ on top for comparison. What they observe, then, is something like the following.

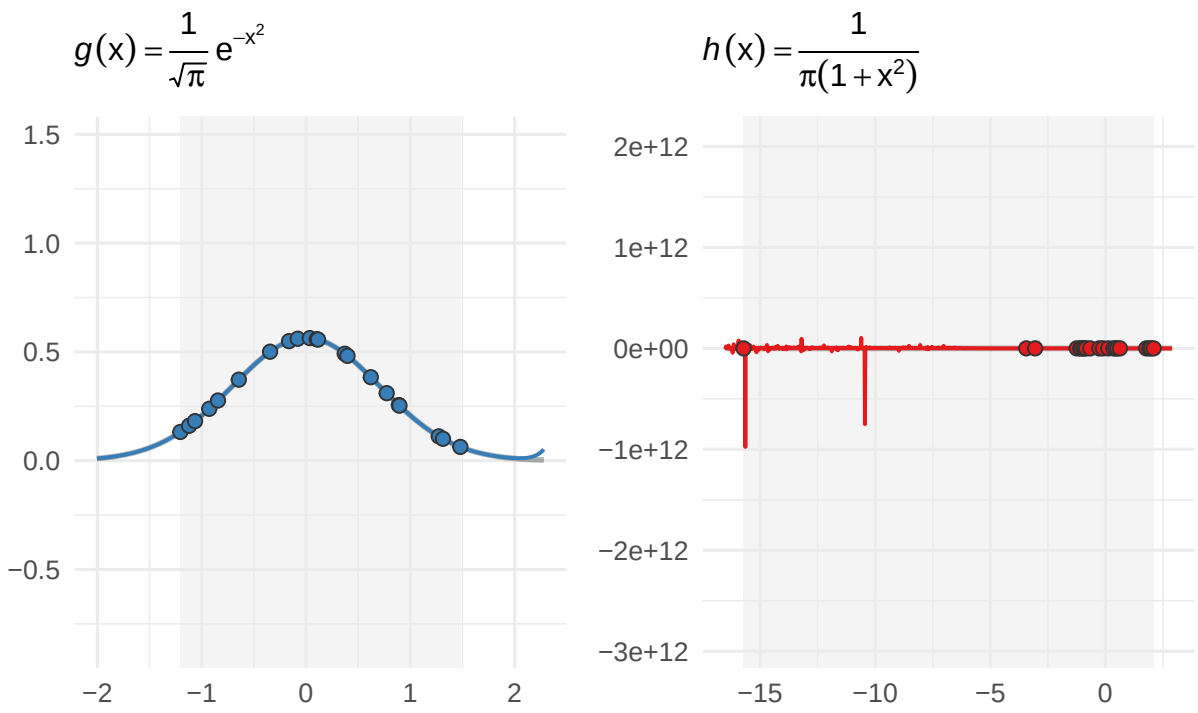
Interpolation at observed sample locations: $n = 9$

Degree−8 interpolant; shaded region = range of observed data



The "correct" density lines in dark gray here are overlaid for our understanding, but again, it is worth re-emphasizing that the entirety of what the analysts know is subsumed by the plots and curves in blue and red, respectively. And, at this point, based on what we see in the shaded region corresponding to the range of predictors actually represented in our observed data, it seems as if both analysts would be reassured they are on the right track.

It is really only when the second batch of data rolls in that things begin to unravel for the analyst of $\mathcal{H}$. We suppose that after receiving a second wave of $n_2 = m_2 = 12$ samples, the analysts simply roll them in with their existing data comprised of $n_1 = m_1 = 9$ samples. Together, the combined data $\mathcal{G} = \{(x_i, g(x_i))\}_{i=1}^{m}$ and $\mathcal{H} = \{(x_j, h(x_j))\}_{j=1}^{n}$ consists of $n = m = 21$ total sample points. Again, the data is perfect, no noise, yada yada — they proceed to re-fit the interpolating polynomials, at which point they observe the following.



The analyst of $\mathcal{G}$ is, deservedly, quite thrilled. The degree-20 interpolant has essentially merged with the true curve: each wave of data has only seemed to improve the fit.

The analyst of $\mathcal{H}$ is, also deservedly, quite appalled. One of the twelve new observations landed at $x \approx -15$ — far from the cluster near the origin that had characterized the first batch. What results is a polynomial whose behavior is necessarily so erratic as to describe a scattering of points $(x_j, h(x_j))$ which is seemingly devoid of structural integrity in the presence of this "seeming outlier." Of course, as the oracles, we know that at inputs $x \approx -15$, the DGP h yields

$$h(-15) = \frac{1}{\pi}\frac{1}{1+(-15)^2} \approx 0.0014,$$

and that we may (somewhat loosely at this point) "interpret this evaluation probabilistically."

We quickly contrast this, for instance, with the competing DGP g's evaluation of the same "probability" (which, to be somewhat more precise about the interpretation, could be viewed as "how often the DGP based on g produces sample points $x_i \in \mathcal{G}$ near $x = -15$"):

$$g(-15) = \frac{1}{\sqrt{\pi}} e^{-(-15)^2} \approx 1.084 \times 10^{-98}.$$

That is: while the Cauchy density h regards an observation near $x = -15$ as uncommon, it is still *entirely legitimate* — roughly a 1-in-700 event, and this is something we could have anticipated observing on a sample of size $n = 21$ fairly often under the same DGP. On the other hand, the Gaussian density g regards the same observation near $x = -15$ as, for all practical purposes, *impossible*. This is not a colloquial use of the word impossible — it is *emphatic*, and one should quickly observe that the reciprocal of the number of atoms in the observable universe

is, on the lower end, 10^{-83} — which is to say that $g(-15) \approx 10^{-98}$ is *incomprehensibly small*. We can remain, for all practical purposes, absolutely certain that the analyst of $\mathcal{G}$ will never receive a similarly destabilizing data point as that which has already plagued $\mathcal{H}$, and not because she is fortunate — but because her DGP g is *structurally incapable* of producing one.

Yet, of course, these analysts remain unaware of their folly. Suppose the analyst of $\mathcal{H}$, furious over the "contaminated data," rushes to the experimental team and insists that they've really messed it up this time — "we were rolling *just fine* at $n = 9$ — what happened, guys?"

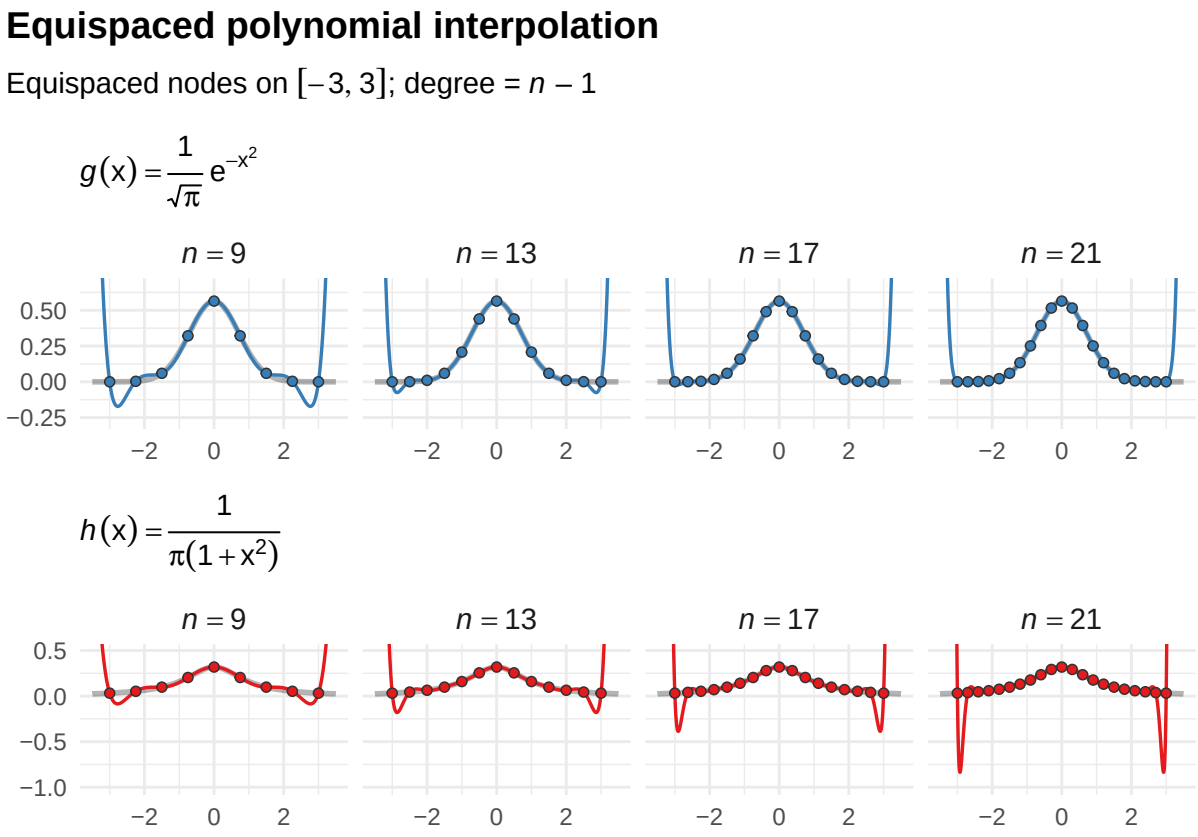
And the truth is that he *isn't* crazy — but *no* amount of data the team could convince him otherwise. Indeed, suppose we, acting as benevolent oracles, granted each analyst data access to some new data:

$$\mathcal{G}_n = \{(x_i, g(x_i))\}_{i=1}^n, \quad \mathcal{H}_n = \{(x_i, h(x_i))\}_{i=1}^n,$$

where the x_i we're evaluating each of the functions g and h at are *identical* — in particular, both analysts receive data generated from the *same* set of equispaced nodes

$$x_1 < x_2 < \cdots < x_n, \quad \text{uniformly spaced on } [-3, 3].$$

In effect, what we're doing is "circumventing the DGP and allowing the analysts to see how their polynomial approximations would work deployed on *uniform partitions* of $[-3, 3]$." What we deliver the analysts, and their corresponding polynomial approximations, are reflected at four sample sizes $n = 9, 13, 17$, and 21 in the figure below.



Chasing the Dragon

It is strange indeed that these DGPs, each capable of producing data which is so ostensibly

similar to the other, exhibit such strikingly different behavior in their respective polynomial interpolants. These "fangs" we observe in the plot of the Cauchy DGPs data emerge gradually as we continue to increase the sample size n. What is somewhat interesting to observe is that, at $n = 9$, both the Gaussian and Cauchy DGPs produce data which are "mildly fanged" — yet, the Gaussian's fangs "curl in" as n increases. It would seem as if, the more points we supply, the more violently the Cauchy polynomial coefficients must thrash around in order to "hit all of the points in the data" — which, again, would seem quite strange, considering the curves are *so ostensibly similar*. Why is the Cauchy DGP just "a bit too unwieldy" for our approximating polynomial to properly interpolate its points?

The answer reveals itself all at once when we look at the actual coefficients comprising each of these polynomials. First, remind yourself once more of the Taylor series corresponding to (the unnormalized) g and h:

$$g(x) = e^{-x^2} \sim 1 - x^2 + \frac{1}{2}x^4 - \frac{1}{6}x^6 + \cdots = \sum_{k=0}^{\infty} \frac{(-1)^k}{k!} x^{2k}, \qquad x \in \mathbb{R}.$$

and

$$h(x) = \frac{1}{1+x^2} \sim 1 - x^2 + x^4 - x^6 + \cdots = \sum_{k=0}^{\infty} (-1)^k x^{2k}, \qquad |x| < 1,$$

Now, look at the table below.

Interpolating polynomials for $g(x) = e^{-x^2}$ vs. Taylor series $T[g]$

	$T[g]$	p_5	p_9	p_{13}	p_{21}
x^0	1	1.0000	1.0000	1.0000	1.0000
x^2	-1	-0.4931	-0.9473	-0.9977	-1.0000
x^4	$\frac{1}{2}$	0.0424	0.3549	0.4859	0.5000
x^6	$-\frac{1}{6} \approx -0.1667$	—	-0.0558	-0.1434	-0.1666
x^8	$\frac{1}{24} \approx 0.0417$	—	0.0030	0.0254	0.0415
x^{10}	$-\frac{1}{120} \approx -0.0083$	—	—	-0.0024	-0.0082
x^{12}	$\frac{1}{720} \approx 0.0014$	—	—	0.0001	0.0013

Interpolating polynomials for $h(x) = \dfrac{1}{1+x^2}$ vs. Taylor series $T[h]$

	$T[h]$	p_5	p_9	p_{13}	p_{21}
x^0	+1	1.0000	1.0000	1.0000	1.0000
x^2	-1	-0.3769	-0.8127	-0.9570	-0.9983
x^4	+1	0.0308	0.3386	0.7007	0.9694
x^6	-1	—	-0.0581	-0.3077	-0.8327
x^8	+1	—	0.0032	0.0718	0.5540
x^{10}	-1	—	—	-0.0081	-0.2589
x^{12}	+1	—	—	0.0003	0.0806

In both cases, the interpolating polynomial's coefficients are *converging to the Taylor coefficients* of the respective DGP. For the Gaussian DGP g, this is good news: the Taylor coefficients

$$c_{2k} = \frac{(-1)^k}{k!} \to 0 \quad \text{as} \quad k \to \infty,$$

and, with a rate of decay behaving like $1/k!$, these coefficients will eventually dominate any fixed evaluation of x^{2k} at a given $x \in \mathbb{R}$. What we interpret, then, is that the interpolant's convergence to the true Gaussian is a direct consequence of the Taylor series' convergence: each additional term the interpolant learns contributes a mere $|c_{2k}| = 1/k!$ in magnitude to the overall approximation — a factor shrinking so rapidly that the polynomial can "afford to incorporate it into the existing approximation without destabilizing the existing fit elsewhere." The true coefficient of the x^{12} term is $c_{12} = 1/720 \approx 0.0014$ — a correction so small that the decision to include or exclude it only affects the approximation's behavior at fairly large x, and certainly not within a neighborhood close to 0, say $x \in [-3, 3]$.

For the Cauchy DGP h, the interpolating coefficients are converging *just as faithfully* — but, here, the Taylor coefficients are

$$c_{2k} = (-1)^k,$$

which alternate in sign and remain at unit magnitude $|c_{2k}| = 1$ forever. By $n = 21$, the coefficient of x^4 has reached 0.97, headed for $+1$; the coefficient of x^6 sits at -0.83, headed for -1; that of x^8 at 0.55 is headed for $+1$, and so on. Each new term the interpolant of h seeks to learn is *just as large* as the last.

Furthermore, the series $\sum(-1)^k x^{2k}$ *diverges* for $|x| \geq 1$ — which is to say, the vast majority of the interval $[-3, 3]$. The "fangs at the boundary" are not an artifact of the procedure; they exhibit a polynomial trying desperately to conform to the rules at a smattering of locations it wasn't equipped to handle, that the approximation simply isn't realistic anywhere.

Taming the Dragon

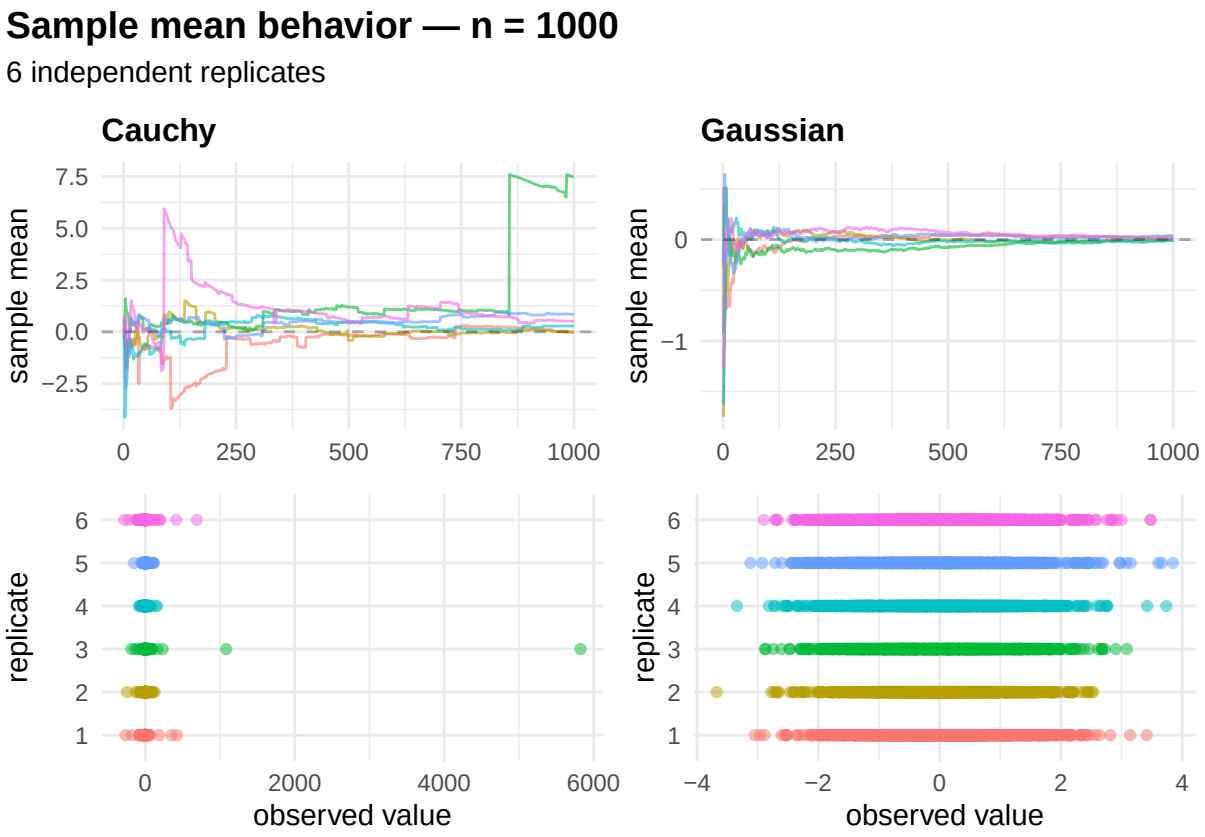
This, then, would seem to mark the end of the parable; an unsatisfying end, to be sure, but an instructive one nonetheless. Indeed, what have we learned from the analysts of $\mathcal{G}$ and $\mathcal{H}$? That *memorization* is not the same as *learning*. Sometimes, we happen upon a data generating process which is well-behaved, and others we're forced to reckon with a process which seems purposefully adversarial — and what we've observed here is that the line separating the situations is far too fine for comfort. We're forced to exercise due diligence in understanding the mechanics of the Cauchy's elusive behavior; indeed, we imagine that if we weren't equipped to handle a function as simple as $\frac{1}{1+x^2}$, we'd remain quite hopeless in modeling the complexities of data generating processes one encounters in practice.

But we also stress that, here, *diligence itself* is a bit of a dragon. What we mean is that, no matter how many times we've recalled the parable's lessons, we find it much harder to willingly "throw away information" than it is to try and squeeze every last drop of it from the given data. It is not a failure to compromise in such situations; after recognizing analytic pursuits aren't working, such compromise *is* diligence exemplified in its purest form. The real struggle as we see it is discerning when matters are "truly hopeless," and those where some other recourse at our disposal should we just avert our gaze to the horizon. Indeed, we reassure you now that it is most often the latter case which is closer to reality — but, more aptly, reality tends to live somewhere in the middle. Inhabiting this middle with *confidence* is, in effect, the data analyst's eternal pursuit.

Looking ahead, we choose to close with an intuitive gesture towards something we'll strive to

make precise moving forward. Concretely, one way we can understand the parable is through the lens of **moments**. Indeed, we're able to verify is that the Cauchy distribution has no finite mean: $\int_{-\infty}^{\infty} |x| \cdot h(x)\, dx = \infty$. More dramatically, it has no finite moments of *any* order — $\mathbb{E}[X^{2k}] = \infty$ for every $k \geq 1$. The law of large numbers, which guarantees $\overline{x}_n \to \mu$ for the Gaussian, simply does not apply for the Cauchy. The reason is not pathological — it's because the Cauchy's tails are heavy enough that a single sufficiently extreme observation can drag the running average by an arbitrarily large amount, and such observations arrive often enough that the average never recovers.

What we can seek to understand eventually is that the reasons that the Taylor coefficients of h refuse to decay and that the moments of the corresponding distribution refuse to exist are, in effect, *the same*. We'll make this connection precise when we introduce the Fourier transform and its statistical analogue, the characteristic function; for now, we leave the parable with the following figure, whose idiosyncrasies we encourage you to attempt to decipher according to the intuition developed here.



3.2 The Importance of a Basis

3.2.1 Polynomials

A Brief Review

Let us now attempt to reconcile what we learned from the parable with a presentation minding the rigor of the matter a bit more forthrightly. At this point, we encourage you to remind yourself of the definitions we presented at the beginning of this chapter — in particular, we recall that we began by investigating a noiseless system of equations taking the form

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2, \qquad i = 1, \dots, n.$$

Consider now a generalization of the basic form above: we might write it as

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_k x_i^k, \qquad i = 1, \dots, n.$$

To better facilitate the generalization, we recognize that, if we define the monomials

$$\phi_0(x) := 1, \qquad \phi_1(x) := x, \qquad \phi_2(x) := x^2, \quad \dots, \quad \phi_k(x) := x^k, \quad \dots,$$

then by defining the **design matrix** (or **model matrix**)

$$\Phi := \begin{pmatrix} \phi_0(x_1) & \phi_1(x_1) & \cdots & \phi_k(x_1) \\ \phi_0(x_2) & \phi_1(x_2) & \cdots & \phi_k(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_0(x_n) & \phi_1(x_n) & \cdots & \phi_k(x_n) \end{pmatrix} \in \mathbb{R}^{n \times p}, \qquad p := k + 1,$$

and similarly stacking the rest of the model in vector form:

$$y := \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} \in \mathbb{R}^n, \qquad \beta := \begin{pmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_k \end{pmatrix} \in \mathbb{R}^p,$$

then we can write the noiseless linear model in the compressed matrix form given by

$$y = \Phi\beta.$$

What we observed in the first act of the parable were a series of data,

$$\mathcal{D}_2 := \{(0,2),(1,3)\}, \quad \mathcal{D}_3 := \{(0,2),(1,3),(3,8)\}, \quad \mathcal{D}_4 := \{(0,2),(1,3),(3,8),(4,6)\},$$

and, having been told these data were generated in accordance with a *perfect* DGP, we proceeded to fit a polynomial model that **interpolated** the samples $(x_i, y_i) \in \mathcal{D}_k$ exactly. In particular, then, what we stated was that the "polynomials of lowest degree interpolating the data" were given by

$$p_2(x) := 2 + x, \qquad p_3(x) := 2 + \frac{1}{2}x + \frac{1}{2}x^2, \qquad p_4(x) = 2 - x + \frac{5}{2}x^2 - \frac{1}{2}x^3$$

Furthermore, what we seemed to discover, at least implicitly, was that these interpolating-polynomials-of-lowest-degree were *unique* — we emphasize "of-lowest-degree" since, if we admitted a class of polynomials one degree higher for each data set, our solution class suddenly becomes *infinite*. This was what we found initially, when we attempted to fit a quadratic model containing $p = 3$ monomial terms ϕ_0, ϕ_1, ϕ_2 to our original data $\mathcal{D}_2$ containing only $n = 2$ samples.

Notice, importantly, that the class of solutions became infinite as soon as the number of monomials p exceeded the number of samples in our data n. In matrix notation, this would correspond with a design matrix Φ of the form

$$\Phi = \begin{pmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \end{pmatrix} \in \mathbb{R}^{2\times 3};$$

observe that this matrix is *wider than it is tall*. Such a "wide matrix" should be compared with, say,

$$\Phi = \begin{pmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \\ 1 & x_4 & x_4^2 \end{pmatrix} \in \mathbb{R}^{4\times 3},$$

i.e., a matrix which is *taller than it is wide*. Such a "tall matrix" would coincide with, for instance, our attempt to fit a quadratic model in $p = 3$ monomial terms to interpolate the data containing $n = 4$ samples $\mathcal{D}_4$.

Now, what we observe is that neither of these cases — the "wide" matrix ($n < p$) nor the "tall" matrix ($n > p$) — coincides with a model that actually came out of the parable. Indeed, we *gestured* toward the possibility, but what we found was that the quadratic model on $\mathcal{D}_2$ produced an *infinite number* of solutions — at $n = 2$ and $p = 3$ the quadratic class is too flexible to *uniquely* identify a solution $\beta \in \mathbb{R}^3$. Yet, at $n = 4$ and $p = 3$, the quadratic class isn't flexible enough to *produce* a valid solution $\beta \in \mathbb{R}^3$ at all — i.e., a solution *doesn't exist*.

Instead, when we fit p_2 to $\mathcal{D}_2$ at $n = 2$, we used a *linear* model comprised of $p = 2$ monomial terms $\phi_0(x) = 1, \phi_1(x) = x$, and we claimed the solution $\beta = (2, 1)^\top \in \mathbb{R}^2$ was *unique*. When we fit p_3 to $\mathcal{D}_3$ at $n = 3$, we used a *quadratic* model comprised of $p = 3$ monomial terms $\phi_0(x) = 1, \phi_1(x) = x, \phi_2(x) = x^2$, and we claimed the solution $\beta = (2, 1/2, 1/2)^\top \in \mathbb{R}^3$ was unique. It would appear that, in order to obtain a solution to the noiseless system $\beta \in \mathbb{R}^p$ which *exists and is unique* given a sample of size n, we would, in general, require the columns of Φ be comprised of exactly $p = n$ monomial terms $\phi_0, \phi_1, \dots, \phi_{n-1}$.

Indeed, at least in the non-trivial case, we have the following result.

Theorem 3.2.1 (Polynomial Interpolation)

Suppose $x_1, x_2, \dots, x_n \in \mathbb{R}$ be *distinct* real numbers ($x_1 \neq x_2 \neq \cdots \neq x_n$), and let $y_1, y_2, \dots, y_n \in \mathbb{R}$ be arbitrary. Then, there *exists a unique* polynomial

$$p \in \Pi_{n-1} := \{\text{polynomials of degree at most } n-1\} := \left\{ \sum_{j=0}^{n-1} a_j x^j : \;\; a_0, \dots, a_{n-1} \in \mathbb{R} \right\},$$

such that

$$p(x_i) := a_0 + a_1 x_i + a_2 x_i^2 + \cdots + a_{n-1} x_i^{n-1} = y_i, \qquad i = 1, \dots, n.$$

Proof:

The proof asserts both (i) *existence* of a polynomial $p \in \Pi_{n-1}$ such that $p(x_1) = y_1, \dots, p(x_n) = y_n$, and that (ii) this polynomial p is *unique*. Such a proof requires us

to show both parts separately. Here, we focus on the proof of uniqueness — indeed, we defer the existence argument to a *constructive one* presented in the subsequent material.

So, to show uniqueness, we often proceed by supposing the converse and deriving a contradiction. Namely, here, begin by supposing that $p \in \Pi_{n-1}$ and $q \in \Pi_{n-1}$ were *distinct* polynomials (i.e., at least one of their coefficients disagree) of degree at most $n-1$ satisfying $p(x_i) = y_i$ and $q(x_i) = y_i$ for all $i = 1, \dots, n$. Then, since the right sides would be equal, we subtract each side to force the requirement that

$$r(x_i) := p(x_i) - q(x_i) = 0, \qquad i = 1, \dots, n.$$

On the other hand, when we add or subtract polynomials $p, q \in \Pi_{n-1}$, we're certainly not increasing the degree of the result (e.g., $\underbrace{(3x+5)-(x-2)}_{p(x)-q(x)} = \underbrace{2x+7}_{r(x)}$ — we'll formalize this in a tick). Indeed,

$$p, q \in \Pi_{n-1} \implies r := p - q \in \Pi_{n-1};$$

i.e., if p and q are each of degree at most $n-1$, then so too is $r = p - q$ at most of degree $n-1$ (where, quite possibly, r's degree could shrink as a result of the subtraction).

What the constraint $r(x_i) = 0$, $i = 1, \dots, n$ says in this light is that r, a polynomial of degree at most $n-1$, must vanish at n distinct points $x_1, \dots, x_n$. Again deferring the rigorous discussion, we can imagine that at $n = 2$, $r \in \Pi_{n-1}$ is a polynomial of degree $n - 1 = 1$ (i.e., a line), and we're constraining this line to vanish at exactly $n = 2$ points (x_1, y_1) and (x_2, y_2) — it is, of course, *impossible* to do this without setting the line perfectly on the x-axis (i.e., setting r to be the constant line $r(x) = 0$ for all x).

In general, the same sort of argument shows that in order for $r \in \Pi_{n-1}$ to satisfy n constraints, it must be that r is the **zero** (or **trivial**) polynomial,

$$r \equiv 0 \in \Pi_{n-1}.$$

Finally, since $0 \equiv r = p - q$, we have $p = q \in \Pi_{n-1}$, which establishes the claimed uniqueness.

‖

The proof above leaned on two assertions we haven't yet formally justified: that (i) Π_{n-1} is "closed under subtraction" — that $p, q \in \Pi_{n-1}$ implies $p - q \in \Pi_{n-1}$ — and that (ii) a polynomial of degree at most $n-1$ which vanishes at n distinct points *must* be the zero polynomial. Both of these are consequences of the *algebraic structure* of polynomials, in a manner we now seek to make precise.

Theorem 3.2.2 (Properties of Polynomials (I))

Let Π_k denote the set of all polynomials of degree at most k with real coefficients. Then:

(a) (Closure under Linear Combinations) If $p, q \in \Pi_k$ and $\alpha, \beta \in \mathbb{R}$, then

$$\alpha p + \beta q \in \Pi_k.$$

(b) (Root Bound) If $0 \neq p \in \Pi_k$ is not the zero polynomial, then p has at most k real-valued roots.

In establishing the proof of the root bound, we require the following lemma (whose proof, though certainly instructive, we ultimately defer to a guided derivation in the exercises).

Lemma 3.2.3 (Polynomial Factor)

If $p \in \Pi_k$ and $p(a) = 0$ for some $a \in \mathbb{R}$, then there exists a polynomial $q \in \Pi_{k-1}$ such that

$$p(x) = (x - a)q(x), \qquad x \in \mathbb{R}.$$

Proof:

(a) To establish the closure of Π_k, suppose that $p, q \in \Pi_k$ are typical members such that

$$p(x) = \sum_{j=0}^{k} a_j x^j, \qquad q(x) = \sum_{j=0}^{k} b_j x^j, \qquad x \in \mathbb{R}.$$

Then, by defining the addition $p + q$ by

$$p(x) + q(x) := \sum_{j=0}^{k} (a_j + b_j) x^j, \qquad x \in \mathbb{R},$$

and multiplication of p and q by scalars $\alpha, \beta \in \mathbb{R}$ respectively by

$$\alpha p(x) := \sum_{j=0}^{k} (\alpha a_j) x^j, \qquad \beta q(x) := \sum_{j=0}^{k} (\beta b_j) x^j,$$

what we verify is that

$$\alpha p(x) + \beta q(x) := \sum_{j=0}^{k} (\alpha a_j + \beta b_j) x^j$$

is a polynomial of degree k, so that $\alpha p + \beta q \in \Pi_k$. (The key point here is that we verify the coefficients of the linear combination remain valid and, hence, constitute a well-defined polynomial $\alpha p + \beta q \in \Pi_k$ simply by inheriting it from the real coefficients in the summations, $\alpha a_j + \beta b_j \in \mathbb{R}$.)

(b) For the Root Bound, let $0 \neq p \in \Pi_k$ be a nontrivial, degree-k polynomial of the form

$$p(x) = c_0 + c_1 x + c_2 x^2 + \cdots + c_k x^k.$$

Then, if p has no real roots, we're done (verify why this is obvious!); otherwise, let a be a root of p. By the Polynomial Factor Lemma, we have

$$p(x) = (x - a)q(x), \qquad q \in \Pi_{k-1}.$$

If $q \equiv 0$, then $p(x) = (x-a) \cdot 0 = 0$ for all x, implying $p \equiv 0 \in \Pi_k$ is the trivial polynomial — a contradiction to our original assumption. In particular, it must be that $q \in \Pi_{k-1}$ is nontrivial.

In particular, then: if q has no real roots, we're done (and p has one root, $x = a$). If not and $b \neq a$ is a root of q, then

$$p(b) = \underbrace{(b-a)}_{\neq 0}\ \underbrace{q(b)}_{=0} = 0,$$

so that b is a root of $p \in \Pi_k$ as well. In particular, we may write

$$p(x) = (x-a)(x-b)r(x), \qquad r \in \Pi_{k-2}.$$

We repeat the process of reapplying the Polynomial Factor Lemma until the resultant polynomial on the right has no remaining roots (in which case we conclude it has as many roots as the number of times we were able to apply the Lemma), or until it the polynomial becomes identically a constant $c \in \Pi_0$ after k iterations. In the latter, most extreme case, we ultimately obtain a representation for p of the form

$$p(x) = c(x-a_1)(x-a_2)\cdots(x-a_k).$$

This polynomial has exactly k roots, and is structurally incapable of possessing more. Any other, less extreme possibility terminates when the representation on the right has *fewer* than k roots; in particular, it must be that $p \in \Pi_k$ has at most k roots.

‖

Notice what happened in the proof of the root bound: in the "most extreme case," we started with a polynomial of the form

$$p(x) = c_0 + c_1 x + \cdots + c_k x^k,$$

and we ended with a *new representation* for the *same polynomial*,

$$p(x) = c(x-a_1)(x-a_2)\cdots(x-a_k).$$

What we also seemed to indicate was that, whenever we found a polynomial $p \in \Pi_k$ had less than k roots (say it has $j \leq k$), our repeated applications of the Factor Lemma would proceed and terminate according to

$$\begin{aligned}
&(0) && p(x) = \sum_{i=0}^{k} c_i x^i, && 0 \not\equiv p \in \Pi_k;\\
&(1) && p(x) = (x-a_1)r_1(x), && 0 \not\equiv r_1 \in \Pi_{k-1};\\
&(2) && p(x) = (x-a_1)(x-a_2)r_2(x), && 0 \not\equiv r_2 \in \Pi_{k-2};\\
& && \vdots &&\\
&(j) && p(x) = (x-a_1)(x-a_2)\cdots(x-a_j)r_j(x), && 0 \not\equiv r_j \in \Pi_{k-j}.
\end{aligned}$$

At each line of this process, the object on the left — $p(x)$ — has not changed. Indeed, the polynomial p is a function: a rule that accepts as input a real number $x \in \mathbb{R}$ and returns a real number, $p(x) \in \mathbb{R}$. Two expressions that dictate equivalent rules — i.e., that return the same value at every $x \in \mathbb{R}$ — are *not different polynomials*; they are *different descriptions* of the *same polynomial*.

That is, in the steps of the algorithm outlined above, what we observe is that at step (0), p is expressed as a linear combination of the functions $\{1, x, x^2, \dots, x^k\}$ weighted by real numbers $\{c_0, c_1, c_2, \dots, c_k\}$. On the other hand, at step (j), p is expressed as a product of j linear factors $(x - a_1)(x - a_2) \cdots (x - a_j)$ and a residual polynomial r_j, where the numbers $\{a_1, \dots, a_j\}$ are its roots. The weights c_i and the roots a_i are, in general, completely different numbers, and neither set of numbers "is the polynomial"; they are what we obtain when we *describe* the polynomial in a *particular language*. It just so happens that, here, the monomial expression at step (0) and the factored expression at step (j) describe the same rule p by characterizing two seemingly distinct structural properties of the original polynomial — at step (0), "we know how much each monomial $x_i \in \{1, x, \dots, x^k\}$ comprises the whole polynomial p by looking at the coefficient c_i"; at step (j), "we know where the real roots $\{a_1, a_2, \dots a_j\}$ of p fall in $\mathbb{R}$."

This distinction — between the polynomial p and a particular representation — is not merely philosophical. Indeed, what we understand is that if we recall, for instance, the polynomials from the parable

$$p_2(x) := 2 + x, \qquad p_3(x) := 2 + \frac{1}{2}x + \frac{1}{2}x^2, \qquad p_4(x) = 2 - x + \frac{5}{2}x^2 - \frac{1}{2}x^3,$$

stack the model coefficients in a table as follows:

	β_0	β_1	β_2	β_3
p_2	2	1	—	—
p_3	2	1/2	1/2	—
p_4	2	−1	5/2	−1/2

Reading down the columns of the coefficients, it is only the intercept $\beta_0 = 2$ which remains stable across the three models — in particular, the linear β_1 and quadratic β_2 terms seem to shift, even going so far as changing signs: the linear coefficient attached to β_1 fluctuates from 1 in p_2 to -1 in p_4.

It is genuinely curious, then, that the coefficient $\beta_0 = 2$ — which, recall, we deduced directly from the common sample point $(0, 2)$ present across each of the datasets $\mathcal{D}_2, \mathcal{D}_3, \mathcal{D}_4$ — is the sole estimate that remains stable. Indeed, the reason is visible in the design matrix itself. For the model p_2 given data $\mathcal{D}_2 = \{(0, 2), (1, 3)\}$, the system determining the model, $\mathbf{y} = \mathbf{\Phi B}$, was

$$\begin{pmatrix} 2 \\ 3 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \end{pmatrix}.$$

The system is lower triangular, and we can observe directly that the top row identifies $\beta_0 = 2$ directly. How we obtain β_1, on the other hand, requires us to have identified that $\beta_0 = 2$ from the first constraint, at which point we deduce from the second row that

$$3 = \beta_0 + \beta_1 \implies \beta_1 = 3 - 2 = 1.$$

Still, this deduction was fairly straightforward: it was apparent from the **triangular** form of the matrix $\Phi = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$ that we could "compute the β_i coefficients from the top down." It was straightforward because the first row told us precisely what β_0 was, automatically forcing the value of the only unknown in the second row β_1.

Compare this now to the corresponding system for p_3 on $\mathcal{D}_3 = \{(0,2),(1,3),(3,8)\}$:

$$\begin{pmatrix} 2 \\ 3 \\ 8 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 3 & 9 \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{pmatrix}.$$

Again, we're essentially handed $\beta_0 = 2$ for free in the first row because the first sample point has $x_1 = 0$, at which each monomial $\phi_1(x) = x, \phi_2(x) = x^2, ...$ evaluates to zero. However, what happens in the following rows is immediately less clear — indeed, the second and third rows describe the equations

$$\begin{aligned} \beta_0 + \beta_1 + \beta_2 &= 3, \\ \beta_0 + 3\beta_1 + 9\beta_2 &= 8. \end{aligned}$$

Nothing is *wrong* with these equations; they're just **dense**. Even if we plug in what we know from the first row, $\beta_0 = 2$, and reduce appropriately, we're still left with

$$\begin{aligned} \beta_1 + \beta_2 &= 1, \\ 3\beta_1 + 9\beta_2 &= 6, \end{aligned}$$

which isn't quite helpful precisely because it doesn't tell us what β_1 and β_2 *are*, only what they must *satisfy* in order to form a valid solution to the original system. This is, in effect, what we intend to underscore when we distinguish objects from their representations — i.e., it is clear that the dense representation "works" insofar as characterizing the quadratic model p_3 on $\mathcal{D}_3$, but it does little to reveal the model's *underlying structure*, whatever it may be.

So, to reiterate, if we look to the original system on $\mathcal{D}_3$,

$$\begin{pmatrix} 2 \\ 3 \\ 8 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 3 & 9 \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{pmatrix},$$

one deduces according to the logic deployed above that a solution $\beta \in \mathbb{R}^3$ must satisfy

$$\begin{aligned} \beta_0 &= 2 \\ \beta_1 + \beta_2 &= 1, \\ 3\beta_1 + 9\beta_2 &= 6. \end{aligned}$$

Again, our "issue" with this deduction isn't that it's "wrong," just that it's quite verbose — it tells us exactly what β_0 is, but only seems to "describe" what β_1 and β_2 are.

What we understand from a first course in linear algebra is that, given two equations in two unknowns, we're often able to employ **substitution** or **elimination** to reduce the system into a more explicit form. Indeed, from

$$\begin{aligned} \beta_1 + \beta_2 &= 1, \\ 3\beta_1 + 9\beta_2 &= 6, \end{aligned}$$

we can rearrange $\beta_2 = 1 - \beta_1$, plug in

$$3\beta_1 + 9(1 - \beta_1) = 6 \implies \beta_1 = \frac{1}{2},$$

and from the rearrangement obtain

$$\beta_2 = 1 - \beta_1 = \frac{1}{2}.$$

In particular, any solution $\beta \in \mathbb{R}^3$ solving the original system is forced to satisfy

$$\begin{aligned}\beta_0 &= 2\\ \beta_1 &= \frac{1}{2},\\ \beta_2 &= \frac{1}{2},\end{aligned}$$

which is of the explicit form we desired.

But what can we say about the process in general? Indeed, let us take a step back and perform the same series of operations on the general form of the quadratic model given $n = 3$ samples, which is

$$\begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{pmatrix}.$$

If we go to row-reduce the system in the usual way, we might start by realizing that if we subtract row 1 from rows 2 and 3, we obtain

$$\begin{pmatrix} y_1 \\ y_2 - y_1 \\ y_3 - y_1 \end{pmatrix} = \begin{pmatrix} 1 & x_1 & x_1^2 \\ 0 & x_2 - x_1 & x_2^2 - x_1^2 \\ 0 & x_3 - x_1 & x_3^2 - x_1^2 \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{pmatrix}.$$

We observe that in coefficient matrix, there are common factors of $x_2 - x_1$ in the second row (since $x_2^2 - x_1^2 = (x_2 - x_1)(x_2 + x_1)$) and $x_3 - x_1$ in the third. Supposing as we have been that the nodes x_i are distinct, we may divide by $x_2 - x_1 \neq 0$ and $x_3 - x_1 \neq 0$ to obtain

$$\begin{pmatrix} y_1 \\ \dfrac{y_2 - y_1}{x_2 - x_1} \\ \dfrac{y_3 - y_1}{x_3 - x_1} \end{pmatrix} = \begin{pmatrix} 1 & x_1 & x_1^2 \\ 0 & 1 & x_1 + x_2 \\ 0 & 1 & x_1 + x_3 \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{pmatrix}$$

Now, if we subtract row 2 from row 3 and divide it by $(x_3 - x_2)$, we obtain

$$\begin{pmatrix} y_1 \\ \dfrac{y_2 - y_1}{x_2 - x_1} \\ \dfrac{\dfrac{y_3 - y_1}{x_3 - x_1} - \dfrac{y_2 - y_1}{x_2 - x_1}}{x_3 - x_2} \end{pmatrix} = \begin{pmatrix} 1 & x_1 & x_1^2 \\ 0 & 1 & x_1 + x_2 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{pmatrix}$$

At this point, observe that the coefficient matrix on the right is **upper triangular** — ones on the diagonal, arbitrary entries above the diagonal, and zeros below it. On the left-side, we observe a new kind of structure emerging from the row operations. The first entry is y_1, which is just querying the value of p at the first node x_1. However, the second row is no longer describing a constraint on the position of the polynomial's nodes — i.e., that "at $x = x_2$, $p(x_2) = y_2$" — instead, it constrains

$$\frac{y_2 - y_1}{x_2 - x_1};$$

this is no longer a positional constraint, but rather one controlling the *rate of change* of the polynomial evaluations $y_1 = f(x_1)$ and $y_2 = f(x_2)$ as x moves from x_1 to x_2. We define

$$[y_1, y_2] := \frac{y_2 - y_1}{x_2 - x_1}$$

to be the **first divided difference** of the data $\mathcal{D}_3$, and one immediately recognizes that the first divided difference is precisely the slope of the secant line connecting the points (x_1, y_1) with (x_2, y_2). "The first row of this new system constrains the position of the first node $y_1 = f(x_1)$; the second row looks at the position of that first node $(x_1, f(x_1))$, observes where $(x_2, f(x_2))$ is supposed to be, and forces the slope of the linear component β_1 to be such that the line through (x_1, y_1) also passes through (x_2, y_2)." In other words, we've "exchanged" a constraint on *where the polynomial goes* for a constraint on *how fast it gets there*. The original row 2 said "$p(x_2) = y_2$" — a positional constraint. The new row 2 says "the average rate of change of p moving from x_1 to x_2 equals the first divided difference $[y_1, y_2]$."

The third entry on the left admits an analogous interpretation. Define the **second divided difference** by

$$[y_1, y_2, y_3] := \frac{[y_1, y_3] - [y_1, y_2]}{x_3 - x_2} = \frac{\dfrac{y_3 - y_1}{x_3 - x_1} - \dfrac{y_2 - y_1}{x_2 - x_1}}{x_3 - x_2},$$

and observe that this is a *difference of slopes* normalized by the node spacing between x_2 and x_3. That is, $[y_1, y_2, y_3]$ characterizes "how the linear rate of change must, well, *itself* change" in order for the model to pass through all three points — i.e., it characterizes a **discrete curvature**. As the first divided difference is to a "discrete first derivative," the second divided difference is to a "second discrete derivative," computed directly from the observed data.

In this language of divided differences, what the new system reads is

$$\begin{pmatrix} [y_1] \\ [y_1, y_2] \\ [y_1, y_2, y_3] \end{pmatrix} = \begin{pmatrix} 1 & x_1 & x_1^2 \\ 0 & 1 & x_1 + x_2 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{pmatrix}.$$

(We may understand $[y_1] = y_1$ as a **zeroth divided difference** if we're so inclined).

Viewed as a description of the solution's underlying mechanism, this formulation of the system is much clearer than our initial one. Again, the reason is because this new system is *triangular* — indeed, if one "starts at the bottom row," we start by reading off

$$\beta_2 = [y_1, y_2, y_3]$$

directly. Then, we observe that

$$\beta_1 + (x_1 + x_2)\beta_2 = [y_1, y_2] \implies \beta_1 = [y_1, y_2] - (x_1 + x_2)[y_1, y_2, y_3].$$

Finally, after some simplification, we obtain

$$\beta_0 + x_1\beta_1 + x_1^2\beta_2 = y_1 \implies \beta_0 = y_1 - x_1[y_1, y_2] + x_1x_2[y_1, y_2, y_3]$$

Together, the solution in this language of divided differences is

$$\begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{pmatrix} = \begin{pmatrix} [y_1] - x_1[y_1, y_2] + x_1x_2[y_1, y_2, y_3] \\ [y_1, y_2] - (x_1 + x_2)[y_1, y_2, y_3] \\ [y_1, y_2, y_3] \end{pmatrix}.$$

Indeed, one can verify that these formulas applied to the data $\mathcal{D}_3 = \{(0,2),(1,3),(3,8)\}$ yields the familiar solution

$$\begin{pmatrix}\beta_0\\ \beta_1\\ \beta_2\end{pmatrix} = \begin{pmatrix}2\\ 1/2\\ 1/2\end{pmatrix}.$$

But what we understand from the general formulation is that, really, it was "quite lucky" that the constraint on β_0 reduced to the mere identification $\beta_0 = 2$. The reason for this reduction is just as apparent from the form above — the top row involves a linear combination of divided differences, where all orders besides the zeroth $[y_1]$ are weighted by x_1, which happened to coincide with the data point $(x_1, y_1) = (0, 2)$. Since $x_1 = 0$, the whole thing reduces to $\beta_0 = [y_1] = y_1 = 2$. In general, though, it is really the *quadratic* coefficient which we're able to "read off" in this manner — indeed, we compute the second divided difference y_1, y_2, y_3 directly from the raw data, and we identify this quantity precisely as the quadratic coefficient β_2. We then back-substitute this second order difference in the row constraining $\beta_1 = [y_1, y_2] - (x_1 + x_2)[y_1, y_2, y_3]$, computing the first divided difference $[y_1, y_2]$ along the way to identifying the linear slope β_1. Only then, having already computed the first and second divided differences/β_2 and β_1, do we move to the row constraining $\beta_0 = [y_1] - x_1[y_1, y_2] + x_1x_2[y_1, y_2, y_3]$, where we plug everything in and solve for the intercept β_0.

Changing the Questions

So again, returning to the original form of the system for $\mathcal{D}_3$ given by

$$\begin{pmatrix}2\\ 3\\ 8\end{pmatrix} = \begin{pmatrix}1 & 0 & 0\\ 1 & 1 & 1\\ 1 & 3 & 9\end{pmatrix}\begin{pmatrix}\beta_0\\ \beta_1\\ \beta_2\end{pmatrix},$$

what we understand is that, *mechanically*, row reduction transformed the original system — three *distinct* positional constraints on p that "at $x = x_i$, $f(x_i) = y_i$" — into a new system asking *hierarchical* constraints: a value, a slope running from that value, and the curvature of that same slope.

The way we got from the original system to the hierarchical one was via row operations on the system — namely, we transitioned from

$$\begin{pmatrix}y_1\\ y_2\\ y_3\end{pmatrix} = \begin{pmatrix}1 & x_1 & x_1^2\\ 1 & x_2 & x_2^2\\ 1 & x_3 & x_3^2\end{pmatrix}\begin{pmatrix}\beta_0\\ \beta_1\\ \beta_2\end{pmatrix} \longmapsto \begin{pmatrix}[y_1]\\ [y_1, y_2]\\ [y_1, y_2, y_3]\end{pmatrix} = \begin{pmatrix}1 & x_1 & x_1^2\\ 0 & 1 & x_1 + x_2\\ 0 & 0 & 1\end{pmatrix}\begin{pmatrix}\beta_0\\ \beta_1\\ \beta_2\end{pmatrix}.$$

To make this a bit more rigorous, observe that if we retrace our steps through the row operations, then if the original system on the left above is

$$y = \Phi\beta,$$

then, in lieu of the "row operations" we conducted mechanically, perhaps we could hope to define a matrix L such that

$$Ly = L\Phi\beta := \Psi\beta, \qquad \Psi := L\Phi.$$

That is, in a more expanded notation, perhaps we could hope to find a matrix $L = \Big((\ell_{ij})\Big)$ such that when we left-multiply the system as

$$\begin{pmatrix}\ell_{11} & \ell_{12} & \ell_{13}\\ \ell_{21} & \ell_{22} & \ell_{23}\\ \ell_{31} & \ell_{32} & \ell_{33}\end{pmatrix}\begin{pmatrix}y_1\\ y_2\\ y_3\end{pmatrix} = \begin{pmatrix}\ell_{11} & \ell_{12} & \ell_{13}\\ \ell_{21} & \ell_{22} & \ell_{23}\\ \ell_{31} & \ell_{32} & \ell_{33}\end{pmatrix}\begin{pmatrix}1 & x_1 & x_1^2\\ 1 & x_2 & x_2^2\\ 1 & x_3 & x_3^2\end{pmatrix}\begin{pmatrix}\beta_0\\ \beta_1\\ \beta_2\end{pmatrix},$$

the numbers ℓ_{ij} are chosen such that

$$\begin{pmatrix} \ell_{11} & \ell_{12} & \ell_{13} \\ \ell_{21} & \ell_{22} & \ell_{23} \\ \ell_{31} & \ell_{32} & \ell_{33} \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} [y_1] \\ [y_1, y_2] \\ [y_1, y_2, y_3] \end{pmatrix},$$

and

$$\begin{pmatrix} \ell_{11} & \ell_{12} & \ell_{13} \\ \ell_{21} & \ell_{22} & \ell_{23} \\ \ell_{31} & \ell_{32} & \ell_{33} \end{pmatrix} \begin{pmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \end{pmatrix} = \begin{pmatrix} 1 & x_1 & x_1^2 \\ 0 & 1 & x_1 + x_2 \\ 0 & 0 & 1 \end{pmatrix}.$$

In effect, we're hoping to "turn row operations into matrix multiplication"; it is perhaps "how we'd tell the computer to perform the analogous operations."

Indeed, one can verify that the matrix

$$L := \begin{pmatrix} 1 & 0 & 0 \\ \dfrac{-1}{x_2 - x_1} & \dfrac{1}{x_2 - x_1} & 0 \\ \dfrac{1}{(x_2 - x_1)(x_3 - x_1)} & \dfrac{-1}{(x_2 - x_1)(x_3 - x_2)} & \dfrac{1}{(x_3 - x_1)(x_3 - x_2)} \end{pmatrix}$$

accomplishes precisely this. The numbers themselves aren't nearly as important as the *structure* of L. Indeed, we observe that the entries above the diagonal vanish, $\ell_{12} = \ell_{13} = \ell_{23} = 0$ — i.e., the matrix L is lower triangular. And, when we apply the lower triangular L to the dense matrix by mapping $\Phi \mapsto L\Phi$, we obtain our desired upper triangular matrix,

$$\begin{pmatrix} 1 & 0 & 0 \\ \dfrac{-1}{x_2 - x_1} & \dfrac{1}{x_2 - x_1} & 0 \\ \dfrac{1}{(x_2 - x_1)(x_3 - x_1)} & \dfrac{-1}{(x_2 - x_1)(x_3 - x_2)} & \dfrac{1}{(x_3 - x_1)(x_3 - x_2)} \end{pmatrix} \begin{pmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \end{pmatrix} = \begin{pmatrix} 1 & x_1 & x_1^2 \\ 0 & 1 & x_1 + x_2 \\ 0 & 0 & 1 \end{pmatrix}.$$

Similarly, when we apply L to the data vector $\mathbf{y}$, we obtain

$$\begin{pmatrix} 1 & 0 & 0 \\ \dfrac{-1}{x_2 - x_1} & \dfrac{1}{x_2 - x_1} & 0 \\ \dfrac{1}{(x_2 - x_1)(x_3 - x_1)} & \dfrac{-1}{(x_2 - x_1)(x_3 - x_2)} & \dfrac{1}{(x_3 - x_1)(x_3 - x_2)} \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} [y_1] \\ [y_1, y_2] \\ [y_1, y_2, y_3] \end{pmatrix}.$$

Thus, in its entirety, the row-reduced system reads

$$L\mathbf{y} = L\Phi\beta, \qquad \text{i.e.,} \qquad \begin{pmatrix} [y_1] \\ [y_1, y_2] \\ [y_1, y_2, y_3] \end{pmatrix} = \underbrace{\begin{pmatrix} 1 & x_1 & x_1^2 \\ 0 & 1 & x_1 + x_2 \\ 0 & 0 & 1 \end{pmatrix}}_{U := L\Phi} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{pmatrix}.$$

Taking stock of the whole procedure, we observe that in introducing the matrix L, it acted on the *left* of both $\mathbf{y}$ and Φ — transforming the data into divided differences and the design matrix into upper triangular form — while leaving the unknowns β entirely untouched. What is crucial about the interpretation is that the numbers β_0, β_1, and β_2 still measure the influence of the monomials $\phi_0(x) = 1$, $\phi_1(x) = x$ and $\phi_2(x) = x^2$; yet, what is just as apparent is that the *questions* we're

asking to ascertain these influences have fundamentally changed. Again, the original system asks three positional questions — "where does p go at x_1, x_2, and x_3?" — while the row-reduced system asks three hierarchical constraints — "where does p go at x_1; given this value, how does the slope of p change to attain another value at x_2; given this value and slope, how does the curvature of p change to attain a third value at x_3?"

We stress that the reduced form of the system we derived in the last ribbon *really is* an improvement insofar as understanding the mechanism driving the solution. When we levy a new positional constraint on the polynomial, it isn't really that the coefficients β_i shift according to the "location of the new node," but rather according to "how aggressively we must contort our polynomial to hit the new node, given it must still hit all of the previous nodes." Nevertheless, what we observe is that, in spite of our careful consideration of the *questions* we're asking, the *language of our solution* isn't rendered any cleaner for it.

The divided differences — the hierarchical answers to these new questions — sit on the *left-hand side* of the reduced system. They describe the data $\mathbf{y}$. But the unknowns on the right are still $\beta_0, \beta_1, \beta_2$ — the monomial coefficients. These were the original curiosity — we were wondering why, in the table

	β_0	β_1	β_2	β_3
p_2	2	1	—	—
p_3	2	1/2	1/2	—
p_4	2	−1	5/2	−1/2

the other coefficients continued to bounce around while β_0 remained constant. At this point, we've revealed this to be a "lucky coincidence" — a consequence of the sample point $(x_1, y_1) = (0, 2)$ appearing in every dataset. But the coefficients are "actually" being computed via, e.g.,

$$\begin{pmatrix}\beta_0\\ \beta_1\end{pmatrix} = \begin{pmatrix}[y_1] - x_1[y_1, y_2]\\ [y_1, y_2]\end{pmatrix}, \qquad \begin{pmatrix}\beta_0\\ \beta_1\\ \beta_2\end{pmatrix} = \begin{pmatrix}[y_1] - x_1[y_1, y_2] + x_1x_2[y_1, y_2, y_3]\\ [y_1, y_2] - (x_1 + x_2)[y_1, y_2, y_3]\\ [y_1, y_2, y_3]\end{pmatrix},$$

and so on. And what is quite unfortunate about the form of these solutions is that they're *constantly shifting* to account for the newest higher-order monomial — indeed, we see the same "basic structure" in both of the solutions written above, with β_1 a function of the first divided difference $[y_1, y_2]$ in both cases, and with β_0 a function of $[y_1]$ and $[y_1, y_2]$ in both cases. When we add on a cubic β_3 term, we imagine the solution would look something like

$$\begin{pmatrix}\beta_0\\ \beta_1\\ \beta_2\\ \beta_3\end{pmatrix} = \begin{pmatrix}\underbrace{[y_1] - x_1[y_1, y_2] + x_1x_2[y_1, y_2, y_3]}_{\text{same structure}} + \underbrace{w_0(x_1, x_2, x_3)[y_1, y_2, y_3, y_4]}_{\text{correction}}\\ \underbrace{[y_1, y_2] - (x_1 + x_2)[y_1, y_2, y_3]}_{\text{same structure}} + \underbrace{w_1(x_1, x_2, x_3)[y_1, y_2, y_3, y_4]}_{\text{correction}}\\ \underbrace{[y_1, y_2, y_3]}_{\text{same structure}} + \underbrace{w_2(x_1, x_2, x_3)[y_1, y_2, y_3, y_4]}_{\text{correction}}\\ \underbrace{[y_1, y_2, y_3, y_4]}_{\text{new structure}}\end{pmatrix},$$

which we interpret as "given a new sample (x_4, y_4), takes the basic structure of $(\beta_0, \beta_1, \beta_2)$ derived from $\mathcal{D}_3$ and introduces correction terms: combinations of the new highest order divided difference $[y_1, y_2, y_3, y_4]$ weighted by functions w_j of the sample nodes we've already modeled, x_1, x_2, x_3."

Even though we've asked better questions — ones suggesting a concrete order in which to query the data (first compute $[y_1, y_2, y_3, y_4]$, then $[y_1, y_2, y_3]$, and so on) — we still have to *recompute*

every coefficient any time we introduce a new monomial term. In this sense, the lower-order β_i coefficients are quite unstable.

Yet we can also observe a "kernel of stability" characterizing the coordinates β_i in the solution above. Indeed, we perceive each row as "clean signal plus a contamination":

$$\begin{aligned}
\beta_3 &= \underbrace{[y_1, y_2, y_3, y_4]}_{\text{clean, stable}} \\
\beta_2 &= \underbrace{[y_1, y_2, y_3]}_{\text{stable}} + \underbrace{w_2(x_1, x_2, x_3)[y_1, y_2, y_3, y_4]}_{\text{one correction}} \\
\beta_1 &= \underbrace{[y_1, y_2]}_{\text{stable}} - \underbrace{(x_1 + x_2)[y_1, y_2, y_3] + w_1(x_1, x_2, x_3)[y_1, y_2, y_3, y_4]}_{\text{two cumulative corrections}} \\
\beta_0 &= \underbrace{[y_1]}_{\text{stable}} - \underbrace{x_1[y_1, y_2] + x_1 x_2[y_1, y_2, y_3] + w_0(x_1, x_2, x_3)[y_1, y_2, y_3, y_4]}_{\text{three cumulative corrections}}.
\end{aligned}$$

Supposing we were to identify each β_i by its "most distinguishable atom," we would almost certainly point to the stable ith order divided difference. The corrections — the terms involving higher-order divided differences, weighted by functions of the nodes — are what separate β_j from the "clean" quantity $[y_1, \dots, y_{j+1}]$. As we continue adding more data, we're forced to add additional monomial terms, and we necessarily induce more corrections on the lowest-order coefficients. That is, we'll never stop re-contaminating the lower-order coefficients — again, *this is why* we observe instability in the non-constant coefficients

	β_1	β_2	β_3
p_2	1	—	—
p_3	1/2	1/2	—
p_4	-1	5/2	$-1/2$

But this raises an important, devastatingly obvious question: if every β_j is "really" the jth divided difference $[y_1, \dots, y_{j+1}]$ plus "corrections" that exist only to satisfy the *monomial representation*, then why are we insisting on the monomial representation at all?

Indeed, recall what the polynomial interpolation theorem actually promised: the existence and uniqueness of a *polynomial* $p \in \Pi_{n-1}$ satisfying $p(x_i) = y_i$. Again, at its most basal level, a polynomial is "just a function," so it is "just a rule that assigns inputs $x \in \mathbb{R}$ to outputs $p(x) \in \mathbb{R}$." Intuitively, if we could "list every $x \in \mathbb{R}$ alongside its evaluation $p(x) \in \mathbb{R}$," *never* dictating a "functional form" for the polynomial, and the table would *still characterize* "the same polynomial p." It is immaterial to the polynomial *how* we "come up with this number $p(x)$" — just that we do, and that it is right.

In the same way that a table of evaluations could act as a representation for the polynomial p, its representation in terms of monomials is just that: *a representation*, and nothing more. The subtlety is that, this whole time, we've been operating tacitly under the definition we gave of a polynomial $p \in \Pi_k$ *as a linear combination of monomials*: indeed, recall we *defined* Π_{n-1} to be "the class of polynomials of degree at most $n-1$," where in particular,

$$p \in \Pi_{n-1} \implies \forall x \in \mathbb{R}\colon\ p(x) = \sum_{j=0}^{n-1} a_j x^j \quad \text{for some fixed} \quad a_0, a_1, \dots, a_{n-1} \in \mathbb{R}.$$

That is, the monomial representation of a polynomial $p \in \Pi_{n-1}$ isn't merely a convenient choice — it is, at least as we've set things up so far, baked into the *definition*. So, when we've been saying

"p is a polynomial of degree at most $n-1$," what we've *meant every time* was precisely that "p admits an expansion $\sum a_j x^j$ with real coefficients." This seems to be directly at odds with what we just said though, which is that, in general, a polynomial is "just a class of structured rules" assigning real inputs x to real outputs $p(x)$.

As it turns out, neither perspective is wrong — they both gesture towards a common ground. Indeed, when we defined Π_{n-1}, the class of polynomials of degree at most $n-1$, as $\left\{\sum_{j=0}^{n-1} a_j x^j : \ a_0, \dots, a_{n-1} \in \mathbb{R}\right\}$, we were making two claims at once, and it is worth separating them. The first claim is about *membership*: what does it mean for a function to be "a polynomial of degree at most $n-1$?" This question is answered directly by the monomial representation: a polynomial p is of degree $n-1$ if and only if it can be written $\sum_{j=0}^{n-1} a_j x^j$.

The second claim, which we haven't stated but which we've silently assumed, is about *identity*, and it is what we've spent the last several pages attempting to dismantle. Recall the order of events leading us here: we were given n data points, and, largely based on preexisting intuition, we decided the class of polynomials Π_{n-1} was suitable for interpolating those data points. We then *told you* "how to construct a polynomial" as an expansion in terms of monomials, and from that moment forward, the problem of "finding the polynomial" became synonymous with "finding the monomial coefficients β." But these are *not* the same task! The first asks for a function — a rule, as in our first perspective above. The second asks for a *description* of that function in a *particular language* — the language of monomials. The interpolation theorem guarantees the uniqueness *of the function*. It says *nothing* about privileging any particular description of it.

Formally, what reconciles the two perspectives is the observation that the monomial expansion is *sufficient for membership* — if a function admits a monomial expansion, then it is a polynomial — but it is not *necessary for computation*. In fact, we *already know* a way to do this — given a polynomial in monomial form with j real roots, we can apply the Polynomial Factor Lemma j times, and each iteration will give an *equivalent computational formula* for evaluating $p(x)$ without ever having to evaluate its original monomial form. Similarly, if we're given a polynomial with j real roots, we can "unfactor" it to return an equivalent monomial representation.

It should stand to reason, then, that if we're *given* the factored form of a polynomial and we're *only interested* in evaluating it, then we shouldn't need to redundantly "unfactor" it back into monomial form just to verify it is a polynomial — the factored form should stand as a *certificate of membership* on its own. And if the factored form can certify membership, then so too can *any* representation that can be converted back to monomials when verification is required. The question, then, is not "which representation is a polynomial?" but "which representation is *useful*?"

Changing the Language

Let us pursue such a change of representation and see what emerges. In the previous ribbon, we identified that for the polynomials

$$p_2(x) = \beta_0 + \beta_1 x, \qquad p_3(x) = \beta_0 + \beta_1 x + \beta_2 x^2,$$

their respective coefficients β — which each exist and are unique, given $n = 2$ and $n = 3$ noiseless samples on distinct values of the predictors x_i collected in data $\mathcal{D}_2$ and $\mathcal{D}_3$ — are

$$\begin{pmatrix}\beta_0\\ \beta_1\end{pmatrix} = \begin{pmatrix}[y_1] - x_1[y_1, y_2]\\ [y_1, y_2]\end{pmatrix}, \qquad \begin{pmatrix}\beta_0\\ \beta_1\\ \beta_2\end{pmatrix} = \begin{pmatrix}[y_1] - x_1[y_1, y_2] + x_1 x_2[y_1, y_2, y_3]\\ [y_1, y_2] - (x_1 + x_2)[y_1, y_2, y_3]\\ [y_1, y_2, y_3]\end{pmatrix},$$

where

$$[y_1] = y_1, \qquad [y_1, y_2] = \frac{y_2 - y_1}{x_2 - x_1}, \qquad \text{and} \qquad [y_1, y_2, y_3] = \frac{\frac{y_3 - y_2}{x_3 - x_2} - \frac{y_2 - y_1}{x_2 - x_1}}{x_3 - x_1} = \frac{[y_2, y_3] - [y_1, y_2]}{x_3 - x_1}$$

are the zeroth, first, and second divided differences, respectively.

Now, notice that, in our recapitulation, we subconsciously made a move that gestures toward the broader point of the matter: when we listed the divided differences $[y_1]$, $[y_1, y_2]$, and $[y_1, y_2, y_3]$ at the bottom, we listed them *once* — even though they appear in *both* solutions. The divided differences are not "the $n = 2$ divided differences" and "the $n = 3$ divided differences"; they are *the same quantities*, computed from the same data, serving as common ingredients in both formulas. What changes between the two columns is not the divided differences, but the *corrections around them* — the $-x_1[y_1, y_2]$ in $\beta_0^{(2)}$ (with the superscript denoting the model p_2) becomes $-x_1[y_1, y_2] + x_1 x_2 [y_1, y_2, y_3]$ in $\beta_0^{(3)}$, and so on. But again, it's not that the corrections change entirely erratically — they maintain all previous corrections, just "appending a new one" for the newest higher order term.

But let us observe just one thing. Working within the linear model

$$\begin{pmatrix} \beta_0 \\ \beta_1 \end{pmatrix} = \begin{pmatrix} [y_1] - x_1[y_1, y_2] \\ [y_1, y_2] \end{pmatrix},$$

we have that

$$\beta_1 = [y_1, y_2] \implies \beta_0 = [y_1] - x_1[y_1, y_2] = y_1 - \beta_1 x_1.$$

In particular, we can plug the expression for β_0 back in to the general form of the (linear) polynomial model p_2:

$$p_2(x) = \beta_0 + \beta_1 x = (y_1 - \beta_1 x_1) + \beta_1 x = y_1 + [y_1, y_2](x - x_1).$$

That is,

$$p_2(x) = y_1 + \frac{y_2 - y_1}{x_2 - x_1}(x - x_1).$$

Supposing we were to write

$$m := [y_1, y_2] = \frac{y_2 - y_1}{x_2 - x_1},$$

this reads

$$p_2(x) = y_1 + m(x - x_1).$$

We've just derived the **point-slope form** of a line through (x_1, y_1) with slope $m = \frac{y_2 - y_1}{x_2 - x_1}$ (set so that it also falls on the point (x_2, y_2)). Observe that the monomial coefficients β_0 and β_1 have dissolved — what remains is the data point y_1, the divided difference/linear slope m, and the shifted input variable $(x - x_1)$.

One is often taught the point-slope form of a line in elementary algebra. On the other hand, one is almost never taught the analogous "point-slope-curvature form of a parabola"; yet, the same procedure that led us to point-slope form produces it as well, as we will see below. And indeed, we're led to similar analogues for higher order derivatives, each with a resemblant form to the well-known point-slope. But, rather than state these piecemeal, let us attempt to understand the procedure from another familiar light: namely, the Polynomial Factor Lemma.

Suppose we restate the goal of fitting the linear model above as follows. First, we seek a polynomial $p_2 \in \Pi_1$, the class of linear models, satisfying $p_2(x_1) = y_1$ and $p_2(x_2) = y_2$. Imagine then that we somewhat arbitrarily constrain the line $p_2 \in \Pi_1$ to satisfy $p_2(x_1) = y_1$. Define

$$r_1(x) = p_2(x) - y_1.$$

By construction, we have

$$r_1(x_1) = p_2(x_1) - y_1 = y_1 - y_1 = 0,$$

so that, by the factor lemma, there exists a (constant) polynomial $c_1 \in \Pi_0$ such that

$$r_1(x) = c_1(x - x_1), \qquad \text{i.e.,} \qquad p_2(x) = y_1 + c_1(x - x_1).$$

We're quick to notice the structural parallel with the proof we gave of the polynomial root bound. There, we were given a polynomial p with a *known* root at $x = a$, and the factor lemma told us $p(x) = (x - a)q(x)$ — in other words, we found that

$$p \text{ vanishes at } x = a \iff (x - a) \text{ factors } p.$$

Here, the situation is reversed, but the *logic* remains identical. Indeed, at the start, we *don't* have a polynomial with a known root — neither x_1 nor x_2 are *roots* of p_2, they are *constraints* (or, more aptly, points which are involved in the constraints that $p_2(x_1) = y_1$ and $p_2(x_2) = y_2$). Yet, these constraints are *equations* — which means they can be *rearranged to equal zero*. This is why we define $r_1 := p_2 - y_1$ to measure the residual between y_1 and p_2's output at a point $x \in \mathbb{R}$; it is explicitly so that

$$r_1 \text{ vanishes at } x = x_1 \iff (x - x_1) \text{ factors } r_1,$$

and we obtain

$$r_1(x) = c_1(x - x_1) \qquad (\text{since } (x - x_1) \text{ factors } r_1),$$

which, upon plugging in $r_1 = p_2 - y_1$ and isolating p_2, gives the at this point familiar equation

$$p_2(x) = y_1 + c_1(x - x_1).$$

On the other hand, we've perhaps created enough distance between ourselves and the matter that you've forgotten we know what c_1 is — it is the first divided difference

$$c_1 = m = [y_1, y_2] = \frac{y_2 - y_1}{x_2 - x_1}.$$

This is forced just by comparing the forms of the representations of p_2 — yet, we could have also derived this explicitly using the other constraint $p_2(x_2) = y_2$:

$$y_2 = p_2(x_2) = y_1 + c_1(x_2 - x_1) \iff c_1 = \frac{y_2 - y_1}{x_2 - x_1}.$$

Furthermore, we simply note that haven't heard from our monomial coordinates β_j in a while. This is precisely the point.

Now, we repeat the construction one level higher — i.e., we want a factored form for the quadratic model $p_3 \in \Pi_2$ given samples $i = 1, 2$ as above and the new $(x_3, y_3) = (3, 8)$. We *immediately* notice that, crucially, whatever $p_3 \in \Pi_2$ is, it *must agree* with p_2 at x_1 and x_2. For this reason, we may *start* by defining

$$s(x) := p_3(x) - p_2(x).$$

Since $s = p_3 - p_2 \in \Pi_2$ (cf. Polynomial closure under linear combinations), and

$$s(x_1) = p_3(x_1) - p_2(x_1) = y_1 - y_1 = 0, \qquad s(x_2) = p_3(x_2) - p_2(x_2) = y_2 - y_2 = 0,$$

the situation begins to look analogous to the proof of the root bound yet again. Indeed, s is a polynomial of degree at most 2 with two known roots x_1 and x_2, we apply the factor lemma twice, once to obtain

$$s(x) = (x - x_1)t(x), \qquad t \in \Pi_1,$$

and again (after perhaps noting that $t(x_2) = \frac{s(x_2)}{x_2 - x_1} = 0$ since $x_1 \neq x_2$), we obtain

$$s(x) = (x - x_1)(x - x_2)c_2, \qquad c_2 \in \Pi_0 \equiv \mathbb{R}.$$

Plugging back into $p_3 = p_2 + s$:

$$p_3(x) = p_2(x) + c_2(x - x_1)(x - x_2) = y_1 + [y_1, y_2](x - x_1) + c_2(x - x_1)(x - x_2).$$

This is the "point-slope-curvature form" we alluded to earlier. Indeed, y_1 is the point, $[y_1, y_2]$ is the discrete analogue to the slope, and we can recover the discrete analogue to the curvature, c_2, by utilizing the last constraint $p_3(x_3) = y_3$:

$$y_3 = p_3(x_3) = y_1 + [y_1, y_2](x_3 - x_1) + c_2(x_3 - x_1)(x_3 - x_2),$$

from which we obtain

$$c_2 = \frac{y_3 - y_1 - [y_1, y_2](x_3 - x_1)}{(x_3 - x_1)(x_3 - x_2)} = \frac{\dfrac{y_3 - y_1}{x_3 - x_1} - [y_1, y_2]}{x_3 - x_2} = \frac{[y_1, y_3] - [y_1, y_2]}{x_3 - x_2};$$

i.e., c_2 is the second divided difference $c_2 = [y_1, y_2, y_3]$, and p_3 becomes

$$p_3(x) = y_1 + [y_1, y_2](x - x_1) + [y_1, y_2, y_3](x - x_1)(x - x_2).$$

The pattern is now fully visible, and it generalizes without modification. To construct $p_4 \in \Pi_3$ through all four points of $\mathcal{D}_4$: define $u(x) := p_4(x) - p_3(x)$, observe that u vanishes at x_1, x_2, and x_3 (since p_3 and p_4 agree on the data they share), apply the factor lemma three times to obtain $u(x) = c_3(x - x_1)(x - x_2)(x - x_3)$, and impost $p_4(x_4) = y_4$ to determine $c_3 = [y_1, y_2, y_3, y_4]$. At each stage, the correction $p_n - p_{n-1}$ must vanish at all previous nodes x_i — the factor lemma leaves no alternative but the product, $\prod_{i=1}^{n-1}(x - x_i)$ — and the newest data $p_n(x_n) = y_n$ determines *only* the newest coefficient, $c_n = [y_1, \dots, y_{n-1}, y_n]$. Indeed, information propagates *forward* through the coefficients (divided differences) c_j; this seems useful, and at this point, we'd like to articulate precisely why it is.

Remark.

We note that the Newton construction here has completed the deferred proof of existence we gestured toward in the Polynomial Interpolation theorem. Indeed, uniqueness was established earlier via the root bound, and the Newton interpolant

$$p(x) = \sum_{j=0}^{n-1} [y_1, \dots, y_{j+1}] \prod_{i=1}^{j} (x - x_i) \in \Pi_{n-1}$$

is the explicit construction of a polynomial $p \in \Pi_{n-1}$ satisfying $p(x_i) = y_i$ for each $i = 1, \dots, n$.

Developing a New Basis

We have, over the course, of several ribbons, arrived at two distinct representations of the same interpolating polynomial — one in terms of monomials, and the other in terms of factored products — and we have observed, both concretely and in general, that the latter factored form is *stable under data augmentation*, while the monomial form is not. We now seek to formalize these notions.

Definition 3.2.4 (Basis for Π_k)

A collection of $k+1$ polynomials

$$\mathcal{B} = \{\psi_0, \psi_1, \dots, \psi_k\} \subset \Pi_k$$

is called a **basis** for Π_k if every polynomial $p \in \Pi_k$ can be written as a linear combination of the form

$$p(x) = c_0\psi_0(x) + c_1\psi_1(x) + \cdots + c_k\psi_k(x) = \sum_{j=0}^{k} c_j\psi_j(x)$$

for a *unique* choice of coefficients $(c_0, c_1, \dots, c_k) \in \mathbb{R}^{k+1}$.

That a polynomial $p \in \Pi_k$'s representation under a given basis *exists* (for every $p \in \Pi_k$) and that it is *unique* (no two distinct coefficient vectors under a fixed basis product the same polynomial) are both essential: the first ensures the basis $\mathcal{B}$ is expressive enough to describe all of Π_k, and the second ensures that the coefficients under $\mathcal{B}$ are well-defined and unambiguous.

Indeed, there are two bases we've met so far for the class of polynomials.

Theorem 3.2.5 (Two Bases for Π_k)

Let

$$\Pi_k := \{\text{polynomials of degree at most } k\}.$$

Define the set

$$\mathcal{M}_k = \{1, x, x^2, \dots, x^k\},$$

and given fixed, distinct nodes $x_1, \dots, x_k \in \mathbb{R}$ with $x_1 \neq \cdots \neq x_k$, define

$$\mathcal{N}_k := \left\{1, (x-x_1), (x-x_1)(x-x_2), \dots, \prod_{i=1}^{k}(x-x_i)\right\}.$$

Then both $\mathcal{M}_k$ and $\mathcal{N}_k$ are bases of Π_k, the class of polynomials of degree at most k. In particular, we call $\mathcal{M}_k$ the **monomial basis** of Π_k, and $\mathcal{N}_k$ the **Newton basis** of Π_k.

Proof:

To show that $\mathcal{M}_k$ and $\mathcal{N}_k$ are bases of Π_k, we need to show that (i) every polynomial admits a representation under $\mathcal{M}_k/\mathcal{N}_k$ (existence), and (ii) this representation is unambiguous (uniqueness). That this follows for the monomial basis $\mathcal{M}_k$ is essentially

definitional: every $p \in \Pi_k$ admits a monomial expansion $\sum c_j x^j$ just by our definition of Π_k (showing existence (i)), and that this representation is unique follows by the root bound: if $p(x) = \sum a_j x^j$ and $p(x) = \sum b_j x^j$ for all $x \in \mathbb{R}$, then subtracting both sides gives

$$\sum_{j=0}^{k} (a_j - b_j) x^j = 0 \quad \forall x \in \mathbb{R} \iff a_j = b_j, \quad j = 0, 1, \ldots, k.$$

This means that two representations of p under $\mathcal{M}_k$ must have the same coordinates $a_j = b_j$ for every j; i.e., the coordinates are equal, and the expansion is unique.

That $\mathcal{N}_k$ is a basis follows from the iterative construction developed in the previous ribbon. We encourage the reader to reproduce the proof of existence. That a polynomial $p \in \Pi_k$'s representation under the Newton basis is unique, then, follows by observing that if

$$\sum_{j=0}^{k} c_j \psi_j(x_i) = \sum_{j=0}^{k} d_j \psi_j(x_i), \qquad i = 1, \ldots, k+1,$$

then, going term by term:

(i) At x_1: every Newton basis function besides ψ_0 vanishes since $\psi_j(x_1) = 0$ for $j \geq 1$. The equation for $i = 1$ reduces to $c_0 = d_0$.

(ii) At x_2: $\psi_j(x_2) = 0$ for $j \geq 2$, so the equation reduces to $c_0 + c_1(x_2 - x_1) = d_0 + d_1(x_2 - x_1)$. We already know that $c_0 = d_0$, so this simplifies to $c_1(x_2 - x_1) = d_1(x_2 - x_1)$. Since $x_2 \neq x_1$, we conclude $c_1 = d_1$.

(iii) At x_3: The same logic forces $c_2 = d_2$, using only that $c_0 = d_0$, $c_1 = d_1$, and $(x_3 - x_1)(x_3 - x_2) \neq 0$.

The pattern continues: at each node x_{j+1}, all Newton basis functions of index $> j$ vanish, all coefficients of index $< j$ have already been shown to agree, and the jth coefficient is isolated on both sides — and they're forced to agree, since $\prod_{i=1}^{j} (x_{j+1} - x_i) \neq 0$ whenever the x_i are distinct.

‖

Remark.

It is worth noting that the two uniqueness arguments above are genuinely different in character. The reason why is, essentially, that we stated "what the zero polynomial $0 \in \Pi_k$ is" in terms of the monomial basis: in particular,

$$p \equiv 0 \in \Pi_k \iff \text{inside } p(x) = \sum_{j=0}^{k} a_k x^k, \quad a_0 = a_1 = \cdots = a_k = 0.$$

That is, we've *seemingly defined* the zero polynomial in Π_k relative to the monomial basis, requiring its coordinates in this basis to vanish in order to *be* the zero polynomial. But this goes the *other way* as well: if we know a polynomial *is* the zero polynomial, then its monomial coefficients a_j must vanish for all j. This is why when we deduce $p(x) = \sum a_j x^j = \sum b_j x^j$ and rearrange to $\sum (a_j - b_j) x^j = 0$, we can conclude immediately that each term's coefficients $a_j = b_j$.

On the other hand, in the Newton basis, if we started in a similar way by saying $p(x) = \sum c_j\psi_j(x) = \sum d_j\psi_j(x)$ and subtract $\sum(c_j - d_j)\psi_j(x) = 0$, we can still conclude that their difference is identically zero, but we can *not* use this to immediately conclude $c_j = d_j$ for all j since the ψ_j are not monomials, and we only know the correspondence between "zero polynomial $\iff$ vanishing coordinates" under the monomial basis. Knowing that $\sum(c_j - d_j)\psi_j(x) = 0$ tells us the expression on the left, *when expanded in monomials*, has all zero coefficients. And we could have proceeded by "re-expanding" the expression on the left in terms of monomials (by expanding the ϕ_j and collecting terms), showing it forces the same equivalence; but the node-by-node argument facilitates the conclusion directly, without ever converting to monomials.

What the node-by-node argument above reveals, though, is something we should recognize: it is precisely *forward substitution* through a lower-triangular system. Indeed, to establish uniqueness of representation under the Newton basis, we argued in a sequential manner: at x_1, only ψ_0 contributed, forcing $c_0 = d_0$. At x_2, only ψ_0 and ψ_1 contributed, and with $c_0 = d_0$ from the first round, we force $c_1 = d_1$. Each step relies on the previous ones, and no step could be disturbed by a later one.

This is the *triangular structure* we observed concretely in the parable, now appearing in the abstract. Though we've done so previously, we take the opportunity to once more reformalize what will be a critical notion moving forward.

Definition 3.2.6 (Design Matrix Relative to a Basis)

Given a basis $\mathcal{B} = \{\psi_0, \psi_1, \dots, \psi_k\}$ of Π_k, suppose the nodes $x_1, \dots, x_n$ are distinct. Then, the **design matrix** of $\mathcal{B}$ at the nodes $x_1, \dots, x_n$ is

$$\Psi_{\mathcal{B}} := \begin{pmatrix} \psi_0(x_1) & \psi_1(x_1) & \cdots & \psi_k(x_1) \\ \psi_0(x_2) & \psi_1(x_2) & \cdots & \psi_k(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \psi_0(x_n) & \psi_1(x_n) & \cdots & \psi_k(x_n) \end{pmatrix} \in \mathbb{R}^{n\times(k+1)},$$

Remark.

Here, we index the first basis function as ψ_0; this indexing is nothing more than a notational choice, and we certainly feel free to re-index the first basis member by, say ψ_1, starting at the subscript $j = 1$ instead of $j = 0$, if it serves to clarify matters intuitively.

In the square case where $n = k + 1$, the interpolation problem $p(x_i) = y_i$, $i = 1, \dots, n$ becomes the linear system

$$\mathbf{y} = \Psi\mathbf{c},$$

where $\mathbf{c} = (c_0, \dots, c_k)$ are the coordinates of $p(x)$ in the basis $\mathcal{B}$. This is, for instance, how we proceeded in deriving the Newton coefficients $c_j = [y_1, \dots, y_j]$ — indeed, under the Newton basis

$\mathcal{N}_k = \{1, (x - x_1), \dots, \prod_{i=1}^{k}(x - x_i)\}$, the design matrix is

$$\Psi_{\mathcal{N}_k} = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ 1 & (x_2 - x_1) & 0 & \cdots & 0 \\ 1 & (x_3 - x_1) & (x_3 - x_1)(x_3 - x_2) & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & (x_n - x_1) & \cdots & \cdots & \prod_{i=1}^{k}(x_n - x_i) \end{pmatrix} \in \mathbb{R}^{n \times (k+1)}.$$

When we set

$$\mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, \qquad \mathbf{c} = \begin{pmatrix} c_0 \\ c_1 \\ \vdots \\ c_k \end{pmatrix},$$

this leads to the system

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ 1 & (x_2 - x_1) & 0 & \cdots & 0 \\ 1 & (x_3 - x_1) & (x_3 - x_1)(x_3 - x_2) & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & (x_n - x_1) & \cdots & \cdots & \prod_{i=1}^{k}(x_n - x_i) \end{pmatrix} \begin{pmatrix} c_0 \\ c_1 \\ \vdots \\ c_k \end{pmatrix},$$

where we can sequentially solve for the "point-slope-higher derivative" forms by

$$c_0 = y_1 = [y_1]; \qquad c_0 + c_1(x_2 - x_1) = y_2 \implies c_1 = \frac{y_2 - y_1}{x_2 - x_1} = [y_1, y_2];$$

and so on.

Now, by contrast, the monomial basis $\mathcal{M}_k = \{1, x, x^2, \dots, x^k\}$ yields the following design:

$$\Phi := \Psi_{\mathcal{M}_k} = \begin{pmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^k \\ 1 & x_2 & x_2^2 & \cdots & x_2^k \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^k \end{pmatrix} \in \mathbb{R}^{n \times (k+1)}.$$

We call Φ the **Vandermonde matrix**. The Vandermonde matrix is generally **dense** — i.e., the entries of a dense matrix are not equipped with much, if any, identifiable structure, in contrast with, say, a triangular matrix like the design yielded by the Newton basis.

At this point, we have seen a few matrix structures: the monomial basis $\mathcal{M}_k$ yielding the dense design $\Psi_{\mathcal{M}_k}$, and the Newton basis $\mathcal{N}_k$ yielding a lower-triangular design $\Psi_{\mathcal{N}_k}$. We ultimately prefer the latter triangular structure in $\Psi_{\mathcal{N}_k}$ as produced by the Newton basis precisely because it is *stable under data augmentation*, enabling us to solve the system via *forward-substitution* without destabilizing coefficients derived from previous waves of data. Recall, we arrived at the Newton basis through extensive polynomial reasoning: residuals, the factor lemma, divided differences, back-substitution formulas rearranged and simplified. This was productive, but it was also laborious — and it was specific to the Newton basis.

Now that we have the formal language, we can reverse the process that produced Newton: i.e., instead of deriving a basis through polynomial reasoning and *observing* the corresponding matrix structure, we can *postulate* a desirable matrix structure and reverse-engineer a basis producing it as its design. The factor lemma, which did all the heavy lifting for Newton, turns out to do the heavy lifting here as well.

Example 3.2.7 (Identity Structure):

First, suppose we ask: is there a basis $\mathcal{L}_n = \{\ell_1, \ell_2, \dots, \ell_n\}$ for Π_{n-1} whose design matrix at the nodes $x_1, \dots, x_n$ is the **identity matrix**

$$\Psi_{\mathcal{L}_n} = I_n := \begin{pmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{pmatrix} = \operatorname{diag}(1, 1, \dots, 1) \in \mathbb{R}^{n \times n}?$$

(Observe that we index the basis functions ℓ_j by the sample size n as opposed to prescribing their own index k.)

Given data $\mathcal{D}_n = \{(x_i, y_i)\}_{i=1}^n$, the coefficients $\mathbf{c} = (c_1, c_2, \dots, c_n)^\top$ characterizing the solution under the identity design, should such a set of coefficients exist, would be forced to satisfy $\mathbf{y} = \Psi_{\mathcal{L}_n} \mathbf{c}$, or

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{pmatrix},$$

which would hold if and only if

$$c_j = y_j, \qquad j = 1, 2, \dots, n.$$

So, if we were able to identify functions $\ell_j \in \mathcal{L}_n$ such that

$$\Psi_{\mathcal{L}_n} = \begin{pmatrix} \ell_1(x_1) & \ell_2(x_1) & \cdots & \ell_n(x_1) \\ \ell_1(x_2) & \ell_2(x_2) & \cdots & \ell_n(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \ell_1(x_n) & \ell_2(x_n) & \cdots & \ell_n(x_n) \end{pmatrix} \stackrel{?}{=} \begin{pmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{pmatrix} = I_n,$$

what we would have is that the unique polynomial p of degree at most $n-1$ interpolating the data $\mathcal{D}_n$ admits a representation under the basis $\mathcal{L}_n$ given by

$$p(x) = \sum_{j=1}^n c_j \ell_j(x) = y_1 \ell_1(x) + y_2 \ell_2(x) + \cdots + y_n \ell_n(x).$$

(Verify that p would interpolate $\mathcal{D}_n$ since $\ell_j(x_j) = 1$ for all j and $\ell_j(x_i) = 0$ for $i \neq j$ by construction.)

The only question, then, is whether we *can* identify such functions ℓ_j such that

$$\Psi_{\mathcal{L}_n} = I_n \iff \ell_j(x_i) = \delta_{ij} := \begin{cases} 1 & i = j, \\ 0 & i \neq j. \end{cases}$$

Indeed we can, and the factor lemma tells us how. Fix a node x_j. The function ℓ_j must vanish at every node x_i with $i \neq j$ — that is, ℓ_j must have $n-1$ prescribed roots (notice how this differs from the triangular Newton basis — here, *every* basis function ℓ_j inherits all $n-1$ roots from the data, not just the preceding $j-1$.)

Since $\ell_j \in \Pi_{n-1}$ is a polynomial of degree at most $n-1$, the root bound tells us this is the most roots it can possibly have without vanishing identically to the zero polynomial $0 \in \Pi_{n-1}$.

The factor lemma, applied $n-1$ times (exactly as in the proof of the root bound), forces

$$\ell_j(x) = c\prod_{\substack{i=1\\ i\neq j}}^{n}(x - x_i)$$

for some constant $c \in \mathbb{R}$. The product on the right already accounts for $n-1$ factors, placing ℓ_j at degree exactly $n-1$. What remains a free parameter is the constant c, which we can determine using the last nontrivial constraint that $\ell_j(x_j) = 1$:

$$\ell_j(x_j) = c\prod_{\substack{i=1\\ i\neq j}}^{n}(x_j - x_i) = 1 \implies c = \frac{1}{\prod_{\substack{i=1\\ i\neq j}}^{n}(x_j - x_i)},$$

which yields our final expression for the basis functions $\ell_j \in \mathcal{L}_n$:

$$\ell_j(x) = \prod_{\substack{i=1\\ i\neq j}}^{n}\frac{x - x_i}{x_j - x_i}, \qquad j = 1, 2, \dots, n.$$

Indeed, we have just derived a new basis for Π_{n-1}:

Theorem 3.2.8 (Lagrange Basis)

Given distinct nodes $x_1, \dots, x_n \in \mathbb{R}$, the **Lagrange basis** for Π_{n-1} relative to these nodes is the collection

$$\mathcal{L}_n = \{\ell_1, \ell_2, \dots, \ell_n\},$$

where

$$\ell_j(x) := \prod_{\substack{i=1\\ i\neq j}}^{n}\frac{x - x_i}{x_j - x_i}, \qquad j = 1, 2, \dots, n.$$

The design matrix of $\mathcal{L}_n$ at the nodes is the identity matrix, $\Psi_{\mathcal{L}_n} = I_n$.

(We defer the formal proof that $\mathcal{L}_n$ defines a formal basis of Π_{n-1} to an exercise. Indeed, it is entirely analogous to how we argued for the Vandermonde and Newton bases.)

The Lagrange basis is, in a sense, the maximally convenient representation for *solving* the interpolation problem: the design matrix $\Psi_{\mathcal{L}_n}$ is the identity I_n, so the coefficients c_j are just the data y_j, and there is nothing to compute. But the computational convenience comes at an administrative cost: the Lagrange basis is, in another sense, maximally *inconvenient* under *data augmentation*. The identity matrix has no structure that extends as new data arrives: whenever we receive a new x_{n+1}, every existing basis function ℓ_j — defined relative to a product involving *every node* in the data — must be redefined. Adding one data point changes every basis function — just like how, in the monomial basis, every coefficient *besides* the constant $\beta_0 = 2$ shifted around as we added new data. The constancy of the intercept $\beta_0 = 2$ under the monomial basis was an artifact of the sample point $(x_1, y_1) = (0, 2)$ present in every sample; the difference is that Lagrange is immune to even such "lucky cases of coefficient stability" emerging as artifacts of happenstance. (In some ways, this is still better than the "unpredictable instability" we

encountered under the monomial basis; at least under the Lagrange basis, we *know* the coefficients will be *unstable everywhere, every time.*)

‖

We now consider yet another structure well worth our exploration.

Example 3.2.9 (Orthogonal Structure):
Suppose we ask: is there a basis $\mathcal{O}_n = \{\omega_1, \omega_2, \dots, \omega_n\}$ for Π_{n-1} whose design matrix at the nodes $x_1, \dots, x_n$ has *orthogonal* columns? That is, for which

$$\Psi_{\mathcal{O}_n}^\top \Psi_{\mathcal{O}_n} = \text{diagonal?}$$

Denoting the jth column of $\Psi_{\mathcal{O}_n}$ by $\boldsymbol{\omega}_j = (\omega_j(x_1), \dots, \omega_j(x_n))^\top \in \mathbb{R}^n$, the condition is

$$\boldsymbol{\omega}_j^\top \boldsymbol{\omega}_m = \sum_{i=1}^{n} \omega_j(x_i)\, \omega_m(x_i) = 0, \qquad j \neq m :$$

distinct basis functions, when evaluated at the nodes and stacked into column vectors, must be orthogonal in $\mathbb{R}^n$. If this held, then each coefficient c_j could be extracted independently, via

$$c_j = \frac{\boldsymbol{\omega}_j^\top \mathbf{y}}{\boldsymbol{\omega}_j^\top \boldsymbol{\omega}_j} = \frac{\displaystyle\sum_{i=1}^{n} \omega_j(x_i)\, y_i}{\displaystyle\sum_{i=1}^{n} \omega_j(x_i)^2}.$$

No system to solve. No substitution of any kind, neither forward nor backward. Each coefficient is an inner product of the data with the jth basis function's column, normalized by the basis function's squared norm. And — unlike the Lagrange basis, where the coefficients were also immediately available — no coefficient depends on any other coefficient's value.
Can we construct such a basis? Indeed, the procedure is classical: take the monomials $\{1, x, x^2, \dots\}$ and **orthogonalize** them with respect to the discrete inner product

$$\langle f, g \rangle_n := \sum_{i=1}^{n} f(x_i)\, g(x_i).$$

(One should take care to verify that $\langle \cdot, \cdot \rangle_n$ constitutes a valid inner product on the space of real functions defined on the finitely many nodes $\{x_1, \dots, x_n\}$.)
Start with $\omega_1(x) = 1$. Then define $\omega_2(x) = x - \text{proj}_{\omega_1}(x)$, where $\text{proj}_{\omega_1}(x) = \frac{\langle x, \omega_1 \rangle_n}{\langle \omega_1, \omega_1 \rangle_n} \cdot \omega_1(x) = \frac{\sum x_i}{n}$ is the projection of x onto the constant function at the nodes — i.e., the sample mean $\overline{x}$. So $\omega_2(x) = x - \overline{x}$: the "centered" linear term. Continue: $\omega_3(x) = x^2 - \text{proj}_{\omega_1}(x^2) - \text{proj}_{\omega_2}(x^2)$, which subtracts from x^2 its components along the constant and centered-linear directions. At each step, the new basis function is the "residual" of the next monomial after removing everything that correlates with the previously constructed functions.
This is the **Gram-Schmidt process**, applied to the monomial sequence $\{1, x, x^2, \dots, x^{n-1}\}$ under the discrete inner product $\langle \cdot, \cdot \rangle_n$. The resulting basis $\mathcal{O}_n = \{\omega_1, \dots, \omega_n\}$ consists of polynomials that are orthogonal when evaluated at the nodes — and one can verify that the design matrix $\Psi_{\mathcal{O}_n}$ has orthogonal columns by construction.

‖

Before we conclude the ribbon, we briefly recall our throughline: indeed, the last example should be contrasted with something the reader who has worked through Chapter 2 will recognize immediately. The coefficient formula

$$c_j = \frac{\displaystyle\sum_{i=1}^{n} \omega_j(x_i)\, y_i}{\displaystyle\sum_{i=1}^{n} \omega_j(x_i)^2}$$

is, coefficient for coefficient, the discrete analogue of

$$a_n = \frac{\langle f, \cos(nx)\rangle}{\|\cos(nx)\|^2} = \frac{1}{\pi}\int_{-\pi}^{\pi} f(x)\cos(nx)\,dx$$

— the Fourier coefficient formula. And indeed, at equispaced nodes $x_i = -\pi + 2\pi i/N$, the trigonometric functions $\mathcal{T} = \{1, \cos(x), \sin(x), \cos(2x), ...\}$ produce a design matrix whose columns satisfy exactly the orthogonality condition we postulated:

$$\sum_{i=1}^{N} \cos(mx_i)\cos(nx_i) = 0, \qquad m \neq n.$$

The trigonometric basis achieves this without any Gram-Schmidt computation — the orthogonality is *intrinsic* to the functions, a consequence of the periodicity and symmetry of sines and cosines at equispaced points. This is the discrete manifestation of the continuous orthogonality relations

$$\int_{-\pi}^{\pi} \cos(mx)\cos(nx)\,dx = 0, \qquad m \neq n,$$

that were the foundation of the entire Fourier development in the previous chapter. The data we're considering here in $\mathcal{D}_k$ are *not* equispaced, though; can you figure out why this is an issue for the Fourier basis?

‖

Assessing Basis Behavior

To close the current development, we revisit the example from the first half of the parable and demonstrate the behavior of solutions under several bases of Π_{n-1}. Indeed, recall that in the running example, we received three waves of data

$$\mathcal{D}_2 = \{(0,2),(1,3)\}, \qquad \mathcal{D}_3 = \{(0,2),(1,3),(3,8)\}, \qquad \mathcal{D}_4 = \{(0,2),(1,3),(3,8),(4,6)\},$$

and computed — in the monomial basis $\mathcal{M}_k = \{1, x, x^2, \dots, x^{k-1}\}$ for $k = 2, 3, 4$ — the interpolating polynomials

$$p_2^{(\mathcal{M})}(x) = 2 + x, \qquad p_3^{(\mathcal{M})}(x) = 2 + \frac{1}{2}x + \frac{1}{2}x^2, \qquad p_4^{(\mathcal{M})}(x) = 2 - x + \frac{5}{2}x^2 - \frac{1}{2}x^3.$$

These polynomials are the objects of interest, and we seek to verify that every basis we consider produces exactly these same three polynomials — thereby demonstrating the *uniqueness* component of the Polynomial Interpolation Theorem.

Example 3.2.10 (Newton Basis):
To start, we take as our basis of Π_{n-1} the Newton basis

$$\mathcal{N}_{n-1} = \left\{ 1, (x-x_1), (x-x_1)(x-x_2), \dots, \prod_{i=1}^{n-1}(x-x_i) \right\}.$$

Under $\mathcal{N}_{n-1}$, the interpolating polynomial is

$$p_{n-1}^{(\mathcal{N})}(x) = c_0 \cdot 1 + c_1(x-x_1) + c_2(x-x_1)(x-x_2) + \cdots + c_{n-1}\prod_{i=1}^{n-1}(x-x_i).$$

We know that the coefficients c_j are just the divided differences

$$c_j = [y_1, \dots, y_{j+1}], \qquad j = 0, 1, \dots, n-1,$$

as derived in the previous ribbon. Hence, for the interpolating polynomial of $\mathcal{D}_2$ with $(x_1, y_1) = (0, 2)$ and $(x_2, y_2) = (1, 3)$, we obtain

$$c_0 = [y_1] = y_1 = 2, \qquad c_1 = [y_1, y_2] = \frac{y_2 - y_1}{x_2 - x_1} = \frac{3-2}{1-0} = 1.$$

We thus assemble

$$p_2^{(\mathcal{N})}(x) = c_0 \cdot 1 + c_1(x-x_1) = 2 \cdot 1 + 1 \cdot (x-0) = 2 + x.$$

This is, of course, the *same representation* as what we recovered under the monomial basis $\mathcal{M}$. The difference is that, now, suppose we go to fit the interpolating polynomial through $\mathcal{D}_3$ with the same data for $j = 1, 2$ and the new sample $(x_3, y_3) = (3, 8)$. Then, the interpolating polynomial $p_3^{(\mathcal{N})}$ under the Newton basis *retains these coefficients* from $p_2^{(\mathcal{N})}$:

$$c_0 = [y_1] = 2, \qquad c_1 = [y_1, y_2] = 1,$$

and we only have to compute the new coefficient

$$c_2 = [y_1, y_2, y_3] = \frac{[y_2, y_3] - [y_1, y_2]}{x_3 - x_1} = \frac{\frac{5}{2} - 1}{3-0} = \frac{1}{2}$$

after computing $[y_2, y_3] = \frac{8-3}{3-1} = \frac{5}{2}$.

Thus, at $n = 3$, the polynomial p_3 represented under the Newton basis is

$$p_3^{(\mathcal{N})}(x) = c_0 + c_1(x-x_1) + c_2(x-x_1)(x-x_2) = 2 + x + \frac{1}{2}x(x-1).$$

We can verify upon expanding the factor of $\frac{1}{2}x(x-1)$ that this agrees with the monomial basis $p_3^{(\mathcal{M})}(x) = 2 + \frac{1}{2}x + \frac{1}{2}x^2$.

We can verify that for the model $p_4^{(\mathcal{N})}$, we obtain c_0, c_1, and c_2 as above, along with

$$c_3 = [y_1, y_2, y_3, y_4] = -\frac{1}{2},$$

yielding

$$p_4^{(\mathcal{N})} = 2 + x + \frac{1}{2}x(x-1) - \frac{1}{2}x(x-1)(x-3).$$

In particular, we reveal that once again, Newton leaves the earlier coefficients untouched; data augmentation under $\mathcal{N}$ injects the newest contribution only into the newest coefficient. Gathering the four coefficients into a table, we observe

$$p^{(\mathcal{N})}(x) = \sum_{j=0}^{k} c_j \, \psi_j(x)$$

c	c_0	c_1	c_2	c_3
$\mathcal{D}_2$	2	1	—	—
$\mathcal{D}_3$	2	1	$\frac{1}{2}$	—
$\mathcal{D}_4$	2	1	$\frac{1}{2}$	$-\frac{1}{2}$

ψ	ψ_0	ψ_1	ψ_2	ψ_3
$\mathcal{D}_2$	1	x	—	—
$\mathcal{D}_3$	1	x	$x(x-1)$	—
$\mathcal{D}_4$	1	x	$x(x-1)$	$x(x-1)(x-3)$

We're quick to contrast this with the monomial coefficient table we computed earlier:

$$p^{(\mathcal{M})}(x) = \sum_{j=0}^{k} \beta_j \, \phi_j(x)$$

β	β_0	β_1	β_2	β_3
$\mathcal{D}_2$	2	1	—	—
$\mathcal{D}_3$	2	$\frac{1}{2}$	$\frac{1}{2}$	—
$\mathcal{D}_4$	2	-1	$\frac{5}{2}$	$-\frac{1}{2}$

ϕ	ϕ_0	ϕ_1	ϕ_2	ϕ_3
$\mathcal{D}_2$	1	x	—	—
$\mathcal{D}_3$	1	x	x^2	—
$\mathcal{D}_4$	1	x	x^2	x^3

Indeed, the Newton basis reveals a sense of *stability* among its estimated coefficients c_j absent from the monomial basis's estimated β_j, all while retaining the stability of the basis functions ψ_j themselves.

Example 3.2.11 (Lagrange Basis):
In the Lagrange basis, the situation is quite a bit different. Recall that the Lagrange basis $\mathcal{L}_n = \{\ell_1, \dots, \ell_n\}$ at nodes $x_1, \dots, x_n$ consists of the basis functions

$$\ell_j(x) = \prod_{\substack{i=1 \\ i \neq j}}^{n} \frac{x - x_i}{x_j - x_i}, \qquad j = 1, \dots, n.$$

The coefficients c_j in the expansion

$$p^{(\mathcal{L})}(x) = c_1 \ell_1(x) + \cdots + c_n \ell_n(x)$$

are simply the data values,

$$c_j = y_j, \qquad j = 1, \dots, n.$$

For the data $\mathcal{D}_2 = \{(0,2),(1,3)\}$, the Lagrange basis at the nodes $x_1 = 0$ and $x_2 = 1$ are

$$\ell_1^{(2)}(x) = \prod_{i \neq 1} \frac{x - x_i}{x_j - x_i} = \frac{x - x_2}{x_1 - x_2} = \frac{x-1}{0-1} = 1 - x,$$

and

$$\ell_2^{(2)}(x) = \prod_{i \neq 2} \frac{x - x_i}{x_j - x_i} = \frac{x - x_1}{x_2 - x_1} = \frac{x-0}{1-0} = x.$$

With $c_1 = y_1 = 2$ and $c_2 = y_2 = 3$, we obtain the Lagrange representation of p_2,

$$p_2^{(\mathcal{L})}(x) = y_1 \ell_1^{(2)}(x) + y_2 \ell_2^{(2)}(x) = 2(1-x) + 3(x) = 2 + x,$$

again recovering our familiar form.

Yet, the situation changes completely when we move to augment the data. Indeed, considering $\mathcal{D}_3$ consisting of the *same initial samples* for $j = 1, 2$ along with a new sample $(x_3, y_3) = (3, 8)$, the Lagrange basis at nodes $(x_1, x_2, x_3) = (0, 1, 3)$ becomes

$$\ell_1^{(3)}(x) = \frac{(x-1)(x-3)}{(0-1)(0-3)} = \frac{(x-1)(x-3)}{3},$$
$$\ell_2^{(3)}(x) = \frac{(x-0)(x-3)}{(1-0)(1-3)} = -\frac{x(x-3)}{2},$$
$$\ell_3^{(3)}(x) = \frac{(x-0)(x-1)}{(3-0)(3-1)} = \frac{x(x-1)}{6}.$$

With $(c_1, c_2, c_3) = (y_1, y_2, y_3) = (2, 3, 8)$, the Lagrange representation of p_3 is

$$p_3^{(\mathcal{L})}(x) = 2 \cdot \frac{(x-1)(x-3)}{3} - 3 \cdot \frac{x(x-3)}{2} + 8 \cdot \frac{x(x-1)}{6}.$$

One can verify that the coefficients and basis functions associated with the polynomial p under the Lagrange basis develop as indicated in the table below:

$p^{(\mathcal{L})}(x) = \sum_{j=1}^{n} c_j \, \ell_j(x)$

c	c_1	c_2	c_3	c_4
$\mathcal{D}_2$	2	3	–	–
$\mathcal{D}_3$	2	3	8	–
$\mathcal{D}_4$	2	3	8	6

ℓ	ℓ_1	ℓ_2	ℓ_3	ℓ_4
$\mathcal{D}_2$	$1-x$	x	–	–
$\mathcal{D}_3$	$\frac{(x-1)(x-3)}{3}$	$-\frac{x(x-3)}{2}$	$\frac{x(x-1)}{6}$	–
$\mathcal{D}_4$	$-\frac{(x-1)(x-3)(x-4)}{12}$	$\frac{x(x-3)(x-4)}{6}$	$-\frac{x(x-1)(x-4)}{6}$	$\frac{x(x-1)(x-3)}{12}$

What we observe is that, even though the coefficients c_j are *extremely stable* — indeed, they require no computation at all — the basis functions ℓ_j are *extremely unstable*, changing dramatically with every new sample. Compare with the Newton basis!

Example 3.2.12 (Orthogonal):

Finally, consider the orthogonal basis at the nodes, constructed by applying Gram-Schmidt to the monomial basis $\{1, x, x^2, ...\}$ with respect to the discrete inner product

$$\langle f, g\rangle_n := \sum_{i=1}^{n} f(x_i)g(x_i).$$

Since the inner product itself depends on the node set, we anticipate that *both* the basis functions *and* the coefficients shift under data augmentation. Let us verify this explicitly.

To work in the orthogonal Gram-Schmidt basis, we start by defining the constant basis function

$$\omega_1(x) := 1,$$

and then proceed by iteratively constructing the basis functions

$$\omega_2(x) := x - \text{proj}_{\omega_1}(x), \qquad \omega_3(x) := x^2 - \text{proj}_{\omega_1}(x^2) - \text{proj}_{\omega_2}(x^2),$$

and, in general,

$$\omega_1(x) = 1; \qquad \omega_j(x) = x^{j-1} - \sum_{i=1}^{j-1} \text{proj}_{\omega_i}(x^{j-1}), \qquad j = 2, \dots, n.$$

where we define the **projection** of x^j onto ω_i by

$$\text{proj}_{\omega_i}(x^j) = \frac{\langle x^j, \omega_i\rangle_n}{\langle \omega_i, \omega_i\rangle_n} \cdot \omega_i(x).$$

Now, for the given data $\mathcal{D}_2$ with nodes $(x_1, x_2) = (0, 1)$, we start by setting $\omega_1^{(2)}(x) = 1$, and then compute

$$\omega_2^{(2)}(x) = x - \text{proj}_{\omega_1}(x) = x - \frac{\langle x, \omega_1\rangle_n}{\langle \omega_1, \omega_1\rangle_n} \cdot \omega_1(x)$$

Compute

$$\langle x, \omega_1\rangle_n = \langle x, 1\rangle_n = \sum_{i=1}^{n} x_i \cdot 1 = 0 + 1 = 1,$$

and (for $\mathcal{D}_2$ with $n = 2$)

$$\langle \omega_1, \omega_1\rangle_n = \langle 1, 1\rangle_n = \sum_{i=1}^{n} 1 = n = 2.$$

Thus $\text{proj}_{\omega_1}(x) = \frac{1}{2}$. Together with $\omega_1(x) = 1$ for every x, this gives

$$\omega_2^{(2)}(x) = x - \frac{1}{2}.$$

So, under the orthogonal Gram-Schmidt basis, we have

$$p_2^{(\mathcal{O})}(x) = c_1\omega_1^{(2)}(x) + c_2\omega_2^{(2)}(x),$$

where the coefficients c_j are computed via the inner-product formulas

$$c_j = \frac{\langle \mathbf{y}, \boldsymbol{\omega}_j\rangle_n}{\langle \boldsymbol{\omega}_j, \boldsymbol{\omega}_j\rangle_n} = \frac{\sum_{i=1}^{n} y_i\omega_j(x_i)}{\sum_{i=1}^{n} \omega_j(x_i)^2}.$$

We compute

$$c_1 = \frac{\langle \mathbf{y}, \boldsymbol{\omega}_1^{(2)} \rangle}{\langle \boldsymbol{\omega}_1^{(2)}, \boldsymbol{\omega}_1^{(2)} \rangle} = \frac{y_1 \cdot 1 + y_2 \cdot 1}{1^2 + 1^2} = \frac{2+3}{2} = \frac{5}{2},$$

and

$$c_2 = \frac{\langle \mathbf{y}, \boldsymbol{\omega}_2^{(2)} \rangle}{\langle \boldsymbol{\omega}_2^{(2)}, \boldsymbol{\omega}_2^{(2)} \rangle} = \frac{y_1 \cdot (0 - \frac{1}{2}) + y_2 \cdot (1 - \frac{1}{2})}{(-\frac{1}{2})^2 + (\frac{1}{2})^2} = \frac{-1 + \frac{3}{2}}{\frac{1}{2}} = 1.$$

Assembling,

$$p_2^{(\mathcal{O})}(x) = \frac{5}{2} \cdot 1 + 1 \cdot \left(x - \frac{1}{2} \right) = \frac{5}{2} + x - \frac{1}{2} = 2 + x,$$

which, at this point, is our familiar friend.

We won't illustrate this process for the other datasets we've considered, but one should verify that the tables of coefficients and basis functions under the orthogonal Gram-Schmidt basis $\mathcal{O}_n$ develop as follows as we augment the data $\mathcal{D}_k$:

$$p^{(\mathcal{O})}(x) = \sum_{j=1}^{n} c_j \, \omega_j(x)$$

c	c_1	c_2	c_3	c_4
$\mathcal{D}_2$	$\frac{5}{2}$	1	$-$	$-$
$\mathcal{D}_3$	$\frac{13}{3}$	$\frac{29}{14}$	$\frac{1}{2}$	$-$
$\mathcal{D}_4$	$\frac{19}{4}$	$\frac{13}{10}$	$-\frac{1}{2}$	$-\frac{1}{2}$

ω	ω_1	ω_2	ω_3	ω_4
$\mathcal{D}_2$	1	$x - \frac{1}{2}$	$-$	$-$
$\mathcal{D}_3$	1	$x - \frac{4}{3}$	$x^2 - \frac{22}{7}x + \frac{6}{7}$	$-$
$\mathcal{D}_4$	1	$x - 2$	$x^2 - 4x + \frac{3}{2}$	$x^3 - 6x^2 + \frac{43}{5}x - \frac{6}{5}$

Notice that *nothing here* is stable — neither the basis functions nor the coefficients. One might reasonably wonder whether the orthogonal basis offers any advantage at all. It does, and it is a subtle one: each coefficient c_j is computed by a *single inner product* of the data with $\boldsymbol{\omega}_j$, independently of every other coefficient — there is no system to solve, forward or backward, and no coefficient depends on the value of any other. This is a virtue the other bases do not share: the monomial requires a dense solve, Newton requires forward substitution, and Lagrange (though it demands no solve) depends on every node x_i to assemble any single basis function. Whether this independence is worth the cost of rebuilding both the basis and the coefficients at every augmentation depends, as we shall see, on what one intends to do with them.

‖

This concludes our development of the polynomial interpolation problem. We summarize the taxonomy of bases and their associated design matrix structures in the following table:

Basis of Π_{n-1}	Design Structure	Solution Method	Behavior Under Data Augment
Monomial $\mathcal{M}_{n-1}$	Dense (Vandermonde)	Back-substitution	Unstable; dense
Newton $\mathcal{N}_{n-1}$	Lower triangular	Forward substitution	Stable; triangular
Lagrange $\mathcal{L}_{n-1}$	Identity	Immediate (read off values)	Unstable; redefines every ℓ_j
Orthogonal $\mathcal{O}_{n-1}$	Orthogonal: $\Psi^\top\Psi = \text{diag}$	Inner products	Unstable; redefines every ω_j

Every basis in this table represents the same polynomial — the unique $p \in \Pi_{n-1}$ satisfying $p(x_i) = y_i$. They differ only in the language of representation, and each language carries its own structural virtues and costs. The question "which representation is useful?" — which we raised at the close of the previous green ribbon — now has a nuanced answer: it depends on what you need. Stability under augmentation favors Newton. Immediate coefficients favor Lagrange. Independent coefficient extraction favors orthogonal bases. And the trigonometric basis, alone among those we have seen, provides orthogonality as an intrinsic property of the functions rather than an artifact of the data.

But we have, in all of this, been operating within the confines of a single assumption: that we have n data points, and we fit a polynomial of degree exactly $n-1$ passing through each one. The polynomial interpolation theorem guarantees this is always possible and always unique. What we have *not* addressed is the question that the parable's second act posed most urgently: given that the interpolating polynomial passes through the data perfectly, does it behave well *between* the data points? The coefficient instability we resolved was a problem of *representation*; the Cauchy analyst's catastrophe was a problem of *approximation*. These are not the same problem, and the tools we have developed so far — bases, design matrices, triangularity, orthogonality — do not, by themselves, resolve the second. For that, we will need to leave the finite-dimensional world of Π_{n-1} and ask what happens when the function we are trying to approximate is not itself a polynomial — when the DGP lives in a space that Π_{n-1}, no matter how large n, can only approximate.

3.3 Fitting Points or Functions?

Reopening the Parable

So, at this point what we understand is that if we're given a dataset consisting of finitely many sample points, say $\mathcal{D}_n = \{(x_1, y_1), \dots, (x_n, y_n)\}$ for real valued predictor nodes $x_i \in \mathbb{R}$ and outcome data $y_i \in \mathbb{R}$, we're always able to identify a polynomial of degree at most $n-1$, $p \in \Pi_{n-1}$, such that p interpolates the data $\mathcal{D}_n$ exactly — i.e.,

$$p(x_i) = y_i, \qquad i = 1, \dots, n.$$

The existence and uniqueness of this interpolating polynomial p is guaranteed by the apt Polynomial Interpolation. We may intuit the explicit construction of the unique interpolating polynomial p straightforwardly in terms of the Newton basis, $\mathcal{N}_{n-1} = \left\{1, (x-x_1), (x-x_1)(x-x_2), \dots, \prod_{i=1}^{n-1}(x-x_i)\right\}$, and the uniqueness guaranteed by the interpolation theorem assures us that our choice of representative basis $\mathcal{N}_{n-1}$ over, say, the monomial basis $\mathcal{M}_{n-1} = \{1, x, x^2, \dots, x^{n-1}\}$, is unimportant if our concern is merely in evaluations of $p(x)$. The reason we may prefer the Newton basis is that its coefficients and basis functions are \emph{stable under data augmentation. That is, unlike the monomial basis — whose expansion in terms of basis functions $p^{(\mathcal{M})}(x) = \beta_0 + \beta_1 x + \cdots + \beta_{n-1}x^{n-1}$ sees the coefficients $\beta_0, \dots, \beta_{n-1}$

fluctuate with the addition of a new basis function x^n with coefficient β_n — the Newton basis expansion $p^{(\mathcal{N})}(x) = c_0 + c_1(x - x_1) + \cdots + c_{n-1}\prod_{i=1}^{n-1}(x - x_i)$ has coefficients $c_0, \ldots, c_{n-1}$ which *remain fixed* when we add a new data point x_n with basis function $\prod_{i=1}^{n}(x - x_i)$.

This is all well and good, and we're better off for having taken the time to understand it. But now, let us take a moment to recall the parable's setup. In the first half, we began by issuing some fairly arbitrary data $\mathcal{D}_2 = \{(0, 2), (1, 3)\}$ and simply explored what it would mean for "a quadratic $p \in \Pi_2$ to interpolate that data." After some reflection, we realized that the class of quadratic interpolants was simply too large to identify a unique interpolating quadratic — both $2 + x^2$ and $2 + x$ are in the class Π_2, and both interpolate $\mathcal{D}_2$ perfectly. The solution, then, was to simply *reduce the size* of the class of interpolants under consideration; namely, by considering polynomials $p \in \Pi_1$, the class of linear interpolants (i.e., lines $p(x) = mx + b$). The line $p(x) = 2+x$ is the unique line interpolating the data in $\mathcal{D}_2$, and at the time, we implicitly took uniqueness over a reduced class as a "good enough reason" to prefer the line over the quadratic for interpolating $\mathcal{D}_2$.

Yet, what happened when we appended a new — again, mostly contrived — third sample $(x_3, y_3) = (3, 8)$ to the existing data was a bit discomforting: we realized that *no* linear polynomial could interpolate the new data $\mathcal{D}_3 = \{(0, 2), (1, 3), (3, 8)\}$, and were we to still insist on interpolating the data exactly, we would need to consider enlarging the class of models from the linear polynomials Π_1 to the quadratic polynomials Π_2 after all. On the one hand, it is intuitively clear that this *had to happen* — we're not magicians, nor fortune-tellers, and it wouldn't make sense to chastise ourselves for being wrong about something we *didn't know*. And, again, the data we generated in the first half of the parable were, for all intents and purposes, picked because "they look nice" and "they didn't break anything in the example we had in mind" — which is to say, they were *not* generated by a "true DGP."

But this is what led us to the second half of the parable, and it is the half we've yet to truly reckon with as of this point. Recall that we defined two data generating processes g (the Gaussian) and h (the Cauchy) by their normalized functional forms

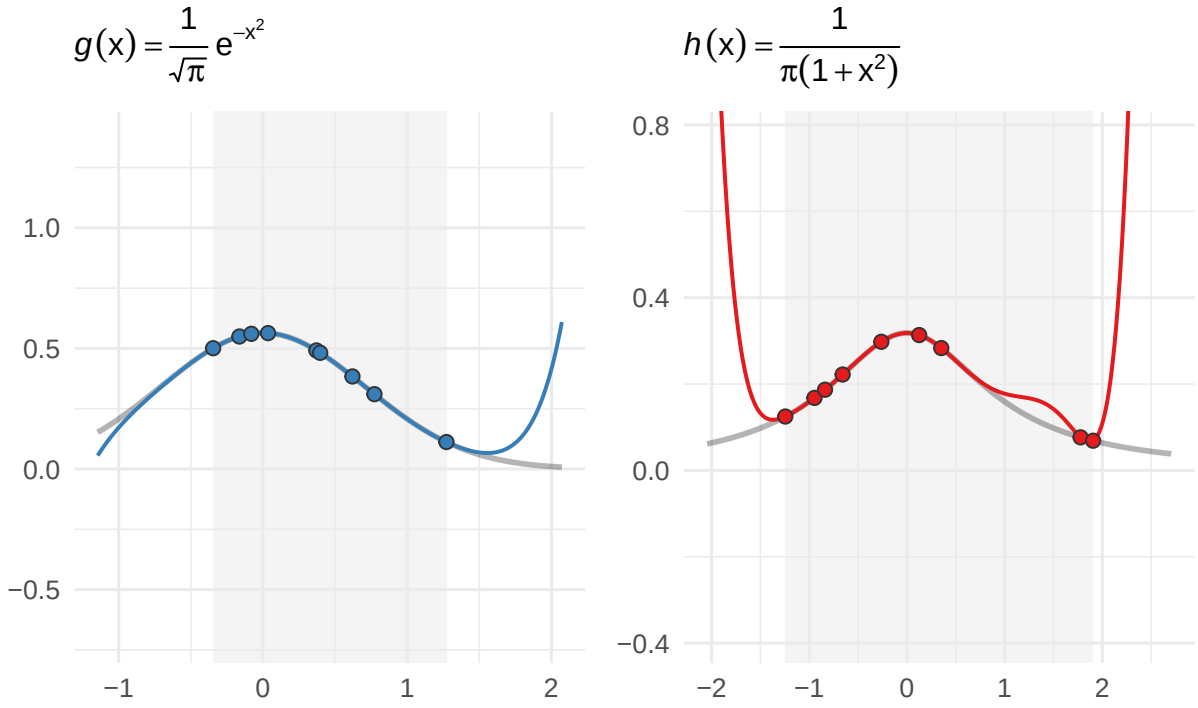
$$g(x) = \frac{1}{\sqrt{\pi}} e^{-x^2}, \qquad h(x) = \frac{1}{\pi} \frac{1}{1 + x^2},$$

generated datasets $\mathcal{G}_n \sim g$ and $\mathcal{H}_n \sim h$, and assigned each to one of two competing analysts. We informed them that the data they'd receive was generated perfectly in accordance with their respective processes — essentially, so that no questions of "data quality" or "measurement error" could save the analysts from what was to follow.

At $n = 9$, the data $\mathcal{G}_n$ and $\mathcal{H}_n$, whose samples were randomly generated by the processes g and h respectively, looked "fairly similar" when placed on a scatterplot. Since the data was "generated perfectly," the analysts supposed that fitting an interpolating polynomial through the data — which we now know is guaranteed to exist and be unique *for each fixed dataset* by the Interpolation Theorem — couldn't be the worst idea, and they proceeded to do so. We recall that they observed the following:

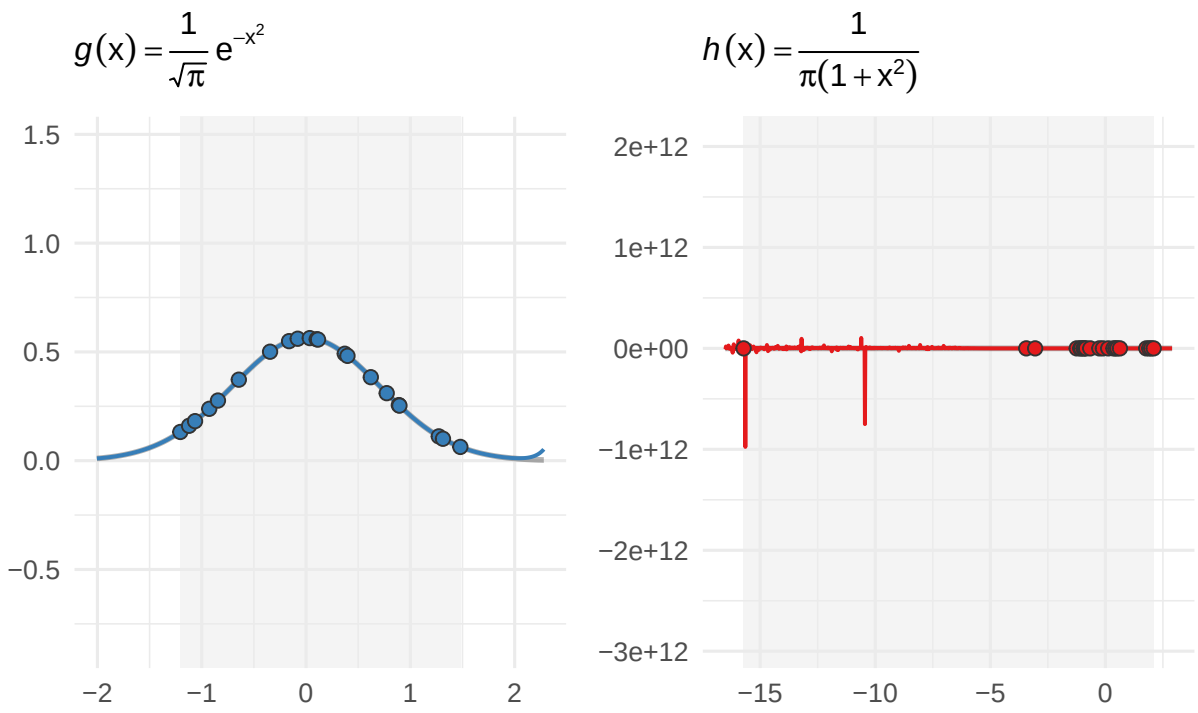
Interpolation at observed sample locations: $n = 9$

Degree−8 interpolant; shaded region = range of observed data



Indeed, these curves "look about the same." There isn't a reason to believe that the one on the left is "proper" and the on on the right "isn't" — and indeed, if the analyst of $\mathcal{H}_9$ was to estimate $h(1.5)$ by the interpolating polynomial $p_9^{(h)}(1.5)$ depicted in the graph above, her answer wouldn't raise eyebrows relative to "how wrong she could have been," say, by just "guessing blindly."

But then, the analysts were cursed by *more data*. In particular, at $n = 21$ (consisting of the first $n_1 = 9$ samples, appended by a new batch of $n_2 = 12$ samples), the analysts observed the following interpolating polynomials:



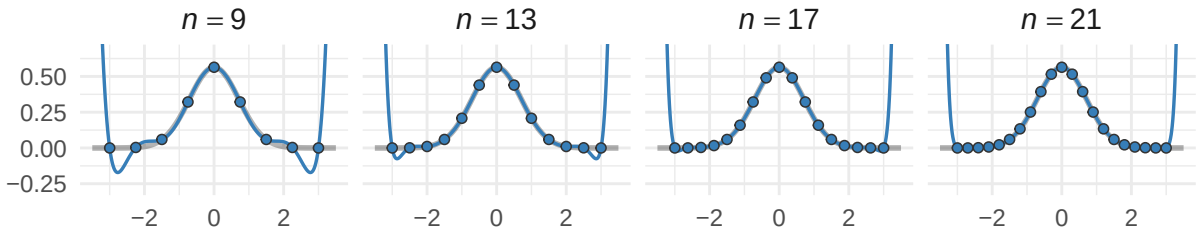
Again the interpolation theorem has *done its job*: it has found an interpolating polynomial capable of bouncing between the outlier generated by the Cauchy density at $x \approx -15$ and the bulk of the samples nearer $x \approx 0$ — but this interpolation is, for all intents and purposes, *completely unusable* for prediction on $h(x)$. We can no longer say that $h(-10)$ is "reasonably approximated" by $p_{21}^{(h)}(-10)$, nor anywhere else — it is a terrible, ugly thing, and we did not discredit the analyst of $\mathcal{H}_{21}$ into thinking she'd believe otherwise.

But, really: what is she to do? "Trim the outlier"? *The data is perfect!* To "trim the outlier" would, in a very precise way, be to discard some kernel of *what h really is*, which is a machine capable of generating large outliers at $x \approx -15$ at small samples of even size $n = 21$. Again, the analysts don't know the functional forms; all they have is the data. As the oracles, we're even able to compute that the probability of the Gaussian DGP g generating such a sample at $x \approx -15$ is on the order of 10^{-98} — negligible compared to the analogous probability of roughly $1/700$ under the Cauchy DGP h. And this all seems to reinforce the notion that it would be *entirely improper* for the analyst to "pretend that data point didn't exist." We can even take it one step further and realize that the problem persists even when we bestow the analysts with data generated on equispaced nodes, where we observe the "Cauchy's fangs" emerge for higher and higher n:

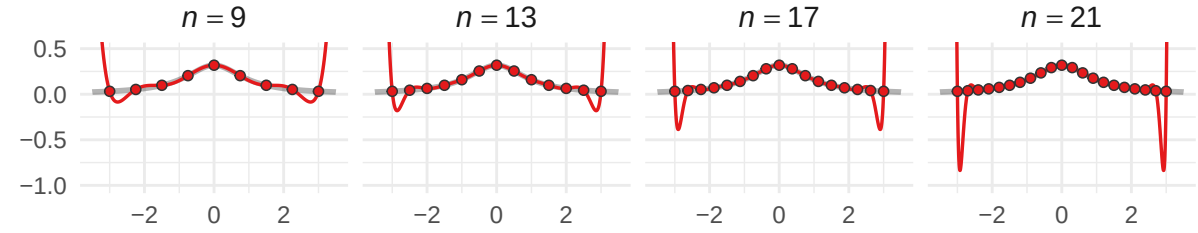
Equispaced polynomial interpolation

Equispaced nodes on $[-3, 3]$; degree = $n - 1$

$$g(x) = \frac{1}{\sqrt{\pi}} e^{-x^2}$$



$$h(x) = \frac{1}{\pi(1 + x^2)}$$



Theoretically, we traced our inability to approximate h back to its Taylor coefficients: those of g decayed like $1/k!$ (that is, quite rapidly), while those of h retained a constant contribution across all terms in its Taylor expansion — i.e., h's Taylor coefficients *don't* decay. New observations always have the potential to completely destabilize what was "beginning to look like" a decent approximation — and what is perhaps scariest is that the "looks like" here is *always* acting to deceive us.

One might wonder, then, whether the issue lay with the polynomial itself as a tool for *approximating the functions* g and h. We stress that this is an entirely different question from what the Polynomial Interpolation Theorem addresses: that result, in part, tells us that polynomials of degree $n-1$ or higher are always capable of *interpolating the points* $(x_1, y_1), \ldots, (x_n, y_n)$ — but it says nothing about whether the resulting polynomial bears any resemblance to a function from which those points may have been drawn. This distinction — between "interpolating points" and "approximating functions" — is what implicitly separated the arbitrarily generated data of the parable's first half from the data generated by the functional forms of the Gaussian and Cauchy DGPs in its second. And one might assume, based on what we've observed thus far — namely, that h isn't well-approximated by its Taylor series — that the answer is *no*: that some functions simply elude approximation by polynomials, with the Cauchy h serving as the face of such a class.

What is genuinely surprising to us, then, is that such an ostensibly reasonable conjecture turns out to be *dead wrong*.

3.3.1 Weierstrass's Game

We begin the section by stating its central result, due to German mathematician Karl Weierstrass (with whom we will become intimately familiar) in 1885, in its standard form.

Theorem 3.3.1 (Weierstrass Approximation Theorem)

Let $f\colon [a,b] \to \mathbb{R}$ be a continuous function on a closed and bounded interval $[a,b] \subset \mathbb{R}$. Then for any tolerance $\epsilon > 0$, there exists a *finite* degree $N = N(\epsilon) \in \mathbb{N}$ such that some polynomial $p \in \Pi_N$ uniformly approximates f on $[a,b]$; that is,

$$\sup_{x\in[a,b]} |f(x) - p(x)| := \|f - p\|_\infty < \epsilon.$$

Before we discuss with what we can *do* with the theorem, we want to linger on its ramifications for just a moment, particularly with regards to the parable. The statement of the Weierstrass Approximation Theorem is short, but what it asserts is quite remarkable. Indeed, the hypothesis on the target $f\colon [a,b] \to \mathbb{R}$ is essentially as weak as we could ask of any function: we require only that f be *continuous* (not differentiable, nor analytic, nor smooth in any sense beyond the barest sense), and we're guaranteed the existence of a class of polynomials Π_N containing a member p approximating f *uniformly well across all of* $[a,b]$ *simultaneously*. What the theorem *doesn't* tell us is, well, how to *find* the integer N, nor the class Π_N, and especially not the polynomial $p \in \Pi_N$ itself. Indeed, the Weierstrass theorem asserts *existence* of these objects, and nothing more.

Remark.

The reader should compare Weierstrass's theorem here with one due to Fejér (Theorem 2.3.10) as stated Chapter 2. What you may notice is that the theorems address similar questions — and indeed, they have similar answers. We ask you to simply stash this observation away for the time being — we'll address it come the intermission below.

We're *immediately* struck by the impossibility of the approximating polynomials being produced by the Taylor series of the target function f. We've already observed a function — the Cauchy density h — with a Taylor series converging only within a fixed radius $|x| < 1$. Relative to its Taylor polynomials, we can thus intuit our inability to find an $N = N(\epsilon)$ and a $p \in \Pi_N$ such that p and h are uniformly close across, say, all of $x \in [-3,3]$. Yet, what Weierstrass guarantees is that a *polynomial* — something of *precisely* the same form as the Taylor series, "$a_0 + a_1 x + a_2 x^2 + \cdots + a_N x^N$ for some N and a_j" — produces such a uniform approximation.

Of course, the theorem makes specific demands of the domain on which we hope to approximate f by a polynomial p. In particular, the requirement that $[a,b]$ be a closed and bounded interval is

essential, and we cannot simply "wave it away." Indeed, suppose we were to instead ask whether there were a polynomial p such that $\sup_{x\in\mathbb{R}} |h(x) - p(x)| < \epsilon$ for our Cauchy density h on *all* of $\mathbb{R}$. We wouldn't have to look very hard to find such an attempt structurally hopeless: $h(x) \to 0$ as $|x| \to \infty$, while *any* nonconstant polynomial p has $|p(x)| \to \infty$ as $|x| \to \infty$, and the difference $|h(x) - p(x)|$ blows up to $+\infty$ along with p in either direction. The constant polynomial $p \equiv 0$ fares no better — it misses $h(0) = 1/\pi$ by a fixed amount and never recovers. Polynomials and decaying functions are simply incompatible "at infinity," and no choice of N nor $p \in \Pi_N$ can rescue the situation. Yet — and this is the entire force of the theorem — on *any* closed and bounded interval $[-R, R]$, no matter how large R is, the Weierstrass theorem applies in full, and a polynomial approximating h to within ϵ *exists*.

Perhaps most importantly for the parable we've been developing, however, is somethign else that the Weierstrass theorem *doesn't* tell us: namely, that the approximating polynomial $p \in \Pi_N$ is anything like the interpolating polynomials our analysts have been constructing. The interpolating polynomial $p_{n-1} \in \Pi_{n-1}$ is the unique element satisfying $p_{n-1}(x_i) = f(x_i)$ at n specified nodes; the Weierstrass polynomial $p \in \Pi_N$, by contrast, is some element of *some* class Π_N satisfying $|f(x) - p(x)| < \epsilon$ at *every* location $x \in [a, b]$. These are answers to fundamentally different questions, and the polynomials answering them are, in general, fundamentally different objects. The Weierstrass polynomial owes the data values $f(x_i)$ *nothing*: it need not pass through any of them exactly, nor even particularly closely at any individual point — it need only stay within ϵ of f uniformly. The interpolating polynomial, conversely, is forced to agree with f precisely at the nodes, and it spends every degree of freedom it has doing so.

Applied to our analyst of $\mathcal{H}_n$ on, say, $[-3, 3]$, then, what the Weierstrass theorem tells us is that for any tolerance $\epsilon > 0$ we care to specify — be it 10^{-3}, 10^{-10}, or any other — there exists a polynomial p such that

$$\sup_{x\in[-3,3]} |h(x) - p(x)| < \epsilon.$$

The polynomial *exists*. It is sitting somewhere in the union $\bigcup_{N=0}^{\infty} \Pi_N$, waiting to be found. The catastrophic interpolating polynomials we constructed in the parable's second half are not it; the "fangs" the analyst observed at the boundary are not what a good polynomial approximation to h on $[-3, 3]$ has to look like at all. There is a polynomial that approximates h to graphical accuracy on the entire interval, and another that does so to within any tolerance we may demand — and the Weierstrass theorem assures us of as much, in spite of the analyst's best efforts to convince herself otherwise.

But the analyst, equipped now with the theorem and her catastrophic interpolant, would be entitled to ask the obvious question: *where is it?* The Weierstrass theorem guarantees the polynomial exists, but it offers her no recipe for constructing one, no bound on the degree she'd need, and no indication of whether her interpolating strategy could be salvaged or whether something altogether different is required. All she is certain is that *if* there is a method that can identify a representative polynomial p within the (nonempty!) set of good approximants to h guaranteed by Weierstrass, she would very much like to understand what it is, and whether it'd help in salvaging her analysis.

Weierstrass By Construction: A Naive Attempt

So, if it weren't already clear, we're *not* going to "prove the Weierstrass Approximation Theorem" — at least, not directly. What we seek to do is *explicitly construct* this uniformly approximating polynomial, which would, of course, suffice for proving the original theorem anyway. But if it weren't already clear, this is easier said than done — we speculate that there is a reason

Karl Weierstrass *wasn't* more explicit in his original statement, and it isn't for lack of intellect. Indeed, we will see that it is *simply hard* to identify the "right" polynomial — to bury the lede just a bit, all we will say now is that this polynomial cannot be constructed using any of the methods we've considered thus far.

Nevertheless, let's try anyway, just to see what happens when we do. Concretely, let us suppose for the duration of this setup that we have an adversary, say Weierstrass himself, and he and we are to play a game. The rules are as follows: we commit, in advance, to a *strategy* for constructing polynomials on $[-1, 1]$. Once we've committed, Weierstrass will hand us a function $f \in C[0, 1]$ and a tolerance $\epsilon > 0$ of his choosing. We win if our strategy produces a polynomial satisfying

$$\|f - p\|_\infty := \sup_{x \in [0,1]} |f(x) - p(x)| < \epsilon;$$

otherwise, we lose.

Now, at this point, the only tools we have at our disposal are the ones this chapter has developed: polynomial bases, interpolation, and the design matrices that accompany them. Weierstrass, for his part, knows this — and, to make things interesting, he offers a concession: he will evaluate f for us at the $n + 1$ equispaced points $0, \frac{1}{n}, \frac{2}{n}, \dots, 1$, and hand us the resulting data

$$\mathcal{D}_n = \left\{ \left(\frac{k}{n}, \; f\left(\frac{k}{n} \right) \right) \right\}_{k=0}^{n}$$

We may use this data however we like, and we are free to request n as large or as small as we wish. The catch, of course, is that Weierstrass chooses f *after* seeing our strategy — and he will choose adversarially.

Round 1

We thus begin our time with Weierstrass by examining the tools at our disposal and asking which, if any, have the "best chance" of defeating Weierstrass out the gate. Given the data $\mathcal{D}_n$ consisting of $n + 1$ samples on equispaced nodes $x_0 = 0, x_1 = 1/n, x_2 = 2/n, \dots, x_n = 1$ with corresponding evaluations $f(0), f(1/n), f(2/n), \dots, f(1)$, we might start by picking our favorite basis from the previous section, adapting it to $\mathcal{D}_n$, and taking as our construction p the unique interpolating polynomial of degree n as guaranteed by the interpolation theorem.

But which basis should we choose? Does it matter? We've seen that the monomial, Newton, Lagrange, and orthogonal bases all produce the *same* interpolating polynomial (again guaranteed by the interpolation theorem). So any basis we choose will yield the same p — the basis is a matter of *representation*, not of substance. So let us simply choose whichever representation is most *revealing* for the task at hand; for reasons that will become clear momentarily, we choose the Lagrange basis

$$\mathcal{L}_n[f(x)] = \sum_{k=0}^{n} f\left(\frac{k}{n} \right) \ell_k(x), \qquad \ell_k(x) = \prod_{\substack{j=0 \\ j \neq k}}^{n} \frac{x - x_j}{x_k - x_j}.$$

We choose this representation not because it is computationally convenient (the Newton basis would be better for such a purpose) — rather, we choose it because it makes the *structure* of the interpolant maximally transparent. Look at the formula: the interpolating polynomial $\mathcal{L}_n[f]$ is a sum of the basis functions ℓ_k, each weighted directly by the evaluations $c_k = f(k/n)$ as given by Weierstrass himself.

In fact, the Lagrange representation reveals something else, something we really ought to have

noticed earlier: the basis functions ℓ_k satisfy a **partition of unity**. Indeed, one should verify that, for *every* $x \in [0,1]$ (not just at the nodes $k/n \in \mathcal{D}_n$ where $\sum \ell_k(x_j) = 1$ is trivial!), we have

$$\sum_{k=0}^{n} \ell_k(x) = \sum_{k=0}^{n} \left(\prod_{\substack{j=0 \\ j \neq k}}^{n} \frac{x - x_j}{x_k - x_j} \right) = 1.$$

(Hint: check this by verifying that if $f \equiv 1$ is the constant function on $[0,1]$ then f is its own interpolant; in particular, use the fact that $\mathcal{L}_n[f(x)] = 1$ for every $x \in [0,1]$.)

So, the interpolant $\mathcal{L}_n[f(x)] = \sum f(k/n)\ell_k(x)$ is a sum of the data values $f(k/n)$ weighted by functions $\ell_k(x)$ that *sum to* 1 *for every* x. Consider, then, how this does and does not resemble some special brands of "linear combination." For concreteness, we restate definitions of some familiar, and perhaps one unfamiliar, kinds of combinations.

Definition 3.3.2 (Special Linear Combinations)

Suppose that $x_0, x_1, \ldots, x_n \in \mathbb{R}$ are fixed real numbers.

(a) Recall that a **linear combination** of the given $x_0, x_1, \ldots, x_n \in \mathbb{R}$ is an expression of the form

$$\sum_{k=0}^{n} w_k x_k : \quad w_k \in \mathbb{R}. \qquad \text{(Linear Combination)}$$

No restrictions are placed on the weights w_k other than they be real numbers.

(b) If in the definition of a linear combination, we impose that the weights **sum to unity**:

$$\sum_{k=0}^{n} w_k = 1,$$

then we obtain an **affine combination**:

$$\sum_{k=0}^{n} w_k x_k : \quad w_k \in \mathbb{R} \quad \text{s.t.} \quad \sum_{k=0}^{n} w_k = 1. \qquad \text{(Affine Combination)}$$

(c) If in the definition of an affine combination, we even further restrict each weight in the partition of unity to be **nonnegative**:

$$w_k \geq 0, \qquad \sum_{k=0}^{n} w_k = 1,$$

then we obtain a **convex combination**:

$$\sum_{k=0}^{n} w_k x_k : \quad w_k \in \mathbb{R} \quad \text{s.t.} \quad w_k \geq 0, \quad \sum_{k=0}^{n} w_k = 1. \qquad \text{(Convex Combination)}$$

We've already observed that the Lagrange basis produces a partition of unity in its weights:

$$\sum_{k=0}^{n} \ell_k(x) = 1, \qquad \forall x \in [0,1].$$

Thus, the Lagrange interpolant

$$\mathcal{L}_n[f(x)] = \sum_{k=0}^{n} f\left(\frac{k}{n}\right) \ell_k(x)$$

is an affine combination of the data values $f(0), f(1/n), \dots f(1)$; the "weights" $\ell_k(x)$ sum to 1, and the "coefficients" are the data values $f(k/n)$ themselves.

But is $\mathcal{L}_n[f]$ a *convex* combination? Indeed, this would require, in addition to the ℓ_k serving as a partition of unity for every $x \in [0,1]$, that each $\ell_k(x) \geq 0$ be nonnegative for every $x \in [0,1]$. Unfortunately, we can immediately verify that the answer is *no* — and, moreover, the reason it is "no" is not an accident of our data, but a structural feature of the Lagrange basis deployed on *any* data. The issue is that each ℓ_k is a polynomial of degree n with exactly n roots — one at each node x_j with $j \neq k$ — and a polynomial of degree n with n distinct real roots *must change sign* at each root. (Intuitively, each root is *simple* — the factor $(x - x_j)$ appears exactly once in the product defining ℓ_k — and a polynomial that arrives at a simple root from the positive side must emerge negative on the other. It cannot "touch the axis and bounce back" as it would at a repeated root like $(x - x_j)^2$; it *crosses through*, and the crossing flips its sign.) To verify this explicitly, one may check that for $n = 2$, the equispaced nodes on $[0,1]$ are $x_0 = 0, x_1 = 1/2, x_2 = 1$, and the Lagrange basis functions are

$$\ell_0(x) = 2(x - \tfrac{1}{2})(x - 1), \qquad \ell_1(x) = -4x(x - 1), \qquad \ell_2(x) = 2x(x - \tfrac{1}{2}).$$

At $x = 1/4$, for instance, we thus have

$$\ell_0(1/4) = \frac{3}{8}, \qquad \ell_1(1/4) = \frac{3}{4}, \qquad \ell_2(1/4) = -\frac{1}{8}.$$

So, in the expansion

$$\mathcal{L}_2[f(1/4)] = \frac{3}{8} f(0) + \frac{3}{4} f(1/2) - \frac{1}{8} f(1),$$

we can observe that the weights *do* form a partition of unity: $\sum \ell_k(1/4) = \frac{3}{8} + \frac{3}{4} - \frac{1}{8} = 1$. Yet, clearly $\ell_2(1/4) = -\frac{1}{8} < 0$, so that $\mathcal{L}_n[f(1/4)]$ is *not a convex* combination of the data $f(x_k)$; it is merely an *affine* combination.

At this point, we acknowledge the obvious: why does any of this matter in defeating Weierstrass? The answer is that affine and convex combinations carry *fundamentally different guarantees* about the structure of their outputs. Indeed, recall from Chapter 2 (Definition 2.4.5 and the surrounding discussion) that a convex combination of values $x_0, x_1, \dots, x_n \in \mathbb{R}$ is always "self-contained" — in particular, if the coefficients $c_k \geq 0$ are nonnegative and form a partition of unity $\sum_k c_k = 1$, we have

$$\min_k x_k \leq \sum_{k=0}^{n} c_k x_k \leq \max_k x_k.$$

This is decidedly *not* a property harbored by affine combinations. (To see this explicitly: in the expression for $\mathcal{L}_2[f(1/4)]$ above, imagine that $0 \leq f(0) \leq f(1/2) << f(1)$ — i.e., $f(1)$ is a

large positive number weighted by the negative coefficient $-1/8$ in $\mathcal{L}_2[f(1/4)]$. Then $\mathcal{L}_2[f(1/4)]$ can certainly be negative, even if we also had $f(x) \geq 0$ nonnegative for all $x \in [0,1]$. Convince yourself that this simply cannot happen when the coefficients are strictly nonnegative!)

What this means for our game against Weierstrass, then, is immediate: the Lagrange interpolant, being merely an *affine* combination of the data values $f(k/n)$, is free to produce values $\mathcal{L}[f(x)]$ *outside the range* of f. If $f\colon [0,1] \to [0,1]$, for instance, there is nothing in the structure of $\mathcal{L}_n[f]$ preventing it from evaluating to -3, $+47$, or any other number it pleases — the negative weights in the Lagrange basis permit the interpolant to escape the **convex hull** of the data entirely. (This is precisely what the "fangs" in the equispaced Cauchy interpolant *are*: the function $h(x) = 1/\pi(1+x^2)$ takes values in $[0, 1/\pi]$, and yet the Lagrange interpolant at equispaced nodes produces values of much larger magnitude near the boundary.)

We wouldn't appear to have much confidence in our interpolating polynomial $\mathcal{L}_n[f]$, if for no other reason than we *already know* its defects: it simply doesn't work for approximating the Cauchy density on equispaced nodes. Nevertheless, it is what we have, and so we submit it: our Round 1 strategy is to take as our approximant

$$p(x) = \mathcal{L}_n[f(x)],$$

the Lagrange interpolant at equispaced nodes.

Weierstrass, who has been watching patiently, hands us $f = h$, the Cauchy density rescaled to $[0,1]$. We already know what happens; we concede the first round to Weierstrass.

Round 2

Before we can regroup, a voice from the gallery interrupts — immediately, we recognize our peer as none other than great Russian mathematician Pafnuty Lvovich Chebyshev. Chebyshev, a household name due to his own concentration inequality (cf. Chapter 2), happens to have spent the better part of the 1850s thinking about precisely the kind of approximation problem we've just lost. Indeed, he and Weierstrass have played this game before — and our new friend is happy to offer insights he's garnered from rounds past, should they aid in our attempts to take Weierstrass down once and for all.

Chebyshev begins by asking us to look more carefully at the Lagrange interpolant. "You know it fails for the Cauchy density," he says, "but do you know *why*?" We gesture vaguely at the fanga, the sign-changing basis functions, the affine-not-convex structure we just diagnosed. "Yes, yes," he retorts, "but there is something more precise." He directs our attention back to the Lagrange formula and asks us to consider the following.

Chebyshev: Fix any $x \in [0,1]$ that is *not* one of the given nodes $x_k = k/n$, and ask: 'how far away is your approximation from the truth?' Define the **interpolation error**

$$e_n(x) := f(x) - \mathcal{L}_n[f(x)].$$

By the interpolation theorem,

$$e_n(x_k) = f(x_k) - \mathcal{L}_n[f(x_k)] = 0, \qquad k = 0, 1, \dots, n,$$

so the error function e_n vanishes at all $n+1$ nodes $e_k = k/n$, $k = 0, 1, \dots, n$.

"Now," Chebyshev continues, "suppose for the moment that the function Weierstrass will choose is sufficiently smooth." We may linger, wondering whether this is an appropriate assumption — we take care to observe that Weierstrass did *not* dictate any smoothness conditions on the functions he's allowed to choose — yet, lacking any other suitable recourse, we choose to follow Chebyshev's lead.

In such a case that f has $n+1$ continuous derivatives on $[0,1]$, Chebyshev asks us to consider the auxilliary function

$$\varphi(t) := \underbrace{f(t) - \mathcal{L}_n[f(t)]}_{=e_n(t)} - \lambda \prod_{k=0}^{n}(t - x_k)$$

where λ is a constant chosen so that φ vanishes at some fixed point $x^\star \in [0,1]$ distinct from the nodes $x_0, x_1, \dots, x_n$. "This may look like a lot of nonsense," he continues, "but just trust me."

Remark.

The reader is encouraged to verify that such a λ exists and is unique — indeed, since $\mathcal{L}_n[f(x_k)] = f(x_k)$ at every node and $x \mapsto \prod_{k=0}^{n}(x_k - x_k) = 0$ at every node, we have $\varphi(x_k) = 0$ for $k = 0, 1, \dots, n$ automatically. We then choose λ to make $\varphi(x^\star) = 0$ as well, which requires

$$\lambda = \frac{f(x^\star) - \mathcal{L}_n[f(x^\star)]}{\prod_{k=0}^{n}(x^\star - x_k)},$$

where the denominator $\prod(x^\star - x_k) \neq 0$ since $x^\star$ is not one of the original nodes.

"So," Chebyshev says, "φ vanishes at $n+2$ distinct points in $[0,1]$: the $n+1$ nodes $x_0, x_1, \dots, x_n$, and the additional, distinct point $x^\star \in [0,1]$ we constructed."

"Oh," he hesitates, "and I suppose I should ask — are you familiar with Rolle's Theorem?"

Theorem 3.3.3 (Rolle's Theorem)

Let $\varphi\colon [a,b] \to \mathbb{R}$ be continuous on $[a,b]$ and differentiable on (a,b), with

$$\varphi(a) = \varphi(b) = 0.$$

Then, there exists at least one point $c \in (a,b)$ such that

$$\varphi'(c) = 0.$$

Proof:

Consider two cases. First, suppose

$$f(x) = 0 \quad \text{for all} \quad x \in [a,b].$$

In this case, any value $c \in [a,b]$ can serve as the c guaranteed by the theorem, as the function is constant on $[a,b]$, whence its derivative is zero.

Second, suppose

$$f(x) \neq 0 \quad \text{for some} \quad x \in (a,b).$$

Clearly f attains a minimum and maximum somewhere in $[a,b]$ (cf. the **Extreme Value Theorem**). Recall by our hypothesis that $f(a) = f(b) = 0$, and that in this case, $f(x) \neq 0$

for some $x \in (a,b)$. Thus, f will have either a positive absolute maximum value at some $c_+ \in (a,b)$, a negative absolute minimum value c_- in (a,b), or both.

Take c to be either c_+ or c_- (or either one), depending on which we've identified.

Then, the open interval (a,b) contains c, and either

- $f(c) = f(c_+) \geq f(x)$ for all $x \in (a,b)$, or
- $f(c) = f(c_-) \leq f(x)$ for all $x \in (a,b)$.

Either way, this means f has a local extremum at c. Since f is differentiable on (a,b) and $c \in (a,b)$ strictly, $f'(c)$ exists, and we conclude that at this point c,

$$f'(c) = 0.$$

$\|$

"Good," says Chebyshev. "Now, recall: φ vanishes at $n+2$ distinct points in $x_0, x_1, \dots, x_n$, and our chosen root $x^\star$. Supposing that $x_j < x^\star < x_{j+1}$ and $0 \leq x_0 < x_1 < \cdots < x^\star < \cdots < x_n \leq 1$, we can clearly apply Rolle's Theorem to φ on each subinterval $[x_i, x_{i+1}]$. In particular, since we've constructed φ such that

$$\varphi(x_0) = \varphi(x_1) = \cdots = \varphi(x^\star) = \cdots = \varphi(x_n) = 0,$$

and since we're supposing f (and hence φ) is continuous and "sufficiently smooth" — which we now restate formally as requiring Weierstrass's chosen function f be at least $n+1$ times differentiable on $(0,1)$ — we can apply Rolle's theorem to find points in each interval $c_i \in (x_i, x_{i+1})$ (along with our extra root $c^\star \in (x^\star, x_{j+1})$) such that

$$\varphi'(c_0) = \varphi'(c_1) = \cdots = \varphi'(c^\star) = \cdots = \varphi'(c_{n-1}) = 0.$$

That is, by one application of Rolle's Theorem, φ' has at least $n+1$ zeros $c_0, c_1, \dots, c_j, c^\star, c_{j+1}, \dots, c_{n-1}$ in $(0,1)$."

Chebyshev pauses. "Do you see the pattern? Apply Rolle's theorem again — this time to φ', which vanishes at $n+1$ distinct points — and conclude that φ'' has at least n zeros. Apply it again: φ''' has at least $n-1$ zeros." We nod somewhat approvingly; "after $n+1$ applications," Chebyshev continues, "we will conclude that $\varphi^{(n+1)}$ has at least one zero in $(0,1)$. Suppose ξ denotes this lone guaranteed zero of $\varphi^{(n+1)}$."

To compute $\varphi^{(n+1)}$ explicitly, recall the definition:

$$\varphi(t) := \underbrace{f(t) - \mathcal{L}_n[f(t)]}_{= e_n(t)} - \lambda \prod_{k=0}^{n} (t - x_k)$$

Differentiating $n+1$ times: the interpolant $\mathcal{L}_n[f]$ is a polynomial of degree at most n, so its $(n+1)$th derivative vanishes identically. The product $\prod_{k=0}^{n}(t-x_k)$ is a **monic polynomial** of degree $n+1$ — that is, its leading term in the monomial basis is t^{n+1} — so its $(n+1)$th derivative is the *constant value* $(n+1)!$. (Verify this explicitly!) Thus,

$$\varphi^{(n+1)}(t) = f^{(n+1)}(t) - 0 - \lambda \cdot (n+1)!.$$

Substituting in our derived constraint $\varphi^{(n+1)}(\xi) = 0$ and solving for λ:

$$\lambda = \frac{f^{(n+1)}(\xi)}{(n+1)!}.$$

"Now substitute back." Recall that λ was defined so that our chosen $x^\star$ was a root satisfying $\varphi(x^\star) = 0$, which gave us

$$\lambda = \frac{f(x^\star) - \mathcal{L}_n[f(x^\star)]}{\prod_{k=0}^{n}(x^\star - x_k)}.$$

We now have two representations for λ, which we can equate and rearrange:

$$\underbrace{\frac{f^{(n+1)}(\xi)}{(n+1)!} = \frac{f(x^\star) - \mathcal{L}_n[f(x^\star)]}{\prod_{k=0}^{n}(x^\star - x_k)}}_{\text{Both} = \lambda} \implies f(x^\star) - \mathcal{L}_n[f(x^\star)] = \frac{f^{(n+1)}(\xi)}{(n+1)!}\prod_{k=0}^{n}(x^\star - x_k).$$

"But the $x^\star \in [0,1]$ you chose was arbitrary," Chebyshev observes. "It can truly be anything you'd like — that is, we may replace $x^\star$ by any $x \in [0,1]$, understanding that the intermediate point $\xi = \xi(x) \in (0,1)$ may depend on our choice. What I can tell you for certain, then, is the following:"

Theorem 3.3.4 (Interpolation Error)

Suppose f has $n+1$ continuous derivatives on $[a,b]$, and let $\mathcal{L}_n[f]$ denote the Lagrange interpolant at nodes $x_0, x_1, \dots, x_n \in [a,b]$. Then for every $x \in [a,b]$, there exists $\xi_x \in (a,b)$ such that

$$f(x) - \mathcal{L}_n[f(x)] = \frac{f^{(n+1)}(x)}{(n+1)!}\prod_{k=0}^{n}(x - x_k).$$

Chebyshev glares — perhaps we look impatient. He told us this was leading somewhere, but at this point even Weierstrass would seem to be nodding off. "Please stick with me" Chebyshev beckons — "I've been playing Karl's game longer than you've been alive, so do forgive me if I'm a bit terse as I'm trying to catch you up."

Perhaps sensing the building anxiety amongst us and the peanut gallery, Chebyshev pleads: "*Just look* at what the Interpolation Error theorem is telling you. Two factors," he says, pointing to the right hand side formula therein. "The left one,

$$\frac{f^{(n+1)}(\xi)}{(n+1)!},$$

is controlled *entirely* by the function f of Weierstrass's choice. It is, in effect, the $(n+1)$th Taylor coefficient of f — its *regularity* at the $(n+1)$th derivative level, divided by a scaling factor $(n+1)!$."

"The right one,

$$\omega_{n+1}(x) := \prod_{k=0}^{n}(x - x_k),$$

is a polynomial of degree $n+1$ determined *entirely* by the nodes $x_0, x_1, \dots, x_n$. It has nothing whatsoever to do with f." Chebyshev taps the board. "Everything about Weierstrass's chosen function is trapped inside the left factor of my Interpolation Error formula. But let me ask: do you understand *how* we control the right factor, ω_{n+1}?"

We pause and stare at the formula above. Indeed, it may not be entirely clear what Chebyshev's point is if all we're thinking about is *changing basis*. But what we know from the Interpolation

theorem is that this choice of representative basis is immaterial to our ability to approximate Weierstrass's function given only information on the $n+1$ equispaced samples $x_0 = 0, x_1 = 1/n, x_2 = 2/n, \ldots, x_n = 1$ on $[0,1]$. This is to say that *once we've fixed* the $n+1$ data points, *any* basis of Π_{n+1} will yield the same interpolating polynomial.

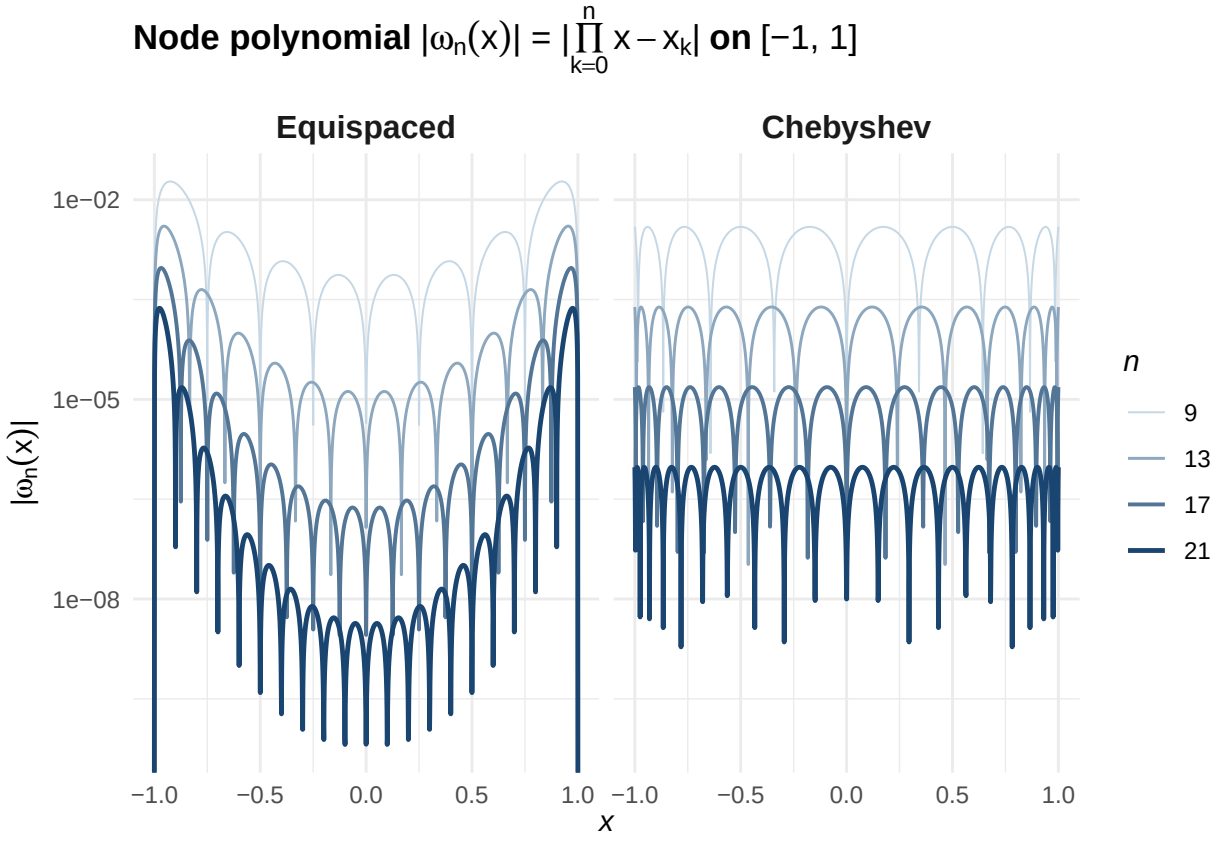
But what Chebyshev is saying is that, really, if we insist that our strategy is based on *interpolating* the points we're given, then Weierstrass has been loading the deck from the start by making them equispaced. "Look at the formula again," continues Chebyshev: "the formula reads

$$f(x) - \mathcal{L}_n[f(x)] = \underbrace{\frac{f^{(n+1)}(\xi)}{(n+1)!}}_{\text{controlled by Weierstrass}} \times \underbrace{\prod_{k=0}^{n}(x - x_k)}_{\omega_{n+1}(x)}.$$

The node polynomial ω_{n+1} is built from the nodes x_k that Weierstrass hands over and nothing else. It is a monic polynomial of degree $n+1$. Its shape, its size, its peaks — all are determined by *where you put the nodes*. It is the one piece of this formula that we haven't tried to touch," he grins, "and I have a proposition, if Karl would care to indulge in a modification to his usual game of three-card monte."

Weierstrass, startled by all this attention, rubs his eyes to think. Finally, he retorts: "It's anything for you Pafnuty," he grins, "as long as I still get to see the nodes you chose *before* I hand you my function."

Chebyshev, a bit surprised by the concession, gestures somewhat excitedly towards the board. "Before," he begins, "Weierstrass handed you $x_k = k/n$, and you accepted. Do you know what ω_{n+1} looks like at equispaced nodes? Let me show you:"

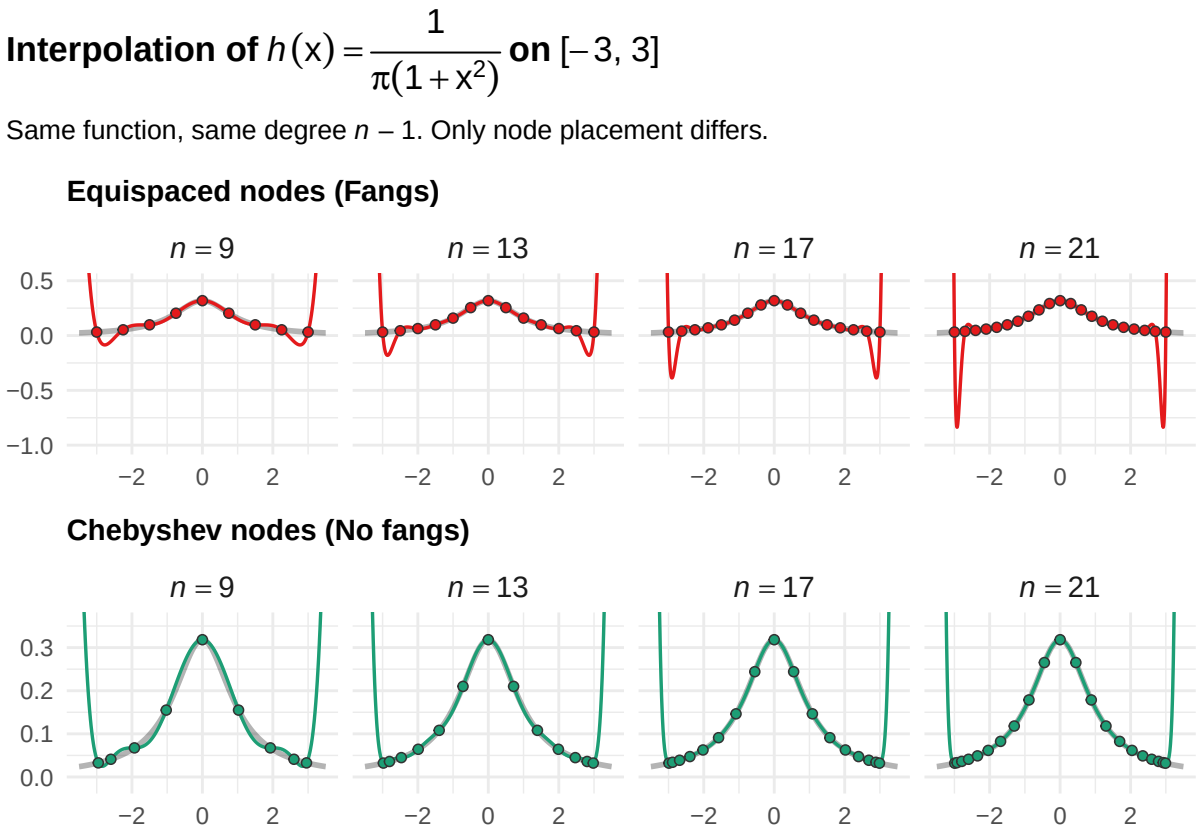


"Without loss of generality," Chebyshev concedes, "I'm working on $[-1, 1]$ — the Chebyshev polynomials live here naturally, and everything transfers to an arbitrary interval $[a, b]$ by the substitution

$$x \mapsto \tfrac{a+b}{2} + \tfrac{b-a}{2} x.$$

With all this in mind, look at the board. What is it telling you? As n grows, *both* panels see $|\omega_{n+1}|$ shrink — but they shrink in *very* different ways. On the equispaced side, the polynomial collapses rapidly further near the middle (the **interior**) of $[-1, 1]$ — you can see the darker curves well below the lighter ones for x near 0 — but the peaks near the **boundary** $x \approx \pm 1$ barely budge. Adding more equispaced nodes helps where the approximation *was already decent*, and does *essentially nothing* where it was catastrophic — recall the Cauchy's fangs, for instance."

Chebyshev taps the right panel. "Now look at mine. The peaks shrink **uniformly** — boundary, interior, everywhere. We control the polynomial *globally*, and we can tame the Cauchy. Take a look at what happens:"



The contrast is immediate. The same function h, the same polynomial degree, the same Lagrange interpolator — and yet the Chebyshev-node interpolants converge visibly toward h as n grows, while the equispaced interpolants tear themselves apart at the boundary. The *only* difference, then, is in the placement of the nodes.

"You see?" Chebyshev perks up. "Your analyst's fangs were *never* intrinsic to the Cauchy density — they were an artifact of the *equispaced nodes* you gave her. The node polynomial ω_{n+1} refused to shrink along the boundary, and that's where the polynomial interpolant *needed the most information* to accurately reproduce the Cauchy's decay. My nodes fix this by clustering near the boundary; precisely where the equispaced nodes leave us most exposed." He pauses.

"And this isn't a trick or heuristic. It is *optimal*; no other function-naïve node placement can beat it."

Speaking on our behalf, suppose we ask whether all of this can be made precise. "Indeed," exclaims a beckoning Chebyshev — "let's return to the board."

Definition 3.3.5 (Chebyshev Polynomial)

A nth **Chebyshev Polynomial** is the function $T_n\colon [-1,1] \to \mathbb{R}$ defined by

$$T_n(x) := \cos(n \arccos(x)), \qquad x \in [-1,1].$$

"Now hang on just a second," we cry — "that is *not* a polynomial!" Chebyshev, exasperated, hangs his head. "They most certainly *are* polynomials: just compute

$$T_0(x) = \cos(0) = 1, \qquad T_1(x) = \cos(\arccos(x)) = x,$$

and if you don't believe me for the higher degrees, use the cosine product-to-sum formula (cf. Theorem 2.2.8 (P1))

$$\cos A \cos B = \frac{\cos(A+B) + \cos(A-B)}{2}$$

applied to $A = \theta$ and $B = n\theta$ and rearrange to obtain

$$\cos((n+1)\theta) = 2\cos(\theta)\cos(n\theta) - \cos((n-1)\theta).$$

Make the substitution $\theta = \arccos x$, so that $x = \cos\theta$, and you obtain

$$\cos((n+1)\arccos x) = 2\underbrace{\cos(\arccos x)}_{=x}\cos(n\arccos x) - \cos((n-1)\arccos x).$$

This looks messy, but it contains *several* terms of our desired form

$$T_k(x) = \cos(k \arccos x);$$

in particular, on switching out the notation, we observe the recurrence

$$T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x).$$

Since $T_0 = 1$ and $T_1 = x$ are polynomials, induction gives that T_n is a polynomial of degree n for every $n \geq 0$. Verify for yourselves:"

$$\begin{aligned}
T_0(x) &= 1, \qquad T_1(x) = x,\\
T_2(x) &= 2xT_1(x) - T_0(x) = 2x^2 - 1,\\
T_3(x) &= 2xT_2(x) - T_1(x) = 4x^3 - 3x,\\
T_4(x) &= 2xT_3(x) - T_2(x) = 8x^4 - 8x^2 + 1,
\end{aligned}$$

where we observe that $T_0 \in \Pi_0 \equiv \mathbb{R}$, $T_1 \in \Pi_1$, and in general, $T_k \in \Pi_k$ the class of polynomials of degree (at most) k.

"Furthermore," Chebyshev adds, "the leading coefficient of T_n is 2^{n-1} for $n \geq 1$ — you can read it off from the recurrence, since each application of $T_{n+1} = 2xT_n - T_{n-1}$ doubles the leading coefficient. So, if you want one of my polynomials to have a leading coefficient of 1 — i.e., if you want the **monic** Chebyshev polynomial of degree n — it is simply $T_n/2^{n-1}$." (Verify this explicitly for a few of the polynomials we computed above; e.g., $T_2/2$ has leading coefficient 1 on the x^2 term.)

"Good," says Chebyshev. "Now, the reason I bring this up is that, originally, I posed the following question: among all monic polynomials of degree $n+1$ on $[-1,1]$, which one has the *smallest possible peak*? That is: I set out to identify

$$\min_{\substack{q \text{ monic} \\ deg\, q = n+1}} \max_{x \in [-1,1]} |q(x)|.$$

What I ended up finding was, of course, that the **minimax** polynomial satisfying the above was the monic Chebyshev polynomial $T_{n+1}/2^n$."

Remark.

Chebyshev has no problem understanding his minimax notation, but especially at a glance, we might find it a bit cumbersome. "It's a minimum, *of* a maximum? How's that matter?" We encourage one to read this notation "from the outside in, then back out, then back in once more." Of course, we elaborate.

First, look to the min and the class it ranges over: here, the monic polynomials q of degree exactly n. So the object we're looking to find is some particular monic $q \in \Pi_n$. *Second*, look to the max: for any fixed candidate q, the expression $\max_{x \in [-1,1]} |q(x)|$ is just a single number — the largest value $|q|$ attains on $[-1,1]$. Different candidates q produce different numbers, and the min asks us to find the candidate whose number is smallest. *Third*, "back out": once we've found the candidate $q^\star$ achieving the minimum, we ask whether we're *certain* no other monic polynomial in Π_n could do better.

To see what this is really asking, let us work at the simplest nontrivial degree: $n = 2$. We're looking for the monic quadratic $q(x) = x^2 + bx + c$ on $[-1,1]$ whose peak $\max_{[-1,1]} |q|$ is as small as possible. An obvious first candidate is $q_1(x) = x^2$:

$$\max_{x \in [-1,1]} |q_1(x)| = \max_{x \in [-1,1]} |x^2| = 1.$$

Can we do better? Notice that q_1 hits its peak at the boundary $x = \pm 1$ and its minimum value of 0 at $x = 0$. The "budget" is lopsided: all the height is at the edges, the interior is wasted. What if we shift the parabola downward to redistribute? Consider

$$q_2(x) = x^2 - \frac{1}{2}.$$

This is still monic and still degree 2. Its values at the key points are

$$q_2(\pm 1) = 1 - \frac{1}{2} = \frac{1}{2}, \qquad q_2(0) = -\frac{1}{2},$$

so that $\max_{[-1,1]} |q_2(x)| = 1/2$. We've *halved* the peak just by subtracting a constant.

Now we "back out": can we do *even* better? What if we shift further — say, $q_3(x) = x^2 - 0.6$? Then $q_3(\pm 1) = 0.4$ (smaller at the boundary), but $q_3(0) = -0.6$ (larger in the interior). The max

is now 0.6, which is *worse*. Shifting up instead — $q_4(x) = x^2 - 0.3$ — gives $q_4(\pm 1) = 0.7$ and $q_4(0) = -0.3$; the max is 0.7, also worse.

What happened with $q_2 = x^2 - 1/2$ is that we chose the constant so that the magnitude at the boundary $|q_2(\pm 1)| = 1/2$ *exactly equals* the magnitude at the interior extremum $|q_2(0)| = 1/2$. The polynomial **equioscillates**: it alternates between $+1/2$ and $-1/2$, and any attempt to lower one peak necessarily raises another. There is no slack left to exploit. This is what makes q_2 optimal.

And indeed: $q_2(x) = x^2 - 1/2 = (2x^2 - 1)/2 = T_2(x)/2$, the monic Chebyshev polynomial of degree 2, with peak exactly $1/2 = 2^{1-2} = 2^{-1}$, validating the theorem for $n = 2$. The minimax result says this equioscillation characterizes the optimum at *every* degree: the monic $T_n/2^{n-1}$ is the unique monic polynomial of degree n that equioscillates between $\pm 2^{1-n}$ on $[-1, 1]$, and any monic polynomial that fails to equioscillate has a strictly larger peak.

"Now," Chebyshev continues", "the optimal nodes. If I want the node polynomial ω_{n+1} — a degree-$(n+1)$ polynomial, mind you — to equal the minimax polynomial in that class — which we now know is $T_{n+1}/2^n$ — then I certainly need its roots to be the same as T_{n+1}. And where does $T_{n+1}(x) = \cos((n+1)\arccos x)$ vanish?" He looks at us expectantly. "A cosine vanishes at odd multiples of $\pi/2$: that is,

$$\cos\alpha = 0 \iff \alpha = \frac{(2k-1)\pi}{2}, \quad k = 1, 2, 3, ...$$

In particular, letting $\alpha = (n+1)\arccos x$ in the above shows that

$$\cos((n+1)\arccos x) = 0 \iff (n+1)\arccos x = \frac{(2k-1)\pi}{2} \iff x = \cos\left(\frac{(2k-1)\pi}{2(n+1)}\right), \quad k \in \mathbb{N}.$$

I believe, then, that I've earned the right to present you with a formal definition:"

Definition 3.3.6 (Chebyshev Nodes)

The **Chebyshev nodes** of order $n+1$ on $[-1, 1]$ are the $n+1$ roots of the Chebyshev polynomial T_{n+1}:

$$x_k = \cos\left(\frac{(2k-1)\pi}{2(n+1)}\right), \qquad k = 1, 2, ..., n+1.$$

Furthermore, on a general interval $[a, b]$, the Chebyshev nodes may be obtained by the affine map

$$x \mapsto \tfrac{a+b}{2} + \tfrac{b-a}{2}x.$$

"Notice where they sit," Chebyshev says, gesturing at the board. "Near $x = \pm 1$, the nodes cluster together; near $x = 0$, they spread apart. This is the opposite of what your intuition wants — you'd think uniform spacing should be 'fairest' — but the clustering is precisely what I just showed you a moment ago in the Cauchy figure a moment ago. What we can imagine is that the equispaced node polynomial refused to shrink near the boundary because there weren't enough nodes to keep the factors $(x - x_k)$ small. My nodes *compensate* for this by placing more nodes

near the edges, so that each factor $(x - x_k)$ 'stays small' where it matters most, and the product $\omega_{n+1} = \prod(x - x_k)$ shrinks uniformly rather than concentrating nearer the interior."

With the definition and minimax theorem now in hand, Chebyshev formally states his guarantee:

Theorem 3.3.7 (Chebyshev Minimax)

Among all monic polynomials of degree n on $[-1, 1]$, the monic Chebyshev polynomial

$$q^\star(x) := \frac{T_n(x)}{2^{n-1}} = \frac{1}{2^{n-1}} \cos(n \arccos x)$$

achieves the smallest possible peak:

$$\min_{\substack{q \text{ monic} \\ \deg q = n}} \max_{x \in [-1,1]} |q(x)| = \max_{x \in [-1,1]} \left| \frac{T_n(x)}{2^{n-1}} \right| = \frac{1}{2^{n-1}},$$

and no other monic polynomial of degree n achieves this value — that is, $q^\star = T_n/2^{n-1}$ is the *unique minimax solution* over the class of degree n polynomials.

"And so," Chebyshev says, "let me finish what I started. Recall my interpolation error formula:

$$f(x) - \mathcal{L}_n[f(x)] = \frac{f^{(n+1)}(\xi_x)}{(n+1)!} \cdot \omega_{n+1}(x).$$

The right factor $\omega_{n+1}(x) = \prod_{k=0}^{n}(x - x_k)$ is a monic polynomial of degree $n+1$ determined entirely by the nodes. At equispaced nodes, this factor grows near the boundary and refuses to shrink no matter how large n becomes — you saw as much in the figure. At *my* nodes, $\omega_{n+1} = T_{n+1}/2^n$, and the minimax theorem gives

$$\max_{x \in [-1,1]} \left| \omega_{n+1}(x) \right| = \frac{1}{2^n}.$$

Exponentially small in n, mind you. So, the interpolation error satisfies

$$\max_{x \in [-1,1]} \left| f(x) - \mathcal{L}_n[f(x)] \right| \leq \frac{\max \left| f^{(n+1)} \right|}{(n+1)!} \cdot \frac{1}{2^n},$$

where I've substituted the evaluation $|f^{(n+1)}(\xi_x)|$ with the maximum $|f^{(n+1)}|$ attains over $[-1, 1]$.

"Now," Chebyshev continues, "let me revisit your two DGPs with this bound in hand. For the Gaussian,

$$g(x) = \frac{1}{\sqrt{\pi}} e^{-x^2},$$

the factor to the left of $1/2^n$ decays like

$$\frac{\max |g^{(n+1)}|}{(n+1)!} \sim O\left(\frac{1}{n!}\right);$$

indeed, we didn't even *need* my nodes for the Gaussian; the factorial decay inherent to the Taylor expansion of the Gaussian kernel e^{-x^2} overcomes any deficits we incur by haphazard node placement."

"On the other hand, for the Cauchy density

$$h(x) = \frac{1}{\pi(1+x^2)},$$

the left factor satisfies

$$\frac{\max|h^{(n+1)}|}{(n+1)!} = O(1);$$

i.e., it *doesn't decay*. We don't get any help from the Cauchy, but it doesn't hurt us that badly — in particular, it doesn't blow up as $n \to \infty$. Under equispaced nodes, we inherit *no additional help*, and the fangs emerge; by instead *strategically placing* our nodes, we're able to overwhelm the constancy of the left factor by an exponentially decaying right factor, which is why

$$\max_{x\in[-1,1]} |h(x) - \mathcal{L}_n[h(x)]| \leq \underbrace{O(1)}_{\text{Cauchy's (ir-)regularity}} \times \underbrace{\frac{1}{2^n}}_{\text{Chebyshev's nodes}} \overset{n\to\infty}{\longrightarrow} 0,$$

so the fangs disappear." Chebyshev taps the interpolation figure once more for emphasis. "Same Lagrange interpolation, same Cauchy density, same polynomial degree. The *only* change is at the nodes, and the convergence is exponential."

He pauses, then somewhat hastily adds: "Really, on your analysts' interval $[-3, 3]$, the affine map stretches the scaling a bit, but the same principle applies. My nodes cluster where the equispaced polynomial was most exposed, and the improved interpolant converges. What you saw in the figure, then, is precisely the figure in action."

Chebyshev allows himself a bow. We agree: he's earned it.

Weierstrass, on the other hand, does not applaud.

"Very nice, Pafnuty," Weierstrass smirks. He reaches into his coat and produces a new function (or, more aptly, *class* of functions, indexed by parameters $a \in (0, 1)$ and $b \in 2\mathbb{Z} + 1$, i.e., the odd integers):

$$W_{a,b}(x) := \sum_{k=0}^{\infty} a^k \cos(b^k \pi x), \qquad 0 < a < 1, \quad b \text{ odd}, \quad ab > 1 + \tfrac{3}{2}\pi.$$

"I've been working on this one," Weierstrass slinks back in his chair, "just for a couple of wise guys like you."

We stare at the board. It is a sum of cosines — each term $a^k \cos(b^k \pi x)$ oscillating at frequency b^k, weighted by the geometrically decaying (since $a \in (0, 1)$) amplitude, a^k.

Weierstrass, sensing our unease, takes his time. "Before I explain what this function *does*," he says, "let me first ensure you that it *exists*. You're looking at an infinite series, and infinite series require justification." He writes:

Theorem 3.3.8 (Weierstrass M-Test)

Let $\{f_k\}_{k=0}^{\infty}$ be a sequence of functions on a common domain $D \subseteq \mathbb{R}$, and suppose there exists a sequence of constants $M_k \geq 0$ such that

$$|f_k(x)| \leq M_k, \quad \text{for all } x \in D, \quad k = 0, 1, 2, \ldots,$$

with

$$\sum_{k=0}^{\infty} M_k < \infty.$$

Then, the series

$$\sum_{k=0}^{\infty} f_k(x) \text{ converges,}$$

and moreover, this convergence is **uniform** on D.

Furthermore, if each f_k is continuous on D, then the limit of the series $\sum_{k=0}^{\infty} f_k$ is continuous on D.

Proof:

Here, we want to show that $\sum f_k$ converges uniformly on D whenever each term f_k is uniformly bounded for $x \in D$; i.e., if $\{M_k\}$ is a sequence of constants such that $|f_k(x)| \leq M_k$ for all $x \in D$ at each $k = 0, 1, 2, ...$, and if the series of the bounding factors $\sum M_k$ converges, then so too does the series $\sum f_k$ whose terms f_k are bounded by the individual M_k.

Recall that saying a series "converges uniformly" precisely means that the partial sums of the series,

$$S_N(x) = \sum_{k=0}^{N} f_k(x),$$

converge uniformly *as a sequence*. And one way to interpret uniform convergence of a sequence S_n is: for every $\epsilon > 0$, there exists a threshold $N \in \mathbb{N}$ such that for all $n \geq N$,

$$\sup_{x \in D} |S_n(x) - \mathcal{S}(x)| < \epsilon,$$

where $\mathcal{S}$ is the limit. The supremum over $x \in D$ is what makes it uniform — N works for *every* x at once.

The only problem, of course, is that we don't have a proposed limit $\mathcal{S}$. Instead, we leverage the Cauchy criterion. Indeed, each pointwise evaluation $x \mapsto S_n(x)$ at a fixed n is just a real number, $S_n(x) \in \mathbb{R}$, and we know that the real line is *complete*. A sequence of real numbers converges if and only if it is *Cauchy*: that is, if for every $\epsilon > 0$, there exists a threshold N such that for all $n > m \geq N$,

$$|S_n(x) - S_m(x)| < \epsilon.$$

Now, notice that $S_n(x) - S_m(x)$ is just the "tail chunk" of the series from term $m + 1$ to term n:

$$S_n(x) - S_m(x) = \sum_{k=m+1}^{n} f_k(x).$$

Bounding this in absolute value,

$$|S_n(x) - S_m(x)| = \left| \sum_{k=m+1}^{n} f_k(x) \right| \leq \sum_{k=m+1}^{n} |f_k(x)| \leq \sum_{k=m+1}^{n} M_k,$$

where the first inequality is the triangle inequality and the second is the hypothesis that $|f_k(x)| \leq M_k$.

Since $\sum_{k=0}^{\infty} M_k < \infty$ by assumption, the tails of the series vanish: given $\epsilon > 0$, we can choose N such that $\sum_{k=N+1}^{\infty} M_k < \epsilon$. Then, for all $n > m \geq N$,

$$|S_n(x) - S_m(x)| \leq \sum_{k=m+1}^{n} M_k \leq \sum_{k=N+1}^{\infty} M_k < \epsilon.$$

The key observation is that the right-hand side, $\sum_{k=N+1}^{\infty} M_k$, *does not depend on the input* $x \in D$. In particular, the same threshold N works for every $x \in D$ simultaneously. So the partial sums are *uniformly* Cauchy on D: for each fixed x, completeness of $\mathbb{R}$ gives convergence of $S_n(x)$ to some limit $\mathcal{S}(x)$, and the x-independence of N guarantees this convergence is uniform on D.

For the continuity claim: suppose each f_k is continuous on D, and fix a point $x_0 \in D$. We want to show the limit $\mathcal{S}$ is continuous at x_0; if we can, that x_0 is arbitrary will establish the continuity of $\mathcal{S}$ on all of D.

So, for any $\epsilon > 0$, choose N large enough so that

$$\sup_{x \in D} |\mathcal{S}(x) - S_N(x)| < \frac{\epsilon}{3}$$

(possible since $S_N \to \mathcal{S}$ uniformly on D.) Since S_N is a finite sum of continuous functions f_k, S_N is continuous at x_0: there exists $\delta > 0$ such that

$$|x - x_0| < \delta \implies |S_N(x) - S_N(x_0)| < \frac{\epsilon}{3}.$$

Now, set $N = N(\epsilon)$ in order for the first sup bound to hold, and then set $\delta = \delta(\epsilon, N)$ so that the second bound holds. Since at this N we have $|\mathcal{S}(x) - S_N(x)| < \epsilon/3$ for all $x \in D$, it holds at both points x and x_0 we set so that $|x - x_0| < \delta$ — in particular, this forces

$$|x - x_0| < \delta \implies |\mathcal{S}(x) - \mathcal{S}(x_0)| \leq \underbrace{|\mathcal{S}(x) - S_N(x)|}_{<\epsilon/3} + \underbrace{|S_N(x) - S_N(x_0)|}_{<\epsilon/3} + \underbrace{|\mathcal{S}(x_0) - S_N(x_0)|}_{<\epsilon/3} < \epsilon.$$

at the given N and δ. Since $x_0 \in D$ was arbitrary, such an N and δ can be set for any $\epsilon > 0$ to force continuity of the limit $\mathcal{S}$ in a neighborhood about any point $x \in D$, and we conclude that $\mathcal{S} = \sum f_k$ is continuous on D whenever the f_k are continuous on D.

∎

"Good," says Weierstrass. "Now let me apply this to my function. Each term in the series defining $W_{a,b}$ is

$$f_k(x) = a^k \cos(b^k \pi x), \quad 0 < a < 1, \quad b \text{ odd}, \quad ab > 1 + \tfrac{3}{2}\pi.$$

Now, each of the cosine contributions has magnitude $|\cos(b^k \pi x)| \leq 1$ uniformly in x, so we may establish the term-wise bound

$$|f_k(x)| = |a^k \cos(b^k \pi x)| \leq a^k := M_k, \quad \forall x \in \mathbb{R}.$$

Since $0 < a < 1$, the bounding constants $M_k = a^k$ form a convergent geometric series:

$$\sum_{k=0}^{\infty} M_k = \sum_{k=0}^{\infty} a^k = \frac{1}{1-a} < \infty.$$

Thus, by the M-test, we conclude

$$W_{a,b}(x) = \sum_{k=0}^{\infty} a^k \cos(b^k \pi x) \text{ converges uniformly in } x \in \mathbb{R}.$$

Furthermore, since each partial sum is a finite sum of cosines — cosines are continuous — the M-test's continuity guarantee ensures continuity: $W_{a,b} \in C(\mathbb{R})$."

Remark.

Right quick, we want to demonstrate what the M-test is protecting us from. As such, consider the sequence

$$f_k(x) = x^k, \quad x \in [0,1].$$

Each f_k is continuous, yet the sequence converges pointwise to the *discontinuous* function

$$f(x) = \begin{cases} 0 & 0 \le x < 1, \\ 1 & x = 1. \end{cases}$$

Why doesn't the M-test apply here? Each f_k is continuous, yes, but we *can not* bound the terms of the sequence by constants whose infinite series converges. Indeed, because

$$\sup_{x \in [0,1]} x^k = 1, \quad \forall k \in \mathbb{N},$$

the task of setting the constants M_k such that

$$M_k \ge |x^k|, \quad \forall x \in [0,1] \iff M_k \ge \sup_{x \in [0,1]} |x^k| = \sup_{x \in [0,1]} |x|^k = 1, \quad k = 0, 1, 2, ...$$

This makes $\sum M_k = \infty$, and we cannot proceed.

"So," continues Weierstrass, "$W_{a,b}$ exists, converges uniformly, and is continuous on all of $\mathbb{R}$. Now, let me explain what makes it special."

He taps the kth term. "Consider the anatomy of $f_k(x) = a^k \cos(b^k \pi x)$. It has two moving parts: an *amplitude* a^k, which decays geometrically in k since $0 < a < 1$, and a *frequency* b^k, which grows geometrically in k since $b \ge 3$. The explicit form of f_k for several k are

$$\begin{aligned} k = 0 &: & f_0(x) &= \cos(\pi x), \\ k = 1 &: & f_1(x) &= a \cos(b \pi x), \\ k = 2 &: & f_2(x) &= a^2 \cos(b^2 \pi x), \end{aligned}$$

and so on. At $k = 0$, f_0 is just a cosine, but at $k = 1$, we observe b oscillations of amplitude a. At $k = 2$, a growing number b^2 of oscillations of shrinking amplitude a^2. At $k = 10$, b^{10} oscillations — and for a particular value, say $b = 13$, this results in roughly 10^{11} oscillations of f_{10} at a tiny amplitude of a^{10}. The function $W_{a,b}$ is the superposition of *all of these simultaneously*: oscillation riding on faster oscillation, ad infinitum."

"The question," Weierstrass continues, "is which force wins. The decay of the amplitudes a_k wants to make $W_{a,b}$ smooth — if the high-frequency terms $\cos(b^k \pi x)$ were negligible, the function

would inherit the smoothness of its low-frequency terms, and you could differentiate term-by-term without consequence. Yet the growth of the frequencies b^k wants to make $W_{a,b}$ rough — each successive cosine oscillates b times faster than the last, introducing structure at finer and finer scale."

"Oh," he pauses — "and I suppose I haven't told you the best part. Have you tried to differentiate $W_{a,b}$?"

We haven't, but Weierstrass is already at the board. "Suppose, for the sake of argument, that you *could* differentiate $W_{a,b}$ term by term. The derivative of the kth term is

$$f_k'(x) = -\pi(ab)^k \sin(b^k \pi x),$$

which has amplitude $\pi(ab)^k$. Now, we force $ab > 1 + \frac{3}{2}\pi$ by hypothesis, so the derivative amplitudes $(ab)^k$ *grow geometrically* in k. The series of derivatives,

$$\sum_{k=0}^{\infty} f_k'(x) = -\pi \sum_{k=0}^{\infty} (ab)^k \sin(b^k \pi x),$$

has terms whose amplitudes $(ab)^k \to \infty$. The M-test, which saved us a moment ago, now tells us *nothing* — the bounding constants $M_k = \pi(ab)^k$ form a *divergent* geometric series."

"Thus," he lingers, "you simply *cannot* justify differentiating f_k under the sum. But this failure is not merely technical — it is not just that we lack the tools to *justify* the differentiation, it is that the derivative *genuinely does not exist*."

Remark.

The proof that $W_{a,b}'(x)$ fails to exist at *every* $x \in \mathbb{R}$ is more delicate than the convergence argument, and we will not reproduce it — the interested reader may consult Hardy's 1916 treatment, or the proof linked here. What we *can* say, however, is that the intuition is exactly as described: the amplitudes a^k decay fast enough to guarantee convergence of $\sum f_k$ (continuity), but not fast enough to guarantee convergence of $\sum f_k'$ (non-differentiability). The amplitudes a^k and frequencies b^k are in a tug of war, and the condition $ab > 1 + \frac{3}{2}\pi$ ensures that the frequencies "win at the level of derivatives," while "losing at the level of the function itself."

"There is just one more thing I'd like you to observe about my function before we return to the game," Weierstrass presses. "You know that $W_{a,b}$ is continuous everywhere, differentiable nowhere, and built from cosines. But look at what happens when we rearrange my series:

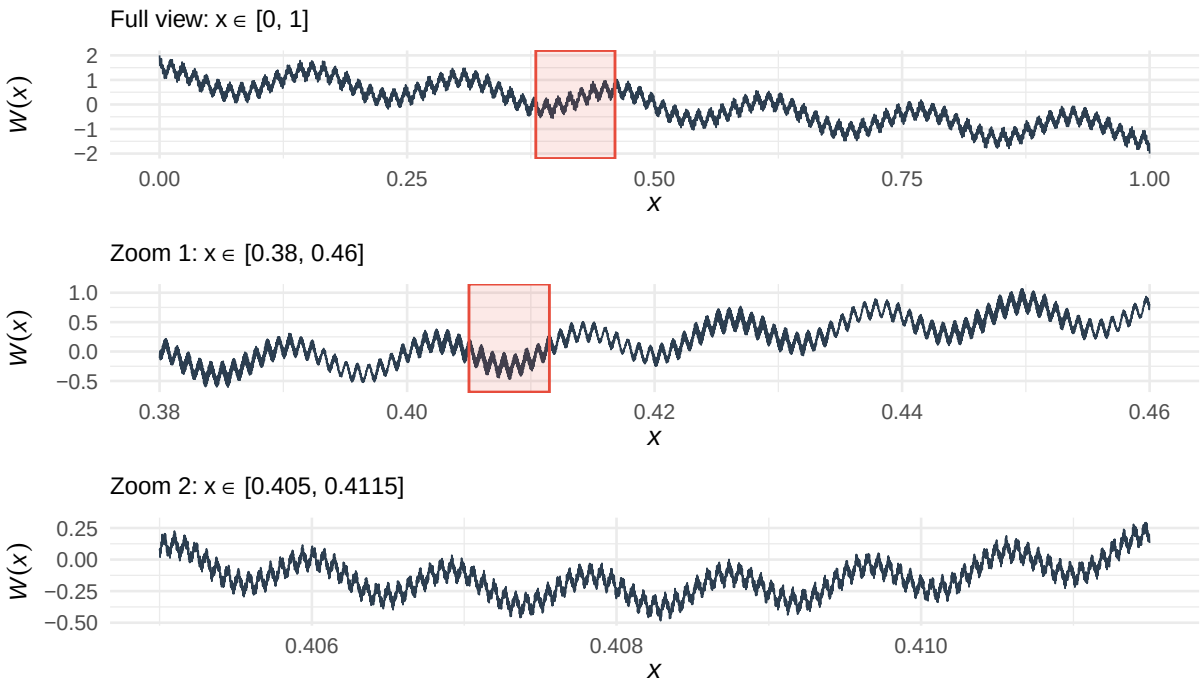
$$W_{a,b}(x) = \cos(\pi x) + a \sum_{k=0}^{\infty} a^k \cos(b^{k+1} \pi x) = \cos(\pi x) + a W_{a,b}(bx).$$

What we can observe in

$$W_{a,b}(x) = \cos(\pi x) + a W_{a,b}(bx)$$

is that $W_{a,b}$ satisfies a **functional equation**: up to a single smooth cosine, $W_{a,b}(x)$ is just a rescaled copy of itself, evaluated at b times the input $x \mapsto W_{a,b}(bx)$, and scaled by a in amplitude."

"Visually," Weierstrass puffs, now vigorously scrawling at the white board, "you might imagine this means the plot of my function looks something like this:"



Weierstrass wipes away the sweat from his brow. "So you see," he manages, "*If* you zoom in on any patch of the graph by a factor of b horizontally and stretch by $1/a$ vertically, and you see *the same function* — not approximately, but *exactly*." He taps the board. "This is the **self-similarity** of $W_{a,b}$. There is no scale at which the function 'looks smooth.' The roughness you see at one magnification reappears, identically, at the next. It is not a surface feature that resolves itself should we 'zoom in enough'; it *is* the function."

Weierstrass constructed $W_{a,b}$ in 1872, thirteen years before the approximation theorem bearing his name. At the time, many mathematicians — Ampère among them — had conjectured that every continuous function must be differentiable "almost everywhere," or at least at "most points." Weierstrass's construction demolished this belief: $W_{a,b}$ is differentiable at *no* points, and it is not pathological or artificial — it is a sum of cosines, among the most natural building blocks in all of analysis.

We now return to the game. Recalling Chebyshev's interpolation error formula reads

$$f(x) - \mathcal{L}_n[f(x)] = \frac{f^{(n+1)}(\xi_x)}{(n+1)!} \cdot \omega_{n+1}(x),$$

the left factor requires $f^{(n+1)}(\xi)$. But $W_{a,b}$ has no first derivative, let alone an $(n+1)$th. The expression $W_{a,b}^{(n+1)}(\xi)$ is not merely large — it is *undefined*. This is not a situation where we can "manipulate ω_{n+1} to overwhelm the left factor," since it simply doesn't exist in this case. The error formula simply does not apply.

The entire framework Chebyshev developed — the decomposition into "DGP regularity" and "node geometry," the minimax optimization, the Chebyshev nodes — requires that f have enough derivatives for the formula to *parse*. For $W_{a,b}$, it does not parse. The error formula is a *conditional*

guarantee: *if* f is "smooth enough," *then* Chebyshev's nodes give exponential convergence. $W_{a,b}$ does not satisfy this condition, so choosing better nodes can't provide enough help to force anything better.

But, to be clear, this is not just a failure of Chebyshev's brand of interpolation — *any* method whose convergence guarantees depend on a smooth target f will fail for $W_{a,b}$ for precisely the same reason: $W_{a,b}$ isn't smooth."

And yet, we emphasize that the Weierstrass Approximation Theorem *still applies* to $W_{a,b}$! Indeed, $W_{a,b}$ is continuous on $[0,1]$, so for every $\epsilon > 0$, a polynomial p with $\|W_{a,b} - p\|_\infty < \epsilon$ is guaranteed by the theorem. Again, the polynomial is guaranteed, but Chebyshev's tools cannot find it — not because the tools themselves are flawed, but because their hypotheses are not met by our proposed target $W_{a,b}$.

If it wasn't clear by this point, we've *lost again*. It is unfortunate: even though Chebyshev helped us beat the Cauchy, Weierstrass's weapon $W_{a,b}$ was simply too wild for the same tools to correct.

Intermission

Chebyshev, a bit dismayed by the whole affair, stares at the board. "Hold on just a moment," he mutters to himself. He scrawls for a moment, turning red in the face — "in fact," he says, more slowly now, as though working it out for himself, "suppose we started with my own polynomials $T_n(x) = \cos(n \arccos x)$). The substitution $x = \cos\theta$ turns this into

$$T_n(x) = \cos(n \arccos x) \implies T_n(\theta) = \cos(n\theta).$$

And that," his expression drops, "is a Fourier cosine function."

He paces for a minute before walking back to the board. "And I can even take this further. Suppose we had a polynomial p expressed in *my* basis — that is, as a linear combination of Chebyshev polynomials

$$p(x) = \sum_{k=0}^{n} c_k T_k(x), \qquad x \in [-1,1].$$

This is a polynomial of degree n on $[-1,1]$. Make the substitution $x = \cos\theta$, so that $\theta = \arccos x$ and θ ranges over $[0,\pi]$ as x ranges over $[-1,1]$. Under this substitution, every Chebyshev polynomial becomes a cosine:

$$T_k(x) = T_k(\cos\theta) = \cos(k\theta).$$

So, the polynomial p, re-expressed as a function of $\theta \in [0,\pi]$, becomes

$$p(\cos\theta) = \sum_{k=0}^{n} c_k \cos(k\theta), \qquad \theta \in [0,\pi].$$

Chebyshev stops and stares at what he's written. He compares

$$W_{a,b}(x) = \sum_{k=0}^{\infty} a^k \cos(b^k \pi x)$$

with his own

$$p(\cos\theta) = \sum_{k=0}^{n} c_k \cos(k\theta).$$

"The coefficients c_k in my Chebyshev expansion," he says slowly, "are the Fourier cosine coefficients of the function $\theta \mapsto p(\cos\theta)$ on $[0,\pi]$. These are not analogous. They are not 'similar in spirit.' They are the *same computation* in different coordinates."

He turns to look at Weierstrass. "Karl, the tool that defeated me — your $W_{a,b}$ — is a sum of cosines. The tool I used to fight back — my T_n — *is* a cosine. The Fourier cosine basis on $[0,\pi]$ and the Chebyshev polynomial basis on $[-1,1]$ are the same basis, connected by $x = \cos\theta$. I have been doing Fourier analysis this entire time. I just didn't know it."

Nobody says anything for a moment. Then Chebyshev sits down.

Remark.

We linger on this connection between Chebyshev polynomials and Fourier series, because we think the reader should too. What Chebyshev has just observed is not a curiosity or a notational coincidence. It is a structural identity: polynomial approximation on $[-1,1]$ in the Chebyshev basis *is* Fourier cosine approximation on $[0,\pi]$, full stop. The change of variables $x = \cos\theta$ converts one into the other, and the conversion is exact — no approximation, no loss, no residual. Everything the reader learned about Fourier series in Chapter 2 — orthogonality, convergence rates, the Dirichlet and Fejér kernels, the role of smoothness in determining coefficient decay — has a polynomial translation on $[-1,1]$, and vice versa.

But then this observation raises a question that, once noticed, is difficult to set aside. Indeed, we recall a remark we made near the statement of Weierstrass's theorem — there, we hearkened *all* the way back to chapter 2, where we *already proved* that continuous functions f on compact intervals $[a,b]$ (here, you could rescale the function affinely such that it is defined on $[-\pi,\pi]$) can be uniformly approximated — not by polynomials, but by the **Cesàro means** of their Fourier series:

$$\sigma_N f(x) = \frac{1}{N+1}\sum_{j=0}^{N} S_j f(x),$$

where

$$S_j f(x) = \frac{a_0}{2} + \sum_{k=1}^{j}[a_k\cos(kx) + b_k\sin(kx)],$$

$$a_0 = \frac{1}{\pi}\int_{\pi}^{\pi} f(x)\,dx, \qquad a_k = \frac{1}{\pi}\int_{-\pi}^{\pi} f(x)\cos(kx)\,dx, \qquad b_k = \frac{1}{\pi}\int_{-\pi}^{\pi} f(x)\sin(kx)\,dx.$$

Fejér's theorem (Theorem 2.3.10) guarantees that $\sigma_N f \rightrightarrows f$ on $[a,b]$ — i.e., that $\sigma_N f$ converges uniformly to f on $[a,b]$, which is precisely the same as saying

$$\|\sigma_N f - f\|_\infty \to 0 \quad \text{as} \quad N \to \infty.$$

or, stated more formally, that given $\epsilon > 0$, we can choose $N = N(\epsilon) \in \mathbb{N}$ based *purely* on ϵ such that

$$n \geq N \implies \|\sigma_N f - f\|_\infty = \sup_{x\in[a,b]} \left|\sigma_n[f(x)] - f(x)\right| < \epsilon$$

To be clear — what this the Cesàro means satisfy *almost everything* we're looking for in a candidate to beat Weierstrass at his own game. Indeed, they *must* be able to beat $W_{a,b}$

So we have to ask: if Fejér already solved the problem back in Chapter 2, why are we talking about it again?

The answer is subtle, but that subtlety is, our view, the single most underserved sentiment in all of mathematics: it concerns the *complexity* of the approximating objects. It is, quite fundamentally, the same distinction we made in Chapter 1: these Cesàro means σ_N are comprised of a finite number of Fourier partial sums S_n, each of which are comprised of sums of sines and cosines $\sin(nx)$ and $\cos(nx)$ weighted by integrals of evaluations of our function f weighted by sines and cosines in the coefficients a_k and b_k.

All of this might sound like a mouthful. And maybe it should — because at this point, we *haven't even remembered what a cosine is.* Again, forget the calculator — how would Euclid have computed a Fourier series?

He couldn't have. Because $\cos(kx)$ is *not* a finite arithmetic object. It is an infinite power series:

$$\cos(kx) = 1 - \frac{(kx)^2}{2!} + \frac{(kx)^4}{4!} - \frac{(kx)^6}{6!} + \cdots = \sum_{m=0}^{\infty} \frac{(-1)^m (kx)^{2m}}{(2m)!}.$$

Every cosine in $\sigma_N f$ — and there are finitely many of them, one for each harmonic $k = 0, 1, \ldots, N$ — *hides* an infinite series. But look at the mth term of the cosine Taylor series:

$$\underbrace{\frac{(-1)^m k^{2m}}{(2m)!}}_{A_m} x^{2m} := A_m x^{2m}.$$

This connects back to Chapter 2, where we observed that $\cos(\cdot)$ is an **even** function — indeed, we observe that the Taylor expansion contains only members of the form

$$\mathcal{C}_m := \{1, x^2, x^4, \ldots, x^{2m}\}.$$

What we might intuitively imagine, then, is that when we add in the analogous $\sin(\cdot)$ terms,

$$\sin(kx) = x - \frac{(kx)^3}{3!} + \frac{(kx)^5}{5!} - \frac{(kx)^7}{7!} + \cdots = \sum_{m=0}^{\infty} \frac{(-1)^m (kx)^{2m+1}}{(2m+1)!},$$

and, again looking to the mth term, setting

$$\underbrace{\frac{(-1)^m k^{2m+1}}{(2m+1)!}}_{B_m} x^{2m+1} := B_m x^{2m+1},$$

that what we observe is the *odd* monomials filling in:

$$\mathcal{S}_m := \{x, x^3, x^5, \ldots, x^{2m+1}\}.$$

Together, $\mathcal{C}_m \cup \mathcal{S}_m = \{1, x, x^2, x^3, x^4, \ldots\}$ — but this is just the monomial basis $\mathcal{M}$. The cosines, expanded in powers of x, contribute only the even monomials x^0, x^2, x^4 ...; the sines contribute only the odd monomials x^1, x^3, x^5 ... Neither family alone spans the space of polynomials. You need *both* the fill out the monomial basis — and this is the *reason* a Fourier series "requires both," and why Cesàro means are then able to approximate *any* continuous function — not *just* the even or odd ones.

The even/odd decomposition of f in Chapter 2 — which, at the time, seemed like a *symmetry property* of trigonometric series — is, viewed through the monomial lens, a statement about *which*

powers of x each family of transcendental functions harbors in the representation *we actually compute*.

So, what we observe in composite is that, when we *really* compute

$$\sigma_N f(x) = \frac{a_0}{2} + \sum_{k=1}^{N} \underbrace{\left(1 - \frac{k}{N+1}\right)}_{:= w_k} \Big[a_k \cos(kx) + b_k \sin(kx)\Big],$$

— where the

$$w_k := 1 - \frac{k}{N+1} = \frac{(N+1)-k}{N+1}, \qquad k = 1, \ldots, N,$$

are the Cesàro weights that taper the Fourier coefficients toward zero — we are *really* computing

$$\begin{aligned}\sigma_N f(x) &= \frac{a_0}{2} + \sum_{k=1}^{N} w_k \left[a_k \sum_{m=0}^{\infty} \frac{(-1)^m k^{2m}}{(2m)!} x^{2m} \;+\; b_k \sum_{m=0}^{\infty} \frac{(-1)^m k^{2m+1}}{(2m+1)!} x^{2m+1}\right] \\ &= \frac{a_0}{2} + \sum_{k=1}^{N} w_k \sum_{m=0}^{\infty} (-1)^m \left[\frac{a_k k^{2m}}{(2m)!}\, x^{2m} \;+\; \frac{b_k k^{2m+1}}{(2m+1)!}\, x^{2m+1}\right].\end{aligned}$$

The outer sum over k is finite (only N terms). The inner sum over m is infinite (each cosine and sine unpacks into infinitely many monomials). If we now swap the order of summation — collecting by monomial degree rather than by frequency — we obtain

$$\begin{aligned}\sigma_N f(x) &= \frac{a_0}{2} \;+\; \sum_{m=0}^{\infty} (-1)^m \underbrace{\left[\sum_{k=1}^{N} \frac{w_k\, a_k\, k^{2m}}{(2m)!}\right]}_{\text{even coefficient } \alpha_{2m}} x^{2m} \;+\; \sum_{m=0}^{\infty} (-1)^m \underbrace{\left[\sum_{k=1}^{N} \frac{w_k\, b_k\, k^{2m+1}}{(2m+1)!}\right]}_{\text{odd coefficient } \alpha_{2m+1}} x^{2m+1} \\ &= \sum_{\ell=0}^{\infty} \alpha_\ell\, x^\ell,\end{aligned}$$

where the coefficient α_ℓ at each monomial degree ℓ is a finite sum (over $k = 1, \ldots, N$) of Fourier coefficients weighted by powers of the frequencies:

$$\alpha_\ell = \begin{cases} \dfrac{a_0}{2} + \displaystyle\sum_{k=1}^{N} \frac{(-1)^m\, w_k\, a_k\, k^{2m}}{(2m)!} & \ell = 2m \text{ (even)}, \\[2ex] \displaystyle\sum_{k=1}^{N} \frac{(-1)^m\, w_k\, b_k\, k^{2m+1}}{(2m+1)!} & \ell = 2m+1 \text{ (odd)}. \end{cases}$$

We encourage you to now entirely discard the explicit forms of the α_ℓ (while perhaps choosing to remember that each is computable, in the Chapter 1 "sense of Euclid"), and instead look to the general form of

$$\sigma_N f(x) = \sum_{\ell=0}^{\infty} \alpha_\ell\, x^\ell$$

The re-indexing in terms of ℓ might read like notational slight of hand, but it isn't — all we did was swap a finite sum (over *frequencies* $k = 1, \ldots, N$) past an infinite sum (over *monomial degrees* $m = 0, 1, 2, \ldots$), and this is always when the outer sum is finite. What the re-indexing *reveals*, however, is that the Cesàro mean was "never really comprised N functions" in the sense

Weierstrass demands — it was always a single infinite power series $\sum_{\ell=0}^{\infty} \alpha_\ell x^\ell$, whose coefficients α_ℓ are finite, computable numbers determined by the Fourier data. The trigonometric representation fragmented this series into "N parallel channels organized by frequency"; the monomial representation "reassembles it into one infinite series organized by degree."

And now the crucial observation: since $\sigma_N f \rightrightarrows f$ *anyway*, even in spite of the fact that it was "always infinite," what we can deduce is that the *tail* of the infinite series $\sum \alpha_\ell x^\ell$ is negligible. Indeed: for any $\epsilon > 0$, there exists a finite M such that

$$\left\| \sum_{\ell=0}^{\infty} \alpha_\ell x^\ell - \sum_{\ell=0}^{M} \alpha_\ell x^\ell \right\|_\infty = \left\| \sum_{\ell=M+1}^{\infty} \alpha_\ell x^\ell \right\|_\infty < \epsilon.$$

The truncated series

$$p_M(x) := \sum_{\ell=0}^{M} \alpha_\ell x^\ell$$

is a polynomial — an element of Π_M, finite in the Euclidean sense — and it satisfies

$$\|f - p_M\|_\infty \leq \underbrace{\|f - \sigma_N f\|_\infty}_{<\epsilon \text{ (Fejér)}} + \underbrace{\|\sigma_N f - p_M\|_\infty}_{<\epsilon \text{ (tail truncation)}} < 2\epsilon.$$

The whole of the peanut gallery seem to let out a collective gasp as we scrawl in our last ϵ. Funnily enough, we don't even appear to realize what we've done — but Chebyshev, seemingly revived from his stuper, leaps straight to the board.

"Do you realize what you've just done?" Chebyshev says, barely containing himself. "You've *proved it*. The Weierstrass Approximation Theorem. You just proved it — right there, on the board — using Fejér's theorem from Chapter 2 and nothing else."

We stare at our own work. He's right.

- Step 1: Fejér guarantees $\|f - \sigma_N f\|_\infty < \epsilon$.
- Step 2: the Cesàro mean is a power series whose tail is negligible, so truncation at degree M costs at most ϵ.
- Step 3: the triangle inequality. The polynomial $p_M \in \Pi_M$ exists and uniformly approximates f to within 2ϵ.

That's Weierstrass's theorem in a nutshell. No really — we just proved it.

Weierstrass, who has been watching from his chair, begins to applaud — slowly, and with a smile that suggests he is not entirely surprised. "Congratulations," he says. "So you've found a polynomial. Now, why don't you just tell try and me: what *is* it?"

We look at our polynomial. Its coefficients are:

$$\alpha_\ell = \sum_{k=1}^{N} \frac{(\text{Cesàro weight})_k \times (\text{Fourier coefficient})_k \times k^\ell}{\ell!}.$$

Suppose we're wondering what the degree M of the polynomial p_M we found is. Immediately, we realize that we *don't* know — it depends on how fast the tail decays, which depends on N, which depends on ϵ, and the relationship between M, N, and ϵ is some function we never bothered to compute.

And the notation we used above is deliberately quite illusory — but, in practice, it matters *what* the coefficients α_ℓ are explicitly. So, what are they? Finite sums of Fourier integrals weighted by factorial-suppressed frequency powers — computable in principle, yet completely unilluminating in practice.

Is the polynomial a convex combination of the data values? This mattered for the whole probabilistic interpretation of the Fejér kernel — it was part of why it was "intuitively appealing" to us at the time. And again, the answer is a resounding "No" — there is no reason the polynomial should be convex, and in general it isn't.

Can our "Cesàro polynomial p_M" overshoot the range of f? Certainly. Does it have any structural properties at all? None that we can see.

And, perhaps most damning of all for p_M: the coefficients α_ℓ are built from the Fourier coefficients a_k and b_k, which are *integrals*:

$$a_k = \frac{1}{\pi}\int_{-\pi}^{\pi} f(x)\cos(kx)\,dx, \qquad b_k = \frac{1}{\pi}\int_{-\pi}^{\pi} f(x)\sin(kx)\,dx.$$

Weierstrass, on the other hand — he handed us *point evaluations*: $f(0), f(1/n), f(2/n), \dots, f(1)$. He *explicitly did not* hand us the ability to integrate f against cosines over $[-\pi, \pi]$. Our proof establishes that the polynomial exists, yes — but the polynomial itself requires information about f that the game never provided. We proved the theorem while violating the rules.

Remark.

The reader who was "only interested" in our game to obtain a constructive proof of Weierstrass's theorem may, thus, feel free to leave. Yet we would encourage you to continue — because we believe the "proof as given" is scarcely illuminating and, moreover, perhaps even obstructive. We do not *feel* the theorem yet, and that is what we aim to do by the end here.

Chebyshev, sensing our deflation, sits back down. "Don't be too hard on yourselves," he says quietly. "You've done something real. The theorem is proved. The polynomial *exists*. But" — and here he pauses — "existence is not construction, and the polynomial you found is, if I may be frank, terrible. It has no closed form. It has no structure. It requires global information about f — integrals against cosines — that no finite set of point evaluations can provide. And it inherits nothing about the geometry of f: it is not a convex combination of anything, it has no reason to stay within the range of f, and its rate of convergence is buried under two layers of uncontrolled approximation."

He looks at the table on the board.

	Ch. 2 (Fourier Series)	**Ch. 3 (Polynomials)**
"Hard" method	Dirichlet partial sums $S_N f$	Lagrange interpolation $\mathcal{L}_n[f]$
"Soft" method	Cesàro means $\sigma_N f$	???
Complexity	Finite × infinite	Finite × finite

"You've shown that the bottom-right entry *exists*," Chebyshev says, "by extracting it from the bottom-left. But the extraction destroyed everything that made the Cesàro means beautiful: the nonnegativity, the convexity, the closed form, the probabilistic interpretation and, perhaps most of all, the clean proof. What's sitting in that bottom-right cell *is* a polynomial, yes — but it

is a polynomial discovered by hiring an orchestra, and then sending everyone home except the pianist." He pauses. "Perhaps it is better," he ponder, "to walk directly to the piano."

We agree. The question has shifted. It is no longer "does a uniformly approximating polynomial exist?" — we have just answered that, and the answer is yes. The question is now: can someone hand us a polynomial that is *good*? One with a closed form, built from the equispaced data $f(0), f(1/n), \dots, f(1)$ and nothing else, whose weights are nonneg, whose convergence proof uses only continuity, and whose structure reveals *why* it works rather than merely *that* it works?

Perhaps we open the floor to questions — our peanut gallery is, after all, not short on talent. Fejér, whose theorem we just used as the load-bearing wall of our proof, shifts uncomfortably in his seat. Runge, whose own name adorns the phenomenon that started this whole mess, winces sympathetically. Faber, who could have told Chebyshev from the start that no node sequence would work for every continuous function, excuses himself entirely.

Yet, none of them can tell us what to build. Fejér's construction was the one we just gutted for parts. Runge identified the disease but not the cure. Faber proved the impossibility but offered no alternative. And Chebyshev — Chebyshev gave us the Fourier connection that made the whole argument possible, and he will, as we are about to discover, give the next man through the door the single most important tool in his proof. But the construction itself is not his to make.

So perhaps we'd have given in and accepted the ugly polynomial as the best we could do; at least, had we not acknowledged a chime from a man who, perhaps unrightfully, we imagine was sat somewhere near the back.

"May I?"

Round 3

We turn. Sergei Natanovich Bernstein has been watching from the back since the beginning.

Bernstein was born in 1880 into a Jewish family in Odessa — the same Russian Empire that Chebyshev (1821–1894) knew his entire life. His mother sent him to Paris, where he studied at the Sorbonne and, in 1904, at the age of twenty-four, solved Hilbert's 19th problem on the analytic nature of solutions to elliptic partial differential equations. It was the kind of debut that should have opened every door in European mathematics.

It did not open the doors in Russia. The Russian Empire did not recognize foreign doctoral degrees for the purpose of university appointment, so Bernstein — having already solved one of Hilbert's twenty-three problems — was required to defend a *second* doctoral thesis at Kharkov in 1913, on the very subject we are now discussing: the approximation of continuous functions by polynomials. The work that concerns us here, published in a two-page paper in the *Communications of the Kharkov Mathematical Society* in 1912, sits between these two dissertations. It is, by any reasonable measure, among the most consequential two pages in the history of analysis.

The years that followed were not kind. Bernstein axiomatized probability theory in 1917 — the first such axiomatization, predating Kolmogorov's measure-theoretic foundations by sixteen years — but his algebraic approach was eventually superseded. He survived the political purges that swept Soviet academia in the 1930s, was forced from Kharkov to the Academy of Sciences in Leningrad, and fled to Kazakhstan during the siege of Leningrad in 1941. He returned after the war, spent his remaining decades at the Steklov Institute in Moscow, and devoted the final years of his life to a labor that, given our narrative, borders on poetic: editing and annotating the collected works of Chebyshev.

The man whose inequality Bernstein is about to borrow spent his last years in Bernstein's care. We imagine the incarnation before us now is the 1912 version: thirty-two years old, eight years past the Sorbonne, still young enough to see something in the equispaced data that everyone else

has overlooked. His training is in partial differential equations, but his instincts — as we are about to discover — are those of a probabilist. He does not introduce himself with a theorem or a formula. He asks a question.

"Give me back the equispaced data."

We — and Chebyshev, and Weierstrass himself — stare. The *equispaced* data? The data from Round 1? The data that Chebyshev just spent the entirety of Round 2 explaining was the problem?

"Yes," Bernstein says. "The evaluations $f(0), f(1/n), f(2/n), \ldots, f(1)$. I won't be needing different nodes." He pauses, then turns to Chebyshev. "In fact — Pafnuty, if I may — I'd like to borrow your inequality."

Chebyshev, bewildered, nods.

3.4 Exercises

Guided integral derivations of Cauchy and Gaussian integrals

Complex Partial fractions on the Cauchy integral

Asymptotic approximations (e.g. Cauchy's first few terms aren't bad)

Divided differences interpretation in different bases

Guided Proof of the Polynomial Factor Lemma

Division/Euclid Algorithm

Fourier basis treatment here since we couldnt with our dummy data (maybe contrast with what happens when you don't have equispaced nodes "what goes wrong on our dummy data")

Padé approximants fixing the Cauchy approximation

Bernstein polynomials/Weierstrass

Chebyshev nodes, Lebesgue constant

Aliasing / polynomial indistinguishavility

Condition number on Vandermonde

Different kinds of numeric integration rules arise directly from interpolating function under Lagrange basis (e.g. at $n = 2$ you get trapezoidal at $n = 3$ Simpson)

Gram-Schmidt on $\mathcal{D}_3$